Ichi Kanaya: Haskell のノート 1.0-Prerelease

目次

1 Haskell 入門 7
1.1 Haskell について 8
1.1.1 Haskell という森 8
1.1.2 関数型プログラミング 9
1.1.3 Haskell コンパイラの準備 11
1.1.4 余談：本書の構成 13
1.1.5 この章のまとめ 13
1.2 カリー風な書き方 14
1.2.1 変数と関数 14
1.2.2 ラムダ式 16
1.2.3 パタンマッチとガード 17
1.2.4 余談：条件式 18
1.2.5 この章のまとめ 18
1.3 さらにカリー風な書き方 20
1.3.1 演算子 20
1.3.2 関数合成と関数適用 21
1.3.3 高階関数 22
1.3.4 余談：IO サバイバルキット 1 23
1.3.5 この章のまとめ 24
1.4 型 26
1.4.1 データ型 26
1.4.2 カリー化 27
1.4.3 多相型と型クラス 29
1.4.4 余談：全称量化子 30
1.4.5 この章のまとめ 30
1.5 リスト 32
1.5.1 リスト 32
1.5.2 畳み込み 35
1.5.3 マップ 36
1.5.4 余談：IO サバイバルキット 2 37
1.5.5 この章のまとめ 39
1.6 再帰 40
1.6.1 関数の再帰適用 40
1.6.2 リストと再帰 41
1.6.3 末尾再帰 43
1.6.4 余談：遅延評価 44
1.6.5 この章のまとめ 45
1.7 Maybe 46
1.7.1 Possibly 46
1.7.2 Maybe 47
1.7.3 リストと Maybe 50
1.7.4 余談: Either 51
1.7.5 この章のまとめ 51
1.8 関手 53
1.8.1 リストの世界・Maybe の世界 53
1.8.2 関手と関手則 55

1.8.3 アプリカティブ関手 57
1.8.4 余談：型をつくる 59
1.8.5 この章のまとめ 61
1.9 モナド 63
1.9.1 バインド演算子 63
1.9.2 モナド則 66
1.9.3 破壊的代入を隠すモナド 67
1.9.4 余談：IO サバイバルキット 3 69
1.9.5 この章のまとめ 73
1.10 IO 74
1.10.1 IO モナド 74
1.10.2 疑似乱数 76
1.10.3 main アクション 77
1.10.4 余談：do 記法 78
1.10.5 この章のまとめ 82
2 Haskell プログラミング 84
2.1 Hello, again. 85
2.2 データ型 86
2.3 文字列・ベクトル 87
2.4 末尾再帰 88
2.5 Y コンビネータ 89
2.6 main の引数と戻り値 90
2.7 継続 91
2.8 状態 92
2.9 テンプレート 93
2.10 他の言語との比較 94
3 プログラミングと代数構造 95
3.1 モノイド 96
3.2 圏論 97
3.3 関数型 98
3.4 関手・アプリカティブ関手 99
3.5 モナド 100
3.6 非決定生 101
3.7 カテゴリ 102
3.8 アロー 103
3.9 クライスリカテゴリ 104
3.10 圏論とプログラミングをつなぐ 105
4 執筆メモ 106
4.0.1 モノイド 106
4.1 直積と直和 108
5 Haskell の深い部分 109
5.1 関数・関手・アプリカティブ関手・モナド 110
5.1.1 関数と関手 110
5.1.2 クライスリスター 111
5.2 型と種 112
5.2.1 型の計算 112
5.2.2 種 112

5.3 マクロと Template Haskell 113
6 執筆ノート 114
6.0.1 数学における圏と関手 114
6.0.2 Haskell における関手型クラスとそのインスタンス 116
6.0.3 Haskell におけるバインド演算子とモナド型クラスのインスタンス 116
6.0.4 memo 117
6.0.5 関数型クラス 117
6.0.6 class and instance 117
参考文献 118
索引 119

種類	字体・表記法	例
変数・関数	イタリック（1 文字）	x, f
アクション	ギリシア文字	φ
有名な変数・関数・定数	ローマン・小文字	$\mathrm{first}, \mathrm{id}, \mathrm{otherwise}$
有名なアクション	ボールド・小文字	$\mathbf{main}, \mathbf{print}$
リスト変数	変数名に s をつける	x_{s}
Maybe 変数	変数名に ? をつける	$x_{?}$
Either 変数	変数名に ! をつける	$x_{!}$
一般のコンテナ変数	変数名に $*$ をつける	x_{*}
値コンストラクタ（引数なし）	ローマン（大文字）	$\mathrm{True}, \mathrm{Nothing}$
値コンストラクタ（引数あり）	ローマン（大文字）	Just_x
有名な値コンストラクタ 1	数学記号	$\varnothing, \emptyset$
有名な値コンストラクタ 2	特別な括弧で包む	$[x], \langle y \rangle, (\!\!(u, v)\!\!)$
有名な値コンストラクタ 3	コロン (:) を含む記号	$x : x_{\mathrm{s}}$
型（引数なし）	ボールドイタリック（1 文字）	$\boldsymbol{a}$
型（引数あり）	ボールドイタリック（1 文字）	$\boldsymbol{m_a}$
有名な型（引数なし）	ボールド・大文字	$\mathbf{Int}$
有名な型（引数あり）	特別な括弧で包む	$[\boldsymbol{a}], [\mathbf{Int}]$
ユニット型	括弧	$(\!\!)$
型クラス	サンセリフ	Num
カインド	星，フラクチュール	$\star, \mathfrak{Type}, \mathfrak{K}$
キーワード	固定幅	`let`
無名パラメタ	ひし形（白）	$\lozenge$
無名型パラメタ	ひし形（黒）	$\blacklozenge$

表 1: 凡例 1

種類	字体・表記法	例
集合（数学）	ブラックボード（1 文字）	$\mathbb{Z}$
圏論的関手（数学）	スクリプト（1 文字）	$\mathscr{F}$
圏（数学）	カリグラフィ（1 文字）	$\mathcal{C}$

表 2: 凡例 2

1 Haskell 入門

本書はプログラミング言語 Haskell の入門書である．それと同時に，本書はプログラミング言語を用いた代数構造の入門書でもある．プログラミングと代数構造の間には密接な関係があるが, 特に関数型プログラミングを実践する時にはその関係を意識する必要が出てくる. 本書はその両者を同時に解説することを試みる.

1.1 Haskell について

これからのプログラマにとって Haskell を無視することはできない．Haskell の「欠点をあげつらうことも，攻撃することもできるが，無視することだけはできない」のだ．それは Haskell がプログラミングの本質に深く関わっているからである．[1]

1.1.1 Haskell という森

Haskell というプログラミング言語を知ろうとすると，従来のプログラミング言語の知識が邪魔をする．モダンで，人気があって，Haskell から影響を受けた言語，たとえば Ruby や Swift の知識さえ，Haskell を学ぶ障害になり得る．ではどのようにして Haskell の深みに到達すればいいのだろうか．

その答えは，一見遠回りに見えるが，一度抽象数学の高みに登ることである．[2], [3]

と言っても，あわてる必要はない．

近代的なプログラミング言語を知っていれば，すでにある程度抽象数学に足を踏み入れているからである．そこで，本書では近代的なプログラマを対象に，プログラミング言語を登山口に抽象数学の山を登り，その高みから Haskell という森を見下ろすことにする．

さて，登山口にどのプログラミング言語を選ぶのが適当であろうか．TIOBE Index 2025 年 12 月版によると「ビッグ 5」として Python，C，C++，Java，C# が挙げられている．[4]

順位の変動はあるが，他の調査でもビッグ 5 は過去何年も変動していないので，当座は妥当な統計であろう．このうち C は「多くのプログラマが読める」以外にメリットが無く，その唯一のメリットさえ最近は怪しくなっているため，登山口候補から外す．残るは C++，Java，C# グループと Python ということになるが，シンプルであり，かつ Haskell と対極にある言語である Python を登山口に選ぶことにした．

本書では Python コードはこのように登場する．

```python
# Python
print("Hello, world.")
```

本書に示すコードは擬似コードではなく，すべて実行可能な本物のコードである．

一部の章でどうしても型に触れないといけない部分がある．Python は動的型付け言語であり，型の説明には不適切であるため，この部分だけ理解の助けとして C++によるコードを例示した．この部分はコードを読まなくても先に進める．

ところで，プログラムのソースコードは現代でも ASCII 文字セットの範囲で書くことが標準的である．Unicode を利用したり，まして文字にカラーを指定したり，書体や装飾を指定することは一般的ではない．たとえば変数 `a` のことを a と書いたり $\boldsymbol{a}$ と書いたり $\check{a}$ と書いたりして区別することはない．

Haskell プログラマもまた，多くの異なる概念を同じ貧弱な文字セットで表現しなければならない．これは，はじめて Haskell コードを読むときに大きな問題になりえる．たとえば Haskell では `[a]` という表記をよく扱う．この `[a]` は `a` という変数 1 要素からなるリストのこともあるし，`a` 型という仮の型から作ったリスト型の場合もあるが，字面からでは判断できない．もし変数はイタリック体，型はボールドイタリック体と決まっていれば，それぞれ $[a]$ および $[\boldsymbol{a}]$ と区別できたところである．

本書は，異なる性質のものには異なる書体を割り当てるようにしている．ただし，どの表現もいつでも Haskell に翻訳できるように配慮している．実際，本書執筆の最大の困難点は，数学的に妥当で，かつ Haskell の記法とも矛盾しない記法を見つけることであった．

1.1.2 関数型プログラミング

プログラマはなぜ Haskell を習得しなければならないのだろう．それは Haskell と**関数型プログラミング**の間に密接な関係があるからである．（本書でこのように**ハイライト**する用語は，巻末の索引に登録されている．）

関数型プログラミングとはプログラミングにおける一種のスローガンのようなもので，どの言語を用いたから関数型でどの言語を用いたから関数型ではない，というものではない．しかし，関数型プログラミングを強くサポートする言語と，そうでない言語とがある．この事情はオブジェクト指向プログラミングとプログラミング言語の関係と似ている．Haskell は関数型プログラミングを強くサポートし，Python はほとんどサポートしない．

関数型プログラミングの特徴を一言で言えば，プログラム中の**破壊的代入**を禁止することである．変数 x に 1 という数値が一度代入されたら，変数 x の値をプログラム中に書き換える，すなわち破壊的代入をすることはできない．この結果，変数の値はプログラムのどこでも，どの時点で読み出しても同じであることが保証される．これを変数の**参照透過性**と呼ぶ．

プログラム全体に参照透過性があると，そのプログラムはブロックに分割しやすく，各々のブロックは再利用しやすい．またプログラムのどの断片から読み始めても，全体の構造を見失いにくい．これが関数型プログラミングとそれを強くサポートする Haskell を習得する理由である．

参照透過性がもたらすもう一つのボーナスは変数の**遅延評価**である．変数はいつ評価しても値が変わらないのだから，コンパイラは変数をできるだけ遅く評価してよい．この遅延評価によって，Haskell コンパイラは他の言語に見られない**無限リスト**を扱う能力を獲得している．

ここで数学とプログラミングの関係について述べておこう．ある方程式を解くためにコンピュータによって数値シミュレーションを行うとか，非常に複雑な微分を機械的に行うとか，プログラミングによって数学をサポートすることは計算機科学の主たる分野の一つであるが，ここではもっと根源的な話をする．

数学者もプログラマも**関数**をよく使う．数学者が使う関数とは，**引数**がいくつかあって，その結果決まる**戻り値**があるようなものだ．一方でプログラマが使う関数というのは，引数と戻り値があるという点はだいたい同じとして，中身に**条件分岐**があったり，**ループ**があったり，変数を書き換えたり，**入出力**をしたりする．

どちらも同じ関数であるのに，なぜこうもイメージが違うのだろうか．

もし，我々が関数型プログラミングの原則を忠実に守り，プログラム中のいかなる破壊的代入をも禁止するとすると，両者の関数は全く同じ性格になる．**逐次実行**も条件分岐もループも，それどころか**定数**さえ，**ラムダ式**という式だけで書けるようになる．あらゆるプログラムが，最終的には単一のラムダ式で書ける．

ところが，入出力，状態変数，例外など，プログラミングに使われる多くのテクニックは関数の副作用を前提としている．参照透過性と副作用を統一的に扱うためには**モナド**という概念が必要である．Haskell はモナドを陽に扱うプログラミング言語である．

♣

関数型プログラミングという考え方自体は，プログラミングに必ずしも束縛されるものではない．

例えばいま，ある英文テキストファイルから，単語を出現頻度順に取り出したいとしよう．こんなとき，プログラマならば次のように考えるのではないだろうか．

「まずは英文テキスト中の大文字を全部小文字にしよう」「それからアルファベットと空白以外を全部捨ててしまおう」「空白を改行に置き換えると，1 行に 1 単語になるな」「ここまで来たら出現頻度順に並べ直すのはわけないことだ」……

これらの言葉を UNIX の言葉に置き換えてみる．ここでは英文テキストを `the-great-gatsby.txt` としておこう．中身はフィッツジェラルドの名作「The Great Gatsby」（華麗なるギャッツビー）だ．最初の「まずは英文テキスト中の大文字を全部小文字にしよう」は **UNIX コマンド**の `tr` を使って次のように実現できる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]'
```

この出力を適当なファイルにリダイレクトすれば，大文字がすべて小文字に置き換わったテキストファイルになる．

次にするのは「それからアルファベットと空白以外を全部捨ててしまおう」だ．これも UNIX コマンド `tr` を使って次のように実現できる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]'
```

これで単語だけのテキストファイルが出来る．次に空白をすべて改行に置き換えてしまおう．もう一度 `tr` を使う．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
```

これで 1 行に 1 単語ずつ並べられたテキストファイルがこれで手に入ることになる．空行もあるが，それは問題ない．このファイルの中身をアルファベット順にソートする．それには `sort` コマンドを使う．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort
```

ソートできれば，同一の連続する行を数え上げて，一つの行にすればいい．それには `uniq` コマンドを `-c` オプション付きで使う．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c
```

最後に，出現頻度順に逆順ソートをかけると出現頻度順になる．それには `sort` コマンドに `-n` と `-r` オプションを付けて以下のようにする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c | sort -n -r
```

この出力の先頭 10 行は，そのまま出現頻度上位 10 傑の単語となる．

♣

さて，上述の出現頻度上位単語の抽出のどこが「関数型プログラミング」の考え方だったのだろうか．

実はこんな風に考えることが出来る．入力を x とする．いまの例では x がファイル `the-great-gatsby.txt` である．出力を y とする．ここに y は出現頻度順に並んだ出現頻度と単語のリストである．我々が欲しいのは

$$y = f(x) \tag{1}$$

となるようなプログラム f だった．プログラム f は簡単には手に入らないので，我々は UNIX コマンドの `tr` とか `sort` とか `uniq` とかをつなぎ合わせて作った．それらを「数学風」に書くと次のようになる．

$$\begin{aligned} f_1 &= \mathrm{tr}_{[A\ldots Z]\to[a\ldots z]} \\ f_2 &= \mathrm{tr}_{\overline{[a\ldots z,\square]}\to\emptyset} \\ f_3 &= \mathrm{tr}_{\square\to\swarrow} \\ f_4 &= \mathrm{sort} \\ f_5 &= \mathrm{uniq}_c \\ f_6 &= \mathrm{sort}_{n,r} \end{aligned} \tag{2}$$

そして x から y を得るために

$$y = f_6(f_5(f_4(f_3(f_2(f_1(x)))))) \tag{3}$$

という計算を行った．括弧が多すぎるので，この式を

$$y = (f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1)(x) \tag{4}$$

と書き直そう．ここに演算子 $\circ$ は**関数合成演算子**だ．

関数合成演算子を使うと，プログラム f を

$$f = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1 \tag{5}$$

と定義することも出来る．これが何を意味しているかと言うと，プログラム f を小さな部分プログラム $f_1, f_2, f_3, f_4, f_5, f_6$ に分解したということだ．

分解したなら，合成する方法が必要になる．この例では，プログラムの各部分が他の部分に依存していないために，合成は数学的な関数合成と同じ方法が使える．

上記の例のように，数学における関数合成をそのままプログラミングに持ち込めれば，部分プログラムの合成も見通しが良いものになる．このような考え方が関数型プログラミングのエッセンスなのである．[5], [6], [7], [8], [9]

1.1.3 Haskell コンパイラの準備

Haskell はコンパイラ言語であり，またほとんどの OS で標準言語という位置づけでもないため，Haskell コンパイラを自身の OS にインストールする必要がある．Linux, macOS, Windows では Haskell Stack という Haskell 言語環境をインストールするのが簡単である．

macOS では，以下のようにインストールすることを推奨する．

まず Homebrew というパッケージマネージャを macOS にインストールする．それには，コマンドシェルから次のようにする．

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Homebrew がインストールできたら，続けて Haskell Stack を次のコマンドでインストールする．

```
$ brew install haskell-stack
```

♣

Windows (Intel)の場合は https://docs.haskellstack.org/en/stable/ から Windows 版インストーラをダウンロードできる．

♣

Haskell Stack をインストールしたら，次のコマンドで新規プロジェクトを作成する．

```
$ stack new myProject
```

自分の関数は `myProject/src/Lib.hs` に書かれている以下のソースコードのうち `someFunc` の定義を書き換えることになる．

```haskell
-- Lib.hs
module Lib
    ( someFunc
    ) where

someFunc :: IO ()
someFunc = putStrLn "someFunc"
```

あるいは `myProject/app/Main.hs` に書かれている `main` の定義を直接書き換えても良い．`Main.hs` は次のようなファイルである．

```haskell
-- Main.hs
module Main (main) where
import Lib

main :: IO ()
main = someFunc
```

計算機科学の習慣に従って “Hello, World!” と表示するプログラムを書いておこう．それには `Lib.hs` を書き換えて次のようにする．

```haskell
-- Lib.hs
module Lib
    ( someFunc
    ) where

someFunc :: IO ()
someFunc = putStrLn "Hello, World!"
```

`Lib.hs`（または `Main.hs`）を書き換えた後は次のコマンドで実行する．最初のビルド時には，必要な Haskell コンパイラやライブラリがダウンロードされるので少々の時間がかかる．

```
$ stack build
$ stack run
```

画面に次のように表示されたら成功である．

```
Hello, World!
```

1.1.4 余談：本書の構成

本書の構成を「余談」として記述する．[1]

本書の第 1 部は抽象数学からプログラミングを俯瞰することにする．俯瞰と言っても，登場する数式はすべて Haskell に翻訳可能である．第 1 部はまず本書で使う記法「カリー風な書き方」を紹介する．またプログラミングの基本要素である条件分岐についても説明する．その後，型と型クラスの基本的な概念を説明し，リストの扱いを説明する．またリストを用いた再帰，畳み込み，マップを説明する．第 1 部の後半では Maybe を紹介し，リストと Maybe の共通点から関手型クラスという Haskell に独特な概念を説明する．ついで関手型クラスを拡張したアプリカティブ関手型クラス，モナド型クラスを紹介し，入出力（IO）をモナド型クラスで扱う方法を示す．第 1 部では途中途中に「IO サバイバルキット」と称して，プログラマにとってどうしても必要となる知識を「とりあえず使える形」で紹介した．

1.1.5 この章のまとめ

- プログラミング言語 Haskell は関数型プログラミングスタイルを強くサポートする．
- 関数型プログラミングとは，数学における関数と関数合成の概念をプログラミングに持ち込んだものである．
- 関数型プログラミングでは参照透過性が重視される．参照透過性とは，変数が破壊的に代入されないこと，関数が同じ引数を与えられた場合には同じ値を返すことを言う．
- プログラミング言語 Haskell を学ぶには，従来のプログラミング言語の知識から一旦離れて，抽象数学の高みに登るほうが近道である．
- 関数型プログラミングのエッセンスはプログラミングのみならず，日常的な UNIX コマンドにも見て取れる．

関数

日本語の「関数」は中国語の「函数（hánshù）」に由来する．教科書によっては日本でも関数のことを函数と書いている．この中国語の函数であるが，箱（函）に入れられた数という意味と，英語のファンクション（function)とと発音が似ているため訳語として採用されたという説がある．

なお英語のファンクションのほうには日本語の「関数」の意味と「機能」の意味の両方がある．C 言語の関数（function）はどちらかと言えば「機能」の意味である．

違う概念には違う単語を割り当てたいものである．

[1] なお本書は脚注も多用している．脚注は読まなくて良い．

1.2 カリー風な書き方

本書では一般の数学書やプログラミングの教科書からは少し異なった記法を用いる．ある概念が発明されてからずっと後になって正しい記法が見つかり，それがきっかけとなって正しく理解されるという現象は歴史上よくあることである．本書でも様々な新しい記号，記法を導入する．

数学やプログラミング言語には書き方に一定の決まりがある．この章ではまず「カリー風の」数式記述方式を見てみることにする．カリー風というのは，数学者ハスケル・カリーから名前を借りた言い方で，筆者が勝手に命名したものだ．

カリー風の書き方は数学の教科書やプログラミングの教科書で見かけるものとは若干違うが，圧倒的にシンプルで Haskell との親和性も高く，慣れてくると非常に読みやすいものなので，本書でも全面的に採用する．

1.2.1 変数と関数

変数 x に値 1 を代入するには次のようにする．[2]

$$x = 1 \tag{6}$$

変数という呼び名に反して，変数の値は一度代入したら変えられない．そこで変数に値を代入するとは呼ばずに，変数に値を**束縛**するという． 式 6 の右辺のように数式にハードコードされた値を**リテラル**と呼ぶ．

リテラルや変数には**型**がある．型は数学者の**集合**と似た意味で，整数全体の集合 $\mathbb{Z}$ に相当する**整数型**や，実数全体の集合 $\mathbb{R}$ に相当する**浮動小数点数型**がある．整数と整数型，実数と浮動小数点型は異なるため，整数型を **Int** で，浮動小数点型を **Float** と **Double** で表すことにする．ここで **Float** と **Double** の違いは有効数字だけである．[3]

型については節 1.4 で詳しく述べる．数学者は変数 x が整数であることを $x \in \mathbb{Z}$ と書くが，本書では $x :: \mathbf{Int}$ と書く．これは記号 $\in$ を別の用途に用いるためである．また型の**型注釈**と束縛をまとめて $x :: \mathbf{Int} = 1$ と書く事もできる．[4]

本書では変数名を原則 1 文字として，イタリック体で表し a および w, x, y, z のような n 以降のアルファベットを使う．

♣

変数の値がいつでも変化しないことを**参照透過性**と呼ぶ．また，後述する関数についても，同じ引数を与えた場合，同じ値がいつも返ることも参照透過性の条件である．

プログラマが変数の値を変化させたい，つまり**破壊的代入**を行いたい理由は**ユーザ入力**，**ループ**，**例外**，**内部状態**の変化，**大域ジャンプ**，**継続**を扱いたいからであろう．しかし，後に見るようにループ，例外，大域ジャンプに相当する操作に変数の破壊的代入は必要ない．ユーザ入力，内部状態の変化，継続に関しても章を改めて取り上げる．参照透過性を強くサポートするプログラミング言語をを**関数型プログラミング言語**と呼ぶ．

♣

整数 x に 1 を足す**関数** f は次のように定義できる．

$$fx = 1 + x \tag{7}$$

[2] Haskell では `x = 1` と書く．
[3] Haskell では `Int` および `Float` と `Double` と書く．
[4] Haskell では $x :: \mathbf{Int}$ を `x :: Int` と書き $x :: \mathbf{Int} = 1$ を `x :: Int = 1` と書く．

ここに x は関数 f の**引数**である．引数は括弧でくるまない．このような書き方を本書では「カリー風」と呼ぶことにする．[5]

なお同じ関数を Python では次のように書くことになる．

```
# Python
def f(x):
  return 1 + x
```

Python や一般的な数学書では引数 x をとる関数 f を $f(x)$ と書くが，括弧は冗長なので今後は fx と書くことにする．[6]

本書では関数名を原則 1 文字として，イタリック体で表し，f, g, h のようにアルファベットの f 以降の文字を使う．ただし有名な関数についてはローマン体で表し，文字数も 2 文字以上とする．たとえば sin などの三角関数や指数関数がそれにあたる．

関数 f に引数 xを「食わせる」ことを**適用**と呼ぶ．もし fx と書いてあったら，それは f と x の積，つまり $f \times x$ ではなく，従来の $f(x)$ すなわち関数 f に引数 x を与えているものと解釈する．高校生向けの数学書でも $\sin x$ のように三角関数に限っては括弧を取って書くことになっているので，まるで馴染みがないということもないだろう．なお，関数はいつも引数の左側に書くことにする．これを「関数 f が変数 x の左から作用する」と言い，また関数 f のことを**左作用素**とも呼ぶ．

変数 x に関数 f を適用して z を得る場合は次のように書く．[7]

$$z = fx \tag{8}$$

♣

複数引数をとる関数を Python や一般的な数学の教科書では $g(x, y)$ と書くが，これも括弧が冗長なので今後は gxy と書く．この場合式 gxy は左を優先して結合する．つまり $gxy = (gx)y$ である．これは引数 y に関数 (gx) が左から作用していると解釈する．関数 (gx) は引数 x に関数 g を作用させて作った関数である．引数に「飢えた」関数 (gx) を**部分適用**された関数と呼ぶ．このように式の左側を優先的に演算していくことを**左結合**と呼ぶ．Haskell の場合，関数適用はいつも左結合である．

部分適用の例を見てみよう．例えばふたつの引数のうち大きい方を返す関数 max は $\max xy$ として使われるが，関数適用は左結合であるから $(\max x)y$ としても同じである．そこで $(\max x)$ だけ取り出すと，これは「引数が x よりも小さければ x を，そうでなければ引数を返す関数」とみなすことができる．[8]

もし関数 $\max x$ と同じものを Python で作りたければ，次のようなテクニックを用いることになる．

```
# Python
def create_max(x):
```

[5] Haskell では `f x = x + 1` と書く．

[6] Haskell では関数 `f` に引数 `x` を適用させることを `f x` と書く．数学や物理学では x をパラメタとする関数を $f(x)$ と書く場合もあるし，f のようにパラメタを省略する場合もある．数学や物理学でパラメタを省略した場合は，$f(x_0)$ の意味で $f|_{x=x_0}$ と書くことがある．

[7] Haskell では `z = f x` と書く．

[8] Haskell では $\max xy$ を `max x y` と書く．なお関数 $(\max x)$ のことを $\max_x$ と書く教科書も多い．関数引数を添え字で表す記法は，本書でも後に採用する．

```
    return lambda y: max(x, y)
```

この Python 関数 `create_max(x)` は我々の $\max x$ と同じ用に，1 引数を取る関数として振る舞う．そのため `create_max(10)(20)` は `20` を返す．このような仕組みは**クロージャ**または関数閉包と呼ばれる．

1.2.2 ラムダ式

関数の正体は**ラムダ式**である．ラムダ式とは，仮の引数をとり，その値をもとになにがしかの演算を行い，その結果を返す式である．ラムダ式は名前のない関数のようなものだ．それゆえ，**無名関数**と呼ばれることもある．

例えば引数 x をとり値 $1+x$ を返すラムダ式を変数 f に代入したい場合，Python では次のように書く．

```
# Python
f = lambda x: 1 + x
```

一方，我々はより簡潔に

$$f = \backslash\, x \mapsto 1 + x \tag{9}$$

と書くことにする．この式の右辺 $\backslash\, x \mapsto 1+x$ は多くの書物で $\lambda x\,.\,1+x$ と記述されるところである．しかし我々はすべてのギリシア文字を変数名のために予約しておきたいのと，ピリオド記号 (.) が今後登場する二項演算子と紛らわしいため，上述の記法を用いる．[9]

ラムダ式は関数である．ラムダ式を適用するには，ラムダ式を括弧で包む必要がある．例を挙げる．

$$(\backslash\, x \mapsto 1 + x)\ 2 \tag{10}$$

この式は結果として 3 を返す．

複数引数をとるラムダ式は例えば $\backslash\, xy \mapsto x + y$ のように引数を並べて書く．

♣

本書では新たに，次のラムダ式記法も導入する．式中に記号 $\diamond$ が現れた場合，その式全体がラムダ式であるとみなす．記号 $\diamond$ の部分には引数が入る．第 n 番目の $\diamond$ には第 n 番目の引数が入る．例えばラムダ式 $\backslash\, xy \mapsto x + y$ は $\diamond + \diamond$ と書いても良い．式を左から読んで 1 番目の $\diamond$ が元々の x すなわち第 1 引数を，2 番目の $\diamond$ が元々の y すなわち第 2 引数を意味する．この省略記法はプログラミング言語 Scheme における `cut` プロシジャに由来する．[10]

♣

数式が長く続くとき，読みやすさのために**局所変数**を導入すると便利である．例えば

$$z = f(1 + x) \tag{11}$$

という式のうち，先に $1+x$ を計算して x' のように名前をつけておきたいこともある．そんなときは次のように書く．[11]

$$z = \texttt{let}\ x' \triangleq 1 + x\ \texttt{in}\ fx' \tag{12}$$

[9] Haskell ではラムダ式 $\backslash\, x \mapsto 1+x$ を `\x -> 1 + x` と書く．
[10] Haskell では，中置演算子に限ってこの表現が使える．例えば $(\diamond + \diamond)$ は単に `(+)` と表現できる．
[11] Haskell では `z = let x' = 1+x in f x'` と書く．

なお，局所変数を後ろに回して

$$z = fx' \ \texttt{where}\ x' \triangleq 1 + x \tag{13}$$

と書いても良い．[12]

局所変数はラムダ式を用いたシンタックスシュガーである．式 12 は次の式と等価である．

$$z = (\backslash\, x' \mapsto fx')(1 + x) \tag{14}$$

♣

関数の定義は，基本的にはラムダ式の変数への束縛である．引数 x をとり値 $1 + x$ を返す関数 f は

$$f = \backslash\, x \mapsto 1 + x \tag{15}$$

と定義できる．ただし，この省略形として

$$fx = 1 + x \tag{16}$$

と書いても良い．[13]

1.2.3 パタンマッチとガード

関数に**スペシャルバージョン**がある場合はそれらを列挙する．例えば引数が 0 の場合は特別に戻り値が -1 であり，その他の場合は $1 + x$ を返す関数 f を考える．このとき f は以下のように定義できる．これを関数の**パタンマッチ**と呼ぶ．[14]

$$\begin{aligned} f0 &= -1 \\ fx &= 1 + x \end{aligned} \tag{17}$$

関数のパタンマッチは，関数の内部に書いても良い．関数内部にパタンマッチを書きたい場合は次のように書く．

$$fx = \texttt{case}\ x\ \texttt{of}\ \begin{cases} 0 \dashrightarrow -1 \\ \square \dashrightarrow 1 + x \end{cases} \tag{18}$$

ここに $\square$ は任意の値の意味である．パタンマッチは上から順番にマッチングしていくため，この場合は 0 以外を意味する．[15]

式 18 は次のような Python コードと等しい．[16]

```python
# Python
def f(x):
  match x:
    case 0:
      return -1
    case _:
      return 1 + x
```

一部のプログラミング言語では**デフォルト引数**という，引数を省略できるメカニズムがあ

[12] Haskell では `y = f x' where x' = 1 + x` と書く．
[13] Haskell では $f = \backslash\, x \mapsto 1 + x$ を `f = \ x -> 1 + x` と書き，一方 $fx = 1 + x$ を `f x = 1 + x` と書く．
[14] Haskell では `f 0 = -1; f x = 1 + x` と書く．
[15] Haskell では `f x = case x of { 0 -> 100; _ -> 1 + x }` と書く．
[16] この構文「構造的パタンマッチング」と呼ばれ Python 3.10 で導入された．

るが，我々は引数をいつも省略しないことにする．[17]

関数定義にパタンマッチではなく**場合分け**が必要な場合は**ガード**を用いる．例えば引数の値が負の場合は 0 を，0 の場合は -1 を，それ以外の場合は $1+x$ を返す関数 f' は以下のように定義する．[18]

$$\begin{aligned} f'x|_{x<0} &= 0 \\ |_{x\equiv 0} &= -1 \\ |_{\text{otherwise}} &= 1+x \end{aligned} \tag{19}$$

ガードは上から順にマッチされる．

1.2.4 余談：条件式

関数定義の場合分けを駆使すれば**条件式**はなくても構わないが，条件式の記法があるのは便利である．Python には次のような**制御構造**としての条件文がある．

```python
# Python
def f(x):
  if x == 0.0:
    return 1.0
  else:
    return sin(x) / x
```

一方，我々は値を持つ**条件式**を考える．我々の条件式とは

$$fx = \texttt{if}\ x \equiv 0.0\ \texttt{then}\ 1.0\ \texttt{else}\ \frac{\sin x}{x} \tag{20}$$

のように `if` 節，`then` 節，及び `else` 節からなるものであって，`then` 節も `else` 節も省略できないものとする．`if` 節の式の値が真 (True) であれば `then` 節の式が評価され，偽 (False) であれば `else` 節の式が評価される．我々の条件式は C における条件演算子（三項演算子）と等しく見えるが，Haskell の場合は遅延評価が行われるため，結果として条件式の**短絡評価**が行われる点が異なる．[19]

1.2.5 この章のまとめ

- 変数への値の代入は $x=1$ のように書く．一度代入された変数は値を変えない．代入は正しくは束縛と呼ぶ．
- 関数定義は $fx = 1+x$ のように書く．関数の引数に括弧はつけない．
- 関数適用は $z = fx$ のように書く．この場合も関数の引数に括弧は付けない．
- 2 引数関数 f' に引数 x, y を与えるときは $z' = f'xy$ のように書く．関数適用は左結合するので $f'xy = (f'x)y$ である．引数に「飢えた」関数 $f'x$ を部分適用された関数と呼ぶ．
- 関数の正体はラムダ式である．関数 $fx = 1+x$ はラムダ式を使った記法 $f = \backslash x \mapsto 1+x$ の略記である．
- 本書では無名パラメタ ($\diamond$) を用いたラムダ式も用いる．無名パラメタを使う場合は $f = 1+\diamond$ のように書く．
- 式には局所変数を導入できる．例えば $z = f(1+x)$ は $z = \texttt{let}\ x' \triangleq 1+x\ \texttt{in}\ fx'$ のように書いても良く，また $z = fx'\ \texttt{where}\ x' \triangleq 1+x$ のように書いても良い．

---

[17] Haskell にもデフォルト引数はない．

[18] Haskell では `f' x | x < 0 = 0  | x == 0 = 100  | otherwise = x + 1` と書く．もっとも，この記法を使う場合は改行を適度に入れたほうが読みやすい．

[19] Haskell では $fx = \texttt{if}\ x \equiv 0.0\ \texttt{then}\ 1.0\ \texttt{else}\ {}^{(\sin x)}/_{x}$ を `f x = if x == 0.0 then 1.0 else (sin x) / x` と書く．

- 関数定義にはスペシャルバージョンを含めることが出来る. 例えば引数が 0 の場合は特別な値を返すときは $f0 = -1$ のように書く.
- 関数定義にはガードを含めることが出来る．例えば引数が負の場合は特別な値を返すときは $f|_{x<0} = 0$ のように書く.
- 式にはパタンマッチを導入できる.
- 条件式は `if` p `then` x `else` y のように書く．もし p が True であれば x が式の値になり，そうでなければ y が式の値になる.

ラムダの理由

我々のラムダ式 $\backslash\, x \mapsto x + 1$ は, かつては $\hat{x}.x + 1$ のように書かれていた. その後 $\Lambda x.x + 1$ に変化し $\lambda x.x + 1$ のように変化したらしい．多くの教科書はこの λ を使う記法を採用している．$\hat{x}$ が Λx に変化したのは形が似ているからである．Haskell が $\backslash$ を使うのは，その形が λ と似ているからである.

近代的なプログラミング言語はラムダ式を陽に記述できる．C++11 では `](int x, int y) { return x + y; }` のようにラムダ式を書くことが出来る．C++のように歴史の長い言語では既存のソースコードに干渉しないように, 新しい文法の導入には制約がある. 記号 `[]` はもともとリテラル，変数名またはキーワード `operator` の後にしかこないため，ラムダ式を表す記号として流用された．[10]

Lisp や Scheme はラムダ式がそのプログラミングの本質であるにも関わらず `lambda` という長いキーワードを用いている．例えば $\backslash\, xy \mapsto x + y$ は `(lambda (x y) (+ x y))` と書かれる．もっとも Lisp プログラマ，Scheme プログラマは `lamda` が 1 文字あるいは空気に見えている可能性はある.

1.3 さらにカリー風な書き方

我々は関数とラムダ式の「カリー風」な書き方を見てきた．この章ではさらに演算子，関数合成についてもカリー風な書き方を見ていく．

1.3.1 演算子

演算子は関数の特別な姿である．演算子は**作用素**と呼んでも良い．どちらも英語の operator の和訳である．演算子は普通アルファベット以外のシンボル 1 個で表現し，変数や関数の前に置いて直後の変数や関数に作用させるか，2 個の変数や関数の中間に置いてその両者に作用させる．例えば $-x$ のマイナス記号 $(-)$ は変数の前に置いて直後の変数 x に作用する演算子であり，$x+y$ のプラス記号 $(+)$ は 2 個の変数の間に置いてその両者 (x, y) に作用する．

1 個の変数または関数に作用する演算子を**単項演算子**と呼び，2 個の変数または関数に作用する演算子を**二項演算子**と呼ぶ．本書では単項演算子はすべて変数の前に置く，すなわち**前置**する．前置する演算子のことを**前置演算子**と呼ぶが，数学者は同じものを左作用素と呼ぶ．

Haskell には**単項マイナス** $(-)$ を除いて他に単項演算子はない．

二項演算子のうちよく使われるものは和 $(+)$，積 $(\times)$，論理和 $(\vee)$，論理積 $(\wedge)$，同値 $(\equiv)$，大なり $(>)$，小なり $(<)$ 等である．二項演算子はたとえ積記号であっても省略できない．二項演算子は多数あるので，その都度説明する．[20]

二項演算子は**中置**することが基本であるが，括弧で包むことで前置することも可能である．任意の二項演算子 $\circ$ について $x \circ y$ 及び $(\circ)xy$ は全く同じ意味である．すなわち

$$(\circ)xy = x \circ y \tag{21}$$

である．従って，二項演算子と 2 引数関数に本質的な差はない．本書では演算子と関数という用語は全く同じ意味で用いる．[21]

一般の関数が左結合であることを思い出すと，二項演算子を関数に見立てた $(\circ)$ についても

$$(\circ)xy = ((\circ)x)y \tag{22}$$

であるから，部分適用が可能である．この式から第 2 引数 y を取り除いて $((\circ)x)$ という「餓えた」1 引数関数を取り出せる．例えば関数 $((+)1)$ は引数に 1 を加える関数である．[22]

前置される二項演算子 $(\circ)$ は，ラムダ式 $(\lozenge \circ \lozenge)$ の無名パラメタ $\lozenge$ を省略したものと考えても良い．また $(\lozenge \circ x)$ や $(x \circ \lozenge)$ から無名パラメタを省略した $(\circ\, x)$ と $(x\, \circ)$ も有効な表現であり，特別に**セクション**と呼ばれる．

二項演算子 $\circ$ に対して $(\circ\, x)$ および $(x\, \circ)$ はそれぞれ以下の通りである．

$$\begin{aligned}(\circ\, x) &= (\lozenge \circ x)\\ (x\, \circ) &= (x \circ \lozenge) = ((\circ)x)\end{aligned} \tag{23}$$

例えば $(1\,+)$ は $((+)1)$ と等価であり，和は**交換則**を満たすので $(+1)$ とも等価である．ただ

[20] Haskell では $\wedge$ を `&&` と書き，$\vee$ を `||` と書く．

[21] Haskell では任意の二項演算子を括弧で包むことで前置演算子として使うことができる．例えば `x + y` と `(+) x y` は同じ結果を返す．

[22] Haskell では $((+)1)$ を `((+)1)` と書く．

し，**単項マイナス** $(-)$ だけは例外で，(-1) はマイナス 1 を表す．[23]

なお，二項演算子の結合性，すなわち左結合か右結合かは，演算子によって異なる．また演算の優先順位を明示的に与えるために括弧が用いられる．

一般の 2 引数関数 f を中置演算子に変換する記号 $\grave{f}$ を今後用いる．この記号を用いると値 fxy のことを $x\,\grave{f}\,y$ と書くことができる．[24]

1.3.2 関数合成と関数適用

ある変数に複数の関数を順に適用することはよくあることである．例えば次のようにしたいことがある．

```
# Python
y = f(x)
z = g(y)
```

あるいは同じことであるが次のようにしたいことがある．

```
# Python
z = g(f(x))
```

このコードを本書の記法で書けば $z = g(fx)$ である．この式から括弧を省略して $z = gfx$ としてしまうと，関数適用は左結合するから $z = (gf)x$ の意味になってしまう．関数 g が引数に関数を取るので無い限り (gf) は無意味なので $z = g(fx)$ の括弧は省略できない．

ここで，引数のことは忘れて，関数 f と関数 g を先に**合成**しておきたいとしよう．その合成を $g \circ f$ と書く．演算子 $\circ$ は**関数合成演算子**と呼ぶ．合成はラムダ式を使って

$$g \circ f = g(f\lozenge) \tag{24}$$

と定義できる．[25]

関数合成演算子は，連続して用いることができる．関数合成演算子は左結合するので，関数 f, g, h について $h \circ g \circ f = (h \circ g) \circ f$ であるが，これを展開すると以下のようになる．

$$\begin{aligned}(h \circ g) \circ f &= (h \circ g)(f\lozenge) \\ &= h(g(f\lozenge)) \\ &= h \circ (g \circ f)\end{aligned} \tag{25}$$

そのため関数合成は順序に依存せず，次の関係が成り立つ．

$$h \circ g \circ f = h \circ (g \circ f) \tag{26}$$

式 26 で示される関係を**関数の合成則**と呼ぶ．

♣

[23] Haskell は $(1+)$ を `(1+)` と書く．また `(-1)` はセクションではなくマイナス 1 を表す（`-1` というリテラルとみなされる）．ただし `(- 1)` のように空白を挟んでも同じくマイナス 1 とみなされる（`1` というリテラルに単項マイナス演算子が適用される）．

[24] Haskell では $x\,\grave{f}\,y$ を `x` ` `f` ` `y` と書く．

[25] Haskell では関数 `g` と関数 `f` の合成は `g . f` である．式 $z = (g \circ f)x$ は `z = (g . f) x` と書く．

Haskell には何もしない関数 id が用意されている．関数 id は引数をそのまま返す関数で，

$$\mathrm{id}\, x = x \tag{27}$$

と定義される．関数 id は**恒等関数**と呼ばれる．

恒等関数は何もしない関数なので，任意の関数 f に対して以下の等式が成り立つ．

$$\mathrm{id} \circ f = f \tag{28}$$

式 28 と 式 26 をまとめて**関数の合成則**と呼ぶ．そこで，関数の合成則を箱に入れて掲載しておこう．

関数の合成則

1. 恒等関数の存在：任意の関数 f に対して恒等関数 id ただし $\mathrm{id} \circ f = f$ が存在する．
2. 結合則：任意の関数 f, g, h について $(h \circ g) \circ f = h \circ (g \circ f)$ が成り立つ．

♡

♣

関数合成演算子は演算子の中で優先順位がもっとも高く，関数適用の次に優先される．一方，関数合成演算子とは逆に，結合の優先順位の低い**関数適用演算子**も考えておくと便利なこともある．関数適用演算子 § を次のように定義しておく．

$$f \,\S\, x = fx \tag{29}$$

演算子 § の優先順位はあらゆる演算子よりも低いものとし，また右結合とする．関数適用演算子を用いて $z = g(fx)$ を書き直すと $z = g \,\S\, fx$ となる．演算子 § の優先順位は足し算よりも低いので $f(x+1)$ は $f \,\S\, x + 1$ と書くこともできる．演算子 § を閉じ括弧のいらない開き括弧と考えてもよい．[26]

関数適用演算子のもう一つの興味深い使い方は，関数適用演算子の部分適用である．セクション $(\S\, x)$ を用いると

$$(\S\, x)f = (\diamond \,\S\, x)f = f \,\S\, x \tag{30}$$

であるから，関数適用演算子を関数と引数の順序を逆に書ける．[27]

1.3.3 高階関数

関数を引数に取ったり，あるいは関数を返す関数のことを**高階関数**と呼ぶことがある．関数合成演算子と関数適用演算子は高階関数の好例である．

他に例えば，引数として整数 n を取り，関数 $fx = n + x$ を返すような関数 g を次のように定義することが出来る．

$$gn = n + \diamond \tag{31}$$

このとき，

$$\begin{aligned} f &= g100 \\ x &= f1 \end{aligned} \tag{32}$$

とすれば $x = 101$ を得る．[28]

[26] Haskell では $g \,\S\, fx$ を `g $ f x` と書く．
[27] Haskell では $(\diamond \,\S\, x)f$ を `($x)f` と書く．
[28] Haskell では $gn = n + \diamond$ を $gn = \backslash\, x \mapsto n + x$ と展開しておいて `g n = \ x -> n + x` と書く．

高階関数は今後度々顔をだすことになる．後で登場する**マップ演算子**や**畳み込み演算子**は高階関数の一種である．

♣

ラムダ式をサポートするほとんどのプログラミング言語は，**レキシカルクロージャ**をサポートする．レキシカルクロージャとは，ラムダ式が定義された時点での，周囲の環境をラムダ式に埋め込む機構である．例えば

$$\begin{aligned} n &= 100 \\ g &= \diamond + n \end{aligned} \tag{33}$$

というラムダ式があるとする．当然我々は関数 g がいつも $g = \diamond + 100$ であることを期待するし，Haskell においてはいつも保証される．[29]

ところが，参照透過性のない言語，言い換えると変数への破壊的代入が許されている言語では，変数 n の値がいつ変わっても不思議ではない．そこで，それらの言語では関数 g が定義された時点での n の値を，関数 g の定義に含めておく．これがレキシカルクロージャの考え方である．

Haskell ではそもそも変数への破壊的代入がないので，関数 g がレキシカルクロージャであるかどうか悩む必要はない．あえて言えば，Haskell ではラムダ式はいつもレキシカルクロージャである．もしあなたのそばの C++プログラマが「え？　Haskell にはレキシカルクロージャが無いの？」などと聞いてきたら，「ええ，Haskell には破壊的代入すらありませんから」と答えておこう．

1.3.4 余談：IO サバイバルキット 1

ここで，実用的な Haskell プログラムについて触れておきたいと思う．

プログラムとは合成された関数である．多くのプログラミング言語では，プログラムそのものに main という名前をつける．本書では **IO モナド**の章で述べる理由によって，main 関数をボールド体で **main** と書く．

実用的なプログラムはユーザからの入力を受け取り，関数を適用し，ユーザへ出力する．Haskell ではユーザからの 1 行の入力を **getLine** で受け取り，変数の値を **print** で書き出せる．ここに **getLine** と **print** は関数（ファンクション）ではあるが，特別に**アクション**とも呼ぶ．関数 **main** もアクションである．

ユーザ入力をただ受け取り，そのままユーザへ出力するプログラムを Haskell で書くと次のようになる．[30]

$$\mathbf{main} = \mathbf{print} \heartsuit \mathbf{getLine} \tag{34}$$

新しい演算子 ♡ は新たな関数合成演算子である**左バインド演算子**であり，アクションとアクションを合成するための特別な演算子である．詳細は**モナド**の章で述べる．

♣

何らかの数値計算を行うプログラムは，ユーザ入力を数値として読み取り，数値に作用する関数を適用し，結果をユーザへ出力する．つまり 式 34 の **print** と **getLine** の間に数値計算を行う任意の関数 f を挿入すれば良いことになる．ただし関数 f は数値を受け取って数値を

[29] Haskell では `n = 100; g = \x -> x + n` と書く．
[30] Haskell では `main = print =<< getLine` と書く．

返すものだから，ユーザ入力すなわち文字列を数値に型変換する必要がある．幸い Haskell は型の違いを吸収する read 関数を提供している．

いま，任意の関数として引数 x の 2 乗を求める関数 f を次のように定義しよう．

$$\begin{aligned} f &:: \mathbf{Double} \to \mathbf{Double} \\ fx &= x \times x \end{aligned} \tag{35}$$

式 35 の 1 行目は関数 f の型を表している．型に関しては後述する．

ユーザからの入力に関数 f を適用してユーザへ出力するプログラムを Haskell で書くと次のようになる．

$$\mathbf{main} = \mathbf{print} \circ f \circ \mathrm{read} \heartsuit \mathbf{getLine} \tag{36}$$

式 35 と式 36 に対応する Haskell コードは次のようになる．ここに `.` は関数合成演算子 ($\circ$) であり `=<<` は左バインド演算子 ($\heartsuit$) である．

```
-- Haskell
f :: Double -> Double
f x = x * x
main = print . f . read =<< getLine
```

Haskell Stack を使う場合は `Main.hs` を次のように書き換えると良いだろう．

```
-- haskell
-- Main.hs
module Main (main) where
f :: Double -> Double
f x = x * x
main :: IO ()
main = print . f . read =<< getLine
```

我々はひとまずこれで，最低限の入出力と数値計算を行うプログラムが書けたわけである．いま，以下のような数値だけを記述したファイル `input.txt` があるとしよう．

```
4
```

ここで，シェルで次のように実行する．

```
$ stack run < input.txt
```

すると，次のような出力が得られる．

```
16.0
```

このように，Haskell ではプログラムの実行によってユーザ入力を受け取り，ユーザへ出力を行うことができる．

いまはまだ「ベイビー・プログラム」であるが，今後はより実用度を増していく．

1.3.5 この章のまとめ

- 二項演算子は 2 引数関数の特別な記法である．任意の二項演算 $x \circ y$ と $(\circ)xy$ は等しい．

- 任意の二項演算子 ◌ から $(◌\,x)$ や $(x\,◌)$ といった単項演算子を作ることができる．これらの単項演算子をセクションと呼ぶ．ただし $(-x)$ は例外的に単項マイナスである．
- 任意の 2 引数関数 f は $\grave{f}$ と書くことで中置演算子となる．例えば fxy と $x\,\grave{f}\,y$ は等しい．
- 適切なふたつの関数は関数合成演算子 $(\circ)$ を用いて合成できる．
- 関数合成では結合則 $(h \circ g) \circ f = h \circ (g \circ f)$ が成り立つ．
- 恒等関数 id が存在する．恒等関数と任意の関数との合成関数は元の関数である．
- 関数適用演算子 § を導入する．$f\,\S\,x$ と fx は等しい．演算子 § はあらゆる演算子よりも優先順位が低く，右結合である．
- Haskell では関数を返す関数を定義できる．このような関数を高階関数と呼ぶ．

式の評価順序

Haskell は参照透過性を持つ言語なので，式がいつ評価されるかを考える必要はない．一方で参照透過性を持たない言語では，式の評価順序をいつも気にしておく必要がある．例えば C 言語は関数引数の評価順序を定めていないので，次のコードは画面に `1 2` を出力する場合もあるし `2 1` を出力する場合もある．

```
int i = 0;
printf("%d %d\n", ++i, ++i);
```

なお C++17 では，関数引数の評価順序は未定義であるものの，逐次評価であることは保証される．すなわち，関数引数は並列には評価されないことが明記された．

1.4 型

Haskell の変数，関数にはすべて型がある．プログラマの言う型とは，数学者の言う集合のことである．本章では Haskell が扱う基本的な型であるデータ型と，パラメトリックな型である多相型，および型の型である型クラスについて述べる．また関数のカリー化についても述べる．

1.4.1 データ型

Haskell のすべての変数，関数には**型**がある．代表的な型には**整数型**，**浮動小数点数型**，**ブール型**，**文字型**がある．整数型を **Int** で，浮動小数点型を **Double** で表すことはすでに述べたとおりである．

型とは変数が取りうる値に言語処理系が与えた制約のことである．Haskell を含む多くのコンパイラ言語は**静的型付け**と言って，コンパイル時までに変数の型が決まっていることをプログラマに要求する．一方，Python のようなインタプリタ言語はたいてい**動的型付け**と言って，プログラムの実行時まで変数の型を決めない．

変数に型の制約を設ける理由は，プログラム上のエラーが減ることを期待するためである．例えば真理値が必要とされるところに整数値の変数が来ることは悪い予兆である．一方で C17 までの C 言語のように全ての変数にいちいち型を明記していくのも骨が折れる．

数学者や物理学者は変数に型の制約を求める一方，新しい変数の型は明記せず読者に推論させる方法をしばしばとる．例えば物理学では，質量 m は「スカラー」という型を持つし，速度 v は「3 次元ベクトル」という型を持つ．スカラーと 3 次元ベクトルの間に足し算は定義されていないため，例えば $m + v$ という表記を見たときに，両者の型を知っていれば直ちにエラーであることがわかる．

Haskell はコンパイラが型推論を行うことで，型が自明の場合は型を省略することができる．[31]

♣

Haskell にはよく使う型が予め用意されている．

Haskell には 2 種類の**整数型**があり，ひとつは**固定長整数型**で，もうひとつは**多倍長整数型**である．我々は前者を **Int** で，後者を **Integer** で表す．多倍長整数型はメモリの許す限り巨大な整数を扱えるので，整数全体の集合に近いのであるが，本書では主に固定長整数型を用いる．この **Int** は C の `int` と似た「計算機にとって都合の良い整数」である．計算機にとって都合の良い整数とは，例えば 64 ビット計算機の場合 -2^{63} から $2^{63} - 1$ の間の整数という意味である．[32]

変数 x の型が **Int**のとき，以下のように**型注釈**を書く．[33]

$$x :: \mathbf{Int} \tag{37}$$

型注釈と値の代入は同時に行える．例えば **Int** 型の変数 x に 1 を代入することは次のように書く．[34]

$$x :: \mathbf{Int} = 1 \tag{38}$$

[31] C23 および C++11 以降は `auto` キーワードを導入し，Haskell と同様の型推論を行うようになった．
[32] Haskell では **Int** を `Int` で表す．
[33] Haskell では `x :: Int` と書く．
[34] Haskell では `x :: Int = 1` と書く．

計算機は残念ながら無限精度の実数を扱えない．そこで標準精度（単精度）の**単精度浮動小数点数型**である **Float** と，**倍精度浮動小数点数型**である **Double** が提供される．[35]

ブール型は論理値 True または False のいずれかしか値をとれない型である．論理型のことを **Bool**と書く．[36]

もう一つ，計算機ならではの型がある．それは **Int** とよく似ているが，特別に文字を扱うために考えられた**文字型** **Char** である．文字といってもその中身は整数である．整数ではあるが，わざわざ別な型とするのには理由がある．[37]

理由の第一は，文字が小さな整数であるため，文字型を独立して定義しておくことでメモリを節約できるのである．特にメモリが高価であった時代はこれが唯一の理由であった．現在でも，整数が一般に 64 ビットを消費するのに対し，UTF-8 文字エンコードを用いている場合，アルファベットは 8 ビットしか消費しない．

理由の第二は，単純に整数と文字が異なるからである．文字を表す変数に整数を代入するのは悪い兆しである．

理由の第三は，文字が数値にエンコードされる方式が可変長である場合に備えて，整数と区別しておくためである．例えば UTF-8 文字エンコードは可変長エンコーディングを行う．

これらのような基本的な型を**データ型**と呼ぶ．

♣

関数にも型がある．例えば整数引数を一つ取り，整数を返す関数 f は

$$f :: \mathbf{Int} \to \mathbf{Int} \tag{39}$$

という型を持つ．この型は

$$f :: \underbrace{\mathbf{Int}}_{x} \to \underbrace{\mathbf{Int}}_{fx} \tag{40}$$

のようにイメージすると良い．これは関数 f が集合 **Int** から集合 **Int** への**写像**であると読む．[38]

Haskell ではすべての変数，関数に型があり，型はコンパイル時に決定されていなければならない．ただし，式から**型推論**が行える場合は型注釈を省略できる．

1.4.2 カリー化

Haskell では，どのような関数であれ引数は 1 個しかとらない．引数が 2 個あるように見える関数として，例えば gxy があったとしよう．ここに g は関数，x と y は変数である．関数適用は左結合であるから，これは $(gx)y$ である．ここに (gx) は引数 y をとる関数であると見ることができる．つまり，関数 g とは引数 x をとり「引数 y をとって値を返す関数 (gx) を返す」関数であると言える．

二項演算 $x + y$ は $(+)xy$ とも書けたことを思い出そう．これも左結合を思い出すと $(+)xy = ((+)x)y$ であるから，y という引数を $((+)x)$ という関数に食わせていると解釈できる．

[35] Haskell では **Float**, **Double** をそれぞれ `Float` と `Double` で表す．
[36] Haskell ではブール型を `Bool` と書く．
[37] Haskell では **Char** を `Char` で表す．
[38] Haskell では `f :: Int -> Int` と書く．なお正確には矢印記号 $(\to)$ は型コンストラクタである．無名の型引数を $\blacklozenge$ で表すと $\to$ は $(\blacklozenge \to \blacklozenge)$ という，ふたつの型引数を取る型コンストラクタである．

ラムダ式の場合は話はもっと単純で，形式的に

$$\backslash\, xy \mapsto x + y = \backslash\, x \mapsto (\backslash\, y \mapsto x + y) \tag{41}$$

のように展開すれば 1 引数にできる．矢印 $\mapsto$ は**右結合**である．そこでこのラムダ式は括弧を省略して

$$\backslash\, xy \mapsto x + y = \backslash\, x \mapsto \backslash\, y \mapsto x + y \tag{42}$$

とも書かれる．

複数引数をとる関数を 1 引数関数に分解することを**カリー化**と呼ぶ．これはこの分野の先駆者である**ハスケル・カリー**の名前に由来する．

整数引数を二つ取り，整数を返す関数 g は

$$g :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \tag{43}$$

という型を持つ．写像の矢印記号は右結合するので，これは

$$g :: \mathbf{Int} \to (\mathbf{Int} \to \mathbf{Int}) \tag{44}$$

と同じ意味である．上式は

$$g :: \underbrace{\mathbf{Int}}_{x} \to \overbrace{\underbrace{\mathbf{Int} \to \overbrace{\mathbf{Int}}^{(gx)y}}_{gx}}^{y} \tag{45}$$

のようにイメージすると良い．自然言語で考えると **Int** 型の引数を一つ取り，**Int** 型の引数を一つ取って **Int** 型の値を返す関数を返す，と読める．

♣

Haskell には**タプル**という型がある．タプルとは，複数の変数を組み合わせたもので，例えば変数 x と y をひとまとめにした $(\!|x, y|\!)$ はタプルである．変数 x, y の型は同じでも良いし，異なっても良い．[39]

タプルの型は，要素の型をタプルにしたものである．例えば **Int** が 2 個からなるタプルの型は次のようになる．[40]

$$z :: (\!|\mathbf{Int}, \mathbf{Int}|\!) \tag{46}$$

いまタプルを引数に取る関数 $f(\!|x, y|\!) = x + y$ があったとしよう．Haskell にはタプルをとる関数をカリー化する関数 curry があり，$(\mathrm{curry}\ f)xy$ は $x + y$ になる．

逆に，カリー化された関数 $f'xy = x + y$ に関しては $(\mathrm{uncurry}\ f')(\!|x, y|\!)$ のように**アンカリー化**することで，タプルに適用することができる．

タプルの中身の個数は 0 個または 2 個以上でなければならず，上限は処理系によって定められている．2 個の変数からなるタプルを特別に**ペア**，3 個の変数からなるタプルを特別に**トリプル**と呼ぶ．中身が 0 個のタプルを $\emptyset$ で表し，特別に**ユニット**と呼ぶ．[41]

ユニット $\emptyset$ の型は**ユニット型**で，変数 x がユニット型の場合は次のように書く．[42]

[39] Haskell では $(\!|x, y|\!)$ を `(x, y)` と書く．
[40] Haskell では `z :: (Int, Int)` と書く．
[41] GHC v8.2.1 は最大 62 個の変数からなるタプルまで生成できる．
[42] Haskell では `x :: ()` と書く．

$$x :: ()\tag{47}$$

ユニット型の変数が取れる唯一の値は**ユニット** ($\emptyset$) である．[43]

1.4.3 多相型と型クラス

整数型 (**Int**) と浮動小数点型 (**Double**) はよく似ている．どちらも値同士を比較可能で，それ故どちらにも等値演算子が定義されている．

整数型の等値演算子は

$$(\equiv) :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Bool}\tag{48}$$

であり，浮動小数点型の等値演算子は

$$(\equiv) :: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Bool}\tag{49}$$

である．

このように型が異なっても（だいたい）同じ意味で定義されている演算子のことを**多相的**な演算子と呼ぶ．等値演算子は多相的な演算子の例である．

具体的な型を指定せずに，仮の変数で表したものを**型パラメタ**と呼ぶ．我々は型パラメタをボールドイタリック体で表す．いま型を表す仮の変数を $\boldsymbol{a}$ として，等値演算子の型を次のように表現してみよう．

$$(\equiv) :: \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool}\tag{50}$$

このような型パラメタを用いた型を総称して**多相型**と呼ぶ．

実は式 50 は不完全なものである．このままでは型 $\boldsymbol{a}$ に何の制約もないため，等値演算の定義されていない型が来るかもしれないからである．そこで，型自身が所属する，より大きな型があるとしよう．そのような抽象度の高い型を我々は**型クラス**と呼ぶ．例えば型 **Bool**, **Int**, **Integer**, **Float**, **Double** は全て等値演算が定義できるので，型クラス Eq に属すとする．この関係を我々は

$$\mathsf{Eq} \supset \mathbf{Bool}, \mathbf{Int}, \mathbf{Integer}, \mathbf{Float}, \mathbf{Double}\tag{51}$$

と書く．ここに $\mathsf{Eq} \supset \boldsymbol{a}$ と書いて「型 $\boldsymbol{a}$ は型クラス Eq の**インスタンス**である」と読む．

式 50 に型クラスの制約を加えてみよう．型 $\boldsymbol{a}$ は型クラス Eq に属さなければならないから，新たな記号 $\Rightarrow$ を使って

$$(\equiv) :: \mathsf{Eq} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool}\tag{52}$$

と書くことにする．[44]

型 $\boldsymbol{a}$ の変数同士の間で大小関係が定義されている場合，かつその型が型クラス Eq に属する場合，その型は型クラス Ord にも属する．型クラス Ord に属する型は比較演算子 $<, \leq, \geq, >$ を提供する．例えば型 **Int** は型クラス Ord に属すが，型 **Bool** は型クラス Ord に属さない．

型 $\boldsymbol{a}$ の変数同士の間で四則演算関係が定義されている場合，かつその型が型クラス Eq に属する場合，その型は型クラス Num にも属する．型クラス Num に属する型は二項演算子 $+, -, *, /$ を提供する．ここに $-$ は二項演算子のマイナスである．

[43] Haskell ではユニットを `()` と書く．ユニットはユニット型の定数である．
[44] Haskell では `(==) :: Eq a => a -> a -> Bool` と書く．記号 $\supset$ は省略する．

型 $\boldsymbol{a}$ が型クラス Ord 及び型クラスNum に属しているとき，かつそのときに限り，型 $\boldsymbol{a}$は型クラス Real にも属する．

型 $\boldsymbol{a}$ の変数について，一つ小さい値を返す関数 pred と一つ大きい値を返す関数 succ が定義されているとき，かつそのときに限り，型 $\boldsymbol{a}$ は型クラス Enum に属する．

型 $\boldsymbol{a}$ が型クラス Real 及び型クラスEnum に属しているとき，かつそのときに限り，型 $\boldsymbol{a}$ は型クラス Integral にも属する．

便利な**型変換演算子**をひとつ紹介しておこう．型変換演関数 fromIntegral は

$$\mathrm{fromIntegral} :: \mathsf{Integral} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{b} \tag{53}$$

という型を持ち，Integral 型クラスの型の変数を任意の型へ変換する．例えば，

$$x :: \mathbf{Double} = \mathrm{fromIntegral}\ 1 :: \mathbf{Int} \tag{54}$$

とすることで，**Double** 型の変数 x に **Int** 型の定数を代入できる．[45]

1.4.4 余談：全称量化子

型 $\boldsymbol{a}$ の変数を引数に取り，型 $\boldsymbol{a}$ の戻り値を返す関数 f の型注釈は

$$f :: \boldsymbol{a} \to \boldsymbol{a} \tag{55}$$

である．この記法は実はシンタックスシュガーで，本来は次のように書くべきものである．[46]

$$f :: \forall \boldsymbol{a} \mapsto \boldsymbol{a} \to \boldsymbol{a} \tag{56}$$

ここに $\forall$ は**全称量化子**という記号で，ラムダ式におけるラムダ記号 (\) に相当する．Haskell は小文字で始まる型名を型パラメタとみなすため，全称量化子を省略しても型を正しく推論できるのである．

例えば

$$f :: \mathsf{Num} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{a} \tag{57}$$

は，本来は

$$f :: \forall \boldsymbol{a} \mapsto \mathsf{Num} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{a} \tag{58}$$

と書くべきところである．[47]

1.4.5 この章のまとめ

- Haskell は静的型付け言語である．Haskell の基本的な型を総称してデータ型と呼ぶ．
- Haskell には 2 種類の整数型 (**Int**, **Integer**) と 2 種類の浮動小数点型 (**Float**, **Double**) がある．
- Haskell には文字型 (**Char**) がある．文字エンコードは UTF-8 である．
- Haskell には論理型 (**Bool**) がある．論理型の値は True と False のみである．
- 変数，関数の型を指示することを型注釈と呼ぶ．型注釈は $x :: \mathbf{Int}$ のように書く．
- Haskell は型推論を行うため，型注釈を省略できる．
- 関数の型は $f :: \mathbf{Int} \to \mathbf{Int}$ のように矢印 ($\to$) を使って書く．
- 複数引数を取る関数を 1 引数関数に分解することをカリー化と呼ぶ．
- 複数変数をひとまとめにするタプル型がある．タプルは (x, y) のように書く．

---

[45] Haskell では `x :: Double = fromIntegral 1 :: Int` と書く．

[46] Haskell では `f :: forall a . a -> a` と書く．記号 $\forall$ が `forall` であり，記号 $\mapsto$ が `.` である．

[47] Haskell では `f :: forall a . Num a => a -> a` と書く．

- 中身のないタプルのことをユニット型と呼ぶ．ユニット型が持てる唯一の値はユニット $(\emptyset)$ である．
- 型はパラメタとして現れることもある．引数と戻り値の型が同じ関数は $\boldsymbol{a} \to \boldsymbol{a}$ という型を持つ．これは $\forall \boldsymbol{a} \Mapsto \boldsymbol{a} \to \boldsymbol{a}$ のシンタックスシュガーである．
- 型クラスを用いて，型パラメタに制約を与えることが出来る．例えば型 $\boldsymbol{a}$ に等値 $(\equiv)$ が定義されていることを要請する場合は $\mathsf{Eq} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a}$ と書く．

べき乗

Haskell には 2 種類のべき乗演算子がある．ひとつは任意の数の整数乗で $x \uparrow n$ である．ここに x は Num 型クラスの変数で n は Integral 型クラスの変数である．式 $x \uparrow n$ を Haskell では `x ^ n` と書く．なお演算子 $\uparrow$ は**クヌースの矢印**とも呼ばれる．

もうひとつは実数の実数乗で x^y である．ここに x, y は Floating 型クラスの変数である．式 x^y を Haskell では `x ** y` と書く．

1.5 リスト

型から作る型を**コンテナ型**と呼ぶ．代表的なコンテナはある型の**ホモジニアスな配列**である**リスト**である．この章ではリストと，リストに対する重要な演算である畳み込み，マップを取り扱う．

1.5.1 リスト

同じ型の値を一列に並べたもの，つまりホモジニアスな配列のことをリストと呼ぶ．Python ではリスト xs を次のように定義できる．

```
# Python
xs = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

我々も 0 から始まり 9 まで続く整数のリストを $[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$ と書くことにしよう．ただし，これでは冗長なので公差が正の**等差数列**に限って簡略化した書き方を許す．例えば 0 から 9 までのリストは $[0, 1, \ldots, 9]$ と書いても良い．また第 2 項が省略された場合は公差が 1 であるとみなす．例えば $[0, \ldots, 9]$ は $[0, 1, \ldots, 9]$ と同じである．[48]

リストの中身の一つ一つの値のことを**要素**と呼ぶ．要素のことは元と呼んでも良いが，本書では要素と呼ぶことにする．要素も元も英語の element の和訳である．

複数の型の要素が混在してもよい配列のことを**ヘテロジニアスな配列**と呼び，ホモジニアスな配列とは区別する．

リストを指す変数は，リストであることを忘れないように変数名に s を付けて

$$x_{\mathrm{s}} = [0, 1, \ldots, 9] \tag{59}$$

と書くことにする．今後 $\blacksquare_{\mathrm{s}}$ の形をした変数名が出てきたら，それはリスト変数である．なお，変数 x とリスト変数 x_{s} は異なる変数であるとする．[49]

♣

リストの型名は，元の型名をブラケットで包むことで表現する．整数型 (**Int**) のリストは [**Int**] と書き，整数のリスト型と呼ぶ．一般に型 $\boldsymbol{a}$ のリストを $[\boldsymbol{a}]$ と書く．仮の型である $\boldsymbol{a}$ は型パラメタである．[50]

型 $\boldsymbol{a}$ から型 $[\boldsymbol{a}]$ を生成する演算子を**リスト型コンストラクタ**と呼んで $[\,]$ と書く．型 $[\boldsymbol{a}]$ は $[\,]_{\boldsymbol{a}}$ のシンタックスシュガーである．[51]

型コンストラクタの概念は Python には無い（必要無い）が，静的型付け言語である C++の**クラステンプレート**が相当する．

$[x]$ のように $\boldsymbol{a}$ 型の変数 x を入れた $[\boldsymbol{a}]$ 型の変数を作る演算子を**リスト値コンストラクタ**と呼ぶ．$[\boldsymbol{a}]$ 型の変数のことを**リスト変数**とも呼ぶ．$\boldsymbol{a}$ 型の変数 x からリスト値コンストラクタを使ってリスト x_{s} を作ることは

$$x_{\mathrm{s}} = [x] \tag{60}$$

[48] Haskell では `[0, 1..9]` または `[0..9]` と書く．ピリオドの数に注意しよう．

[49] Haskell では s を変数名にくっつけて `xs = [0, 1..9]` のように書く習慣がある．なおピリオドの数に注意しよう．

[50] Haskell ではリスト型 $[\boldsymbol{a}]$ を `[a]` と書く．型 $[\boldsymbol{a}]$ は正しくは $\forall \boldsymbol{a} \mapsto [\boldsymbol{a}]$ のことである．

[51] Haskell では $[\,]_{\boldsymbol{a}}$ を `[] a` と書く．これは `[a]` と同じことである．

と書く．[52]

リスト型を表す $[\boldsymbol{a}]$ と，1 要素のリストである $[x]$ の違いにはいつも気をつけておこう．本書では中身がボールドイタリック体ならばリスト型，中身がイタリック体ならリスト値である．

ある型を包み込んだ別の型を一般にコンテナ型と呼ぶ．コンテナ型は多相型の一種である．また，コンテナ型の変数を**コンテナ変数**と呼ぶ．

♣

リストは空でもよい．**空リスト**は $\varnothing$ で表す．[53]

関数 null はリストが空リストかどうかを判定する．リスト x_{s} が空リストの場合 null x_{s} は True を，そうでなければ False を返す．関数 null は

$$\mathrm{null} :: [\boldsymbol{a}] \to \mathbf{Bool} \tag{61}$$

である．

また，同じ要素を繰り返す演算子も用意しておこう．次の**繰り返し演算子** $\circlearrowleft$ は n 個の要素 x を並べたリストを返す．[54]

$$x_{\mathrm{s}} = n \circlearrowleft x \tag{62}$$

このとき，リスト変数 x_{s} の中身は $\underbrace{[x, x, \ldots, x]}_{n}$ となる．

♣

Python では**リスト内包表記**が使える．例えば 0 から 9 までの倍数のリストは次のように作った．

```python
# Python
xs = [2 * x for x in range(0, 10)]
```

ここに `range(a, b)` は `a` から増加する方向に連続する `b` 個の整数からなるリストを返す Python の関数である．

我々も内包表記を

$$x_{\mathrm{s}} = [2 \times x \mid x \in [0, 1, \ldots, 9]] \tag{63}$$

のように書こう．ここに右辺のリストから一つずつ要素を取り出して左辺に代入する演算子 $\in$ を用いた．[55]

内包表記の式は複数あっても良い．例えば

$$x_{\mathrm{s}} = [x + y \mid x \in [0, 1, \ldots, 9], y \in [0, 1, \ldots, 5], x + y > 3] \tag{64}$$

は $0 \leq x \leq 9$ かつ $0 \leq y \leq 5$ の範囲で $x + y > 3$ となる x 及び y から $x + y$ を並べたリストである．[56]

同じことを Python で書くと次のようになる．

[52] Haskell では `xs = [x]` と書く．
[53] Haskell では空リストを `[]` で表す．
[54] Haskell では `xs = n` \`replicate\` `x` と書く．
[55] Haskell では $x_{\mathrm{s}} = [x \times 2 \mid x \in [0, 1, \ldots, 9]]$ を `xs = [x * 2 | x <- [0, 1..9]]` と書く．
[56] Haskell では `xs = [x + y | x <- [0, 1..9], y <- [0, 1..5], x + y > 3]` と書く．

```python
# Python
xs = [x + y for x in range(0, 10) for y in range(0, 6) if x + y > 3]
```

♣

リストは**結合**できる．例えばリスト x_s とリスト y_s を結合したリストは $x_s \divideontimes y_s$ と表現する．リストの結合演算子の型は

$$(\divideontimes) :: [\boldsymbol{a}] \to [\boldsymbol{a}] \to [\boldsymbol{a}] \tag{65}$$

である．[57]

♣

我々は無限リストを持つことができる．例えば自然数を表すリスト n_s は $n_s = [1, 2, ...]$ と書くことができる．[58]

無限リストを扱えるのは，我々がいつも遅延評価を行うからである．遅延評価とは，本当の計算は必要になるまで行わないという方式のことである．

もし本当に無限リストを計算機の上で再現する必要があったなら，計算機には無限のメモリが必要になってしまう．しかし我々は，計算が必要になるまで評価を行わないので，無限リストの中から有限個の要素が取り出されるのを待つことができるのである．演算子 $m \lightning n_s$ はリスト n_s から最初の m 個の要素からなるリストを返す．いま

$$x_s = 5 \lightning n_s \tag{66}$$

とすると，リスト x_s は $x_s = [1, 2, ..., 5]$ という値を持つ．[59]

演算子 $\lightning$ の型は

$$\lightning :: \mathbf{Int} \to [\boldsymbol{a}] \to [\boldsymbol{a}] \tag{67}$$

である．

リスト x_s の n 番目の要素には $x_s \mathbin{!!} n$ とすることでアクセスできる．[60]

♣

文字列は **Char** 型のリストである．実際 **String** 型は

$$\texttt{type}\ \mathbf{String} \stackrel{\text{def}}{=} [\mathbf{Char}] \tag{68}$$

と定義されている．[61]

ここにキーワード `type` はデータ型の別名，つまり**型シノニム**を定義するためのキーワードである．

文字列型のリテラルは次のようにダブルクオートで囲んで書く．[62]

$$t_s = \texttt{"Hello, World!"} \tag{69}$$

リストに対するすべての演算は文字列にも適用可能である．

---

[57] Haskell では $x_s \divideontimes y_s$ を `xs ++ ys` と書く．
[58] Haskell では `ns = [1, 2..]` と書く．
[59] Haskell では `xs = 5 `take` ns` と書く．
[60] Haskell では `xs !! n` と書く．
[61] Haskell では `type String = [Char]` と書く．
[62] Haskell では `ts = "Hello, World!"` と書く．

1.5.2 畳み込み

我々はよくリストの総和を表現するために**総和演算子** ($\sum$) を使う．総和演算子とはリスト $[x_0, x_1, ..., x_n]$ に対して

$$\sum [x_0, x_1, ..., x_n] = x_0 + x_1 + ... + x_n \tag{70}$$

で定義される演算子である．

この表現を一般化してみよう．リスト $[x_0, x_1, ..., x_n]$ が与えられたとき，任意の二項演算子を $\circ$ として

$$\bigcup_a^{\circ} [x_0, x_1, ..., x_n] = a \circ x_1 \circ ... \circ x_n \tag{71}$$

であると定義する．

この新しい演算子 $\bigcup$ は**畳み込み演算子**と呼ばれる．変数 a を**アキュムレータ**と呼ぶ．アキュムレータは右側の引数が空であった場合のデフォルト値と考えても良い．[63]

Python 2.7 には畳み込み演算子に相当する `reduce` 関数があり，リスト `xs` の総和 `s` を次のように求めることが出来た．

```
# Python 2.7
xs = [0, 1, 2, 3, 4, 5]
s = reduce(lambda x, y: x + y, xs, 0)
```

この `reduce` 関数は Python バージョン 3 では外部モジュールに移されてしまったが，Ruby には標準関数 `inject` として受け継がれている．Ruby では次のように書ける．

```
# Ruby
xs = [0, 1, 2, 3, 4, 5]
s = xs.inject(0) { |x, y| x + y }
```

リストの総和をとる演算子 $\sum$ は畳み込み演算子 $\bigcup$ を用いて

$$\sum x_{\mathrm{s}} = \bigcup_0^{+} x_{\mathrm{s}} \tag{72}$$

とすれば得られる．この式は両辺の x_{s} を省略して

$$\sum = \bigcup_0^{+} \tag{73}$$

とも書く．

リストの要素のすべての積をとる演算子 $\prod$ は

$$\prod = \bigcup_1^{\times} \tag{74}$$

とすれば得られる．

畳み込み演算子は第 1（上）引数に「$\boldsymbol{a}$ 型と $\boldsymbol{b}$ 型の引数を取り $\boldsymbol{a}$ 型の戻り値を返す」二項演算子，第 2（下）引数に「$\boldsymbol{a}$ 型」の初期値，第 3（右）引数に「$\boldsymbol{b}$ 型のリスト」すなわち $[\boldsymbol{b}]$ 型

[63] Haskell では $\bigcup_a^{+} x_{\mathrm{s}}$ を `foldl (+)) a xs` と書く．

の変数を取り，$\boldsymbol{a}$ 型の値を返す．従って畳み込み演算子の型は

$$\bigcup :: (\boldsymbol{a} \to \boldsymbol{b} \to \boldsymbol{a}) \to \boldsymbol{a} \to [\boldsymbol{b}] \to \boldsymbol{a} \tag{75}$$

である．

畳み込み演算子には次のようなもう一つのバリエーションがある．

$$\bigcup_{a}^{\circ}[x_0, x_1, ..., x_n] = (x_0 \circ (x_1 \circ ... \circ (x_n \circ a))) \tag{76}$$

これは**右畳み込み**と呼ばれる演算子である．[64]

畳み込み演算子の面白い応用例を示そう．リストの結合演算子 (※) を使うと

$$\bigcup_{\varnothing}^{※}[[0, 1, 2], [3, 4, 5], ...] = [0, 1, 2, 3, 4, 5, ...] \tag{77}$$

であるから，演算子 $\bigcup_{\varnothing}^{※}$ はリストを平坦化する**平坦化演算子**である．平坦化演算子は concat 演算子とも呼ばれることもあるが，基本的な演算子であるため特別な記号をつけておこう．我々は

$$\flat = \bigcup_{\varnothing}^{※} \tag{78}$$

と定義することにする．[65]

1.5.3 マップ

リストの各要素に決まった関数を適用したい場合がある．Python ではリスト `xs` に関数 `f` を適用するときには次のように書く．

```
# Python
f = lambda x: 100 + x
xs = [1, 2, 3, 4, 5]
ys = map(f, xs)
```

このコードは結果として `ys` に `[101, 102, 103, 104, 105]` を入れる．

このように引数として任意の関数 f と任意のリスト $[x_0, x_1, ..., x_n]$ を取り，戻り値として $[fx_0, fx_1, ..., fx_n]$ を返す演算子 $*$ を考えよう．このとき

$$f * [x_0, x_1, ..., x_n] = [fx_0, fx_1, ..., fx_n] \tag{79}$$

であると定義する．この演算子 $*$ をリストの**マップ演算子**と呼ぶ．[66]

リストのマップ演算子はリスト内包表記を使って次のようにも書ける．

$$f * x_{\mathrm{s}} = [fx \mid x \in x_{\mathrm{s}}] \tag{80}$$

しかし，マップは演算子として定義しておいたほうが見通しが良くなる．

[64] Haskell では `foldr (+) xs a` と書く．引数の順序に注意しよう．
[65] Haskell では演算子 $\flat$ の代わりに `concat` 関数を使う．
[66] Haskell では `map f xs` または `f <$> xs` と書く．演算子 `<$>` は `fmap` 演算子の中置バージョンである．

リストのマップ演算子の型は

$$(*) :: (\boldsymbol{a} \to \boldsymbol{b}) \to [\boldsymbol{a}] \to [\boldsymbol{b}] \tag{81}$$

である．矢印 $\to$ は右結合なので，これは

$$(*) :: (\boldsymbol{a} \to \boldsymbol{b}) \to ([\boldsymbol{a}] \to [\boldsymbol{b}]) \tag{82}$$

の意味でもある．念のため上式に注釈を加えると

$$(*) :: \underbrace{(\boldsymbol{a} \to \boldsymbol{b})}_{f} \to \left(\underbrace{[\boldsymbol{a}]}_{[x_0, x_1, \ldots, x_n]} \quad \to \quad \underbrace{[\boldsymbol{b}]}_{[fx_0, fx_1, \ldots, fx_n]} \right) \tag{83}$$

である．ここで f と $(f\,*)$ の型を並べてみると

$$\begin{aligned} f &:: \boldsymbol{a} \to \boldsymbol{b} \\ (f\,*) &:: [\boldsymbol{a}] \to [\boldsymbol{b}] \end{aligned} \tag{84}$$

となり，マップ演算子が何をしているのか一目瞭然になる．

具体例を見てみよう．先程の Python コードの例にあわせて

$$\begin{aligned} f &= \backslash\, x \mapsto 100 + x \\ x_{\mathrm{s}} &= [1, 2, \ldots, 5] \\ y_{\mathrm{s}} &= f * x_{\mathrm{s}} \end{aligned} \tag{85}$$

とすると y_{s} の値は $[101, 102, 103, 104, 105]$ となる．なお式を書く順序は結果に影響を与えないので，好きに並べて良い．

1.5.4 余談：IO サバイバルキット 2

1 行ごとに 3 次元ベクトルが並べられた，以下の入力ファイルがあるとする．

```
1.0 2.0 3.0
4.5 5.5 6.5
```

このようなファイル形式は計算機科学者にとって見慣れたものである．[67]

各行つまり各ベクトルごとに，その**ノルム**を計算して出力するプログラムを書きたいとしよう．まず数列を受け取ってそのノルムを返す関数 norm を次のように定義する．

$$\begin{aligned} \mathrm{norm} &:: [\mathbf{Double}] \to \mathbf{Double} \\ \mathrm{norm}\ \varnothing &= 0.0 \\ \mathrm{norm}\ x_{\mathrm{s}} &= \mathrm{sqrt}\left(\sum [x * x \mid x \in x_{\mathrm{s}}]\right) \end{aligned} \tag{86}$$

式 86 の 1 行目に出てくる型 [**Double**] は **Double** のリストである．

式 86 は関数のパタンマッチングを使っている．引数が空リストつまり $\varnothing$ の場合，関数は 0.0 を返す．それ以外の場合は，リスト内包表記を使ってノルムを計算して返す．

入力ファイル全体を受け取るにはアクション **getContents** を用いる．

[67] 本物の計算機科学者はこのようなファイルに欠損がないか，あらかじめスクリプトを走らせて検査しておくものである．

入力ファイルを 1 行毎のリストにするには関数 lines を用いる．関数 lines の型は

$$\text{lines} :: \textbf{String} \to [\textbf{String}] \tag{87}$$

である．関数 lines を用いると，入力 `"1.0 2.0 3.0\n4.5 5.5 6.5"` が `["1.0 2.0 3.0", "4.5 5.5 6.5"]` になる．

各行を空白で区切ってリストに格納するには関数 words を用いる．関数 words の型は

$$\text{words} :: \textbf{String} \to [\textbf{String}] \tag{88}$$

である．関数 words を用いると，入力 `"1.0 2.0 3.0"` が `["1.0", "2.0", "3.0"]` になる．

文字列を浮動小数点数に変換するには次の関数 readDouble を用いる．

$$\begin{aligned} &\text{readDouble} :: \textbf{String} \to \textbf{Double} \\ &\text{readDouble} = \text{read} \end{aligned} \tag{89}$$

この関数 readDouble は標準関数 read に型注釈を付けただけのものであるが，今回の目的には十分である．例えば `readDouble "1.0"` は `1.0` を返す．

入力ファイルの各行に書かれたベクトルを対象に関数 `norm` を適用して，結果を書き出すには，これまでの関数を結合して次のように書く．

$$\textbf{main} = \textbf{print} \circ (\texttt{norm}\ *) \circ ((\text{readDouble}\ *)\ *) \circ (\text{words}\ *) \circ \text{lines} \heartsuit \textbf{getContents} \tag{90}$$

式 90 は，右辺の右端から次のように読むことが出来る．

1. 入力全体を **getContents** で読み込む．
2. 読み込んだ入力を lines でリストに変換する．
3. 1 行毎 $(*)$ のリストを words で空白で区切ったリストに変換する．
4. 各行について $(*)$ 空白で区切ったリストの 1 単語ずつ $(*)$ を readDouble で浮動小数点数に変換する．
5. 各行について $(*)$ 浮動小数点数のリストのノルムを `norm` で計算する．
6. ノルムのリストを **print** で出力する．

式 90 を Haskell コードで書くと次のようになる．Haskell では，我々のマップ演算子 $(*)$ を `<$>` と書き，我々の左バインド演算子 $(\heartsuit)$ を `=<<` と書く．

```haskell
-- Haskell
norm :: [Double] -> Double
norm [] = 0.0
norm xs = sqrt (sum [x * x | x <- xs])

readDouble :: String -> Double
readDouble = read

main :: IO ()
main = print
         . (norm <$>)
         . ((readDouble <$>) <$>)
         . (words <$>)
         . lines
         =<< getContents
```

式 90 において，アクション **print** に代えて次の **printEach** を用いると，入力と出力を同じ形式にできる．[68]

[68] Haskell では `printEach xs = print `mapM_` xs` と書く．

$$\mathbf{printEach}\ x_{\mathrm{s}} = \mathbf{print} *_{\mathrm{M}}\ x_{\mathrm{s}} \tag{91}$$

演算子 $*_{\mathrm{M}}$ はアクション版のマップ演算子である．アクションについては 節 1.9 で扱う．

1.5.5 この章のまとめ

- ホモジニアスな配列をリストと呼ぶ．型 $\boldsymbol{a}$ のリストを $[\boldsymbol{a}]$ と表記する．リストを意味する記号 [] をリスト型コンストラクタと呼ぶ．リスト $[\boldsymbol{a}]$ は $[]_{\boldsymbol{a}}$ と表記することもできる．
- リストの値は $[x_0, x_1, ..., x_n]$ のように生成する．リスト値を生成する記号 [] をリスト値コンストラクタと呼ぶ．
- 整数かつ公差が正の等差数列は $[0, 1, ..., 10]$ あるいは $[0, ..., 10]$ 途中を省略して書いても良い．
- 習慣的にリスト変数は x_{s} のように s をつけて書く．
- 空リストは $\varnothing$ と表記する．
- リストが空リストかどうかは null 関数で判定できる．
- n 個の x からなるリスト x_{s} は $x_{\mathrm{s}} = n \circlearrowleft x$ と生成できる．
- リストは内包表記でも生成できる．
- リストは結合できる．リスト x_{s} と y_{s} の結合は $x_{\mathrm{s}} \divideontimes y_{\mathrm{s}}$ と表記する．
- リスト x_{s} から最初の n 個を取り出す場合は $n \lightning x_{\mathrm{s}}$ と表記する．
- リストの長さは無限でも良い．
- 文字列とは文字型 (**Char**) のリストであり，型 **String** として定義されている．
- リストは畳み込み演算子 ($\bigcup$) で畳み込みが出来る．
- リストはマップ演算子 ($*$)でマップできる．

リスト型の表記

Haskell では $\boldsymbol{a}$ 型のリストを $[\boldsymbol{a}]$ 型あるいは $[]_{\boldsymbol{a}}$ 型と表記するが，これは他の型との整合性においていくぶん不便である場合がある．新しい Haskell 標準では $[]_{\boldsymbol{a}}$ に加えて $\mathbf{List}_{\boldsymbol{a}}$ という書き方が認められている．

一方，Haskell のリスト値コンストラクタは $[x]$ または $x : \varnothing$ であり $[]_x$ という書き方は許されない．

1.6 再帰

手続き型プログラミングを学習すると，逐次実行，条件分岐，繰り返しで任意のプログラムが書けることを教わるだろう．関数型プログラミングでは，逐次実行を意識することはあまりない．例外は入出力（IO）を扱う場合で，これは 節 1.10 で扱う．また我々は条件分岐について 節 1.2 で見てきた．この節では繰り返し処理のための再帰について学ぶ．

1.6.1 関数の再帰適用

自然数 n の**階乗**は次のように定義される．

$$n! = n \times (n-1) \times \ldots \times 1 \tag{92}$$

数学者は $0! = 1$ と定義するので，この式は

$$\begin{aligned} 0! &= 1 \\ n! &= n \times (n-1)! \end{aligned} \tag{93}$$

という風に「再帰的に」書き直すことが出来る．再帰とは，定義式の両辺に同じ関数名が現れることを指す．[69]

自然数の階乗（factorial）を計算する関数を Python で書くと次のようになる．

```python
# Python
def fact(n):
  if n == 0:
    return 1
  else:
    return n * fact(n - 1)
```

Haskell では後置演算子を定義できないので，Python プログラムに倣って関数 fact として定義しよう．関数は内部で自分自身を適用しても良いので，階乗関数 fact の定義は次のようになる．

$$\text{fact } n = \texttt{case } n \texttt{ of } \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow n \times \text{fact } (n-1) \end{cases} \tag{94}$$

と定義できる．関数が自分自身を適用することを関数の**再帰適用**と呼ぶ．[70]

これで我々は関数の適用，変数の代入，ラムダ式，条件式，再帰の方法を学んだわけである．これだけあれば，原理的にはどのようなアルゴリズムも書くことができる．今日からはカリー風な数学であらゆるアルゴリズムを表現できるのである！

♣

このように関数は再帰的に呼び出せる．いま $n \geq 0$ を前提とすると n 番目のフィボナッチ数を計算する関数 fib を次のように定義できる．

$$\begin{aligned} \text{fib } 0 &= 0 \\ \text{fib } 1 &= 1 \\ \text{fib } n &= \text{fib } (n-1) + \text{fib } (n-2) \end{aligned} \tag{95}$$

二つの自然数の**最大公約数**（GCD）を計算する関数 greatestCommonDivisor も再帰的に定義できる．次の例では関数の型定義も一緒に行うことにした．また中置演算子 % は剰余を表す．

[69] Y コンビネータを使う場合はこの限りではない．

[70] Haskell では `fact n = case n of { 0 -> 1; _ -> n * fact (n - 1) }` と書く．

$$\begin{aligned} \text{greatestCommonDivisor} &:: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \\ \text{greatestCommonDivisor } 0\ y &= y \\ \text{greatestCommonDivisor } xy &= \text{greatestCommonDivisor } (x \,\%\, y)x \end{aligned} \tag{96}$$

このように関数 greatestCommonDivisor が定義されていると

$$n = \text{greatestCommonDivisor } 10\ 15 \tag{97}$$

とすれば $n = 5$ を得る．

1.6.2 リストと再帰

数学におけるリストは自由に考えることが出来るが，計算機上ではその実装も考えておかねばならない．我々はリストを Lisp におけるリスト構造と同じ構造を持つものとする．Lisp におけるリストとは，変数 first と 変数 rest からなるペアの集合である．変数 first がリストの要素を参照し，変数 rest が次のペアを参照する．リストの最後のペアの rest は空リストを参照する．

リストのための特別な表現

$$\text{first} : \text{rest} \tag{98}$$

を用い，変数 first はリストが保持する型，変数 rest はリスト型であるとする．演算子 : を Lisp に倣って **cons 演算子**と呼ぶ．リストの構造を 図 1 で表す．

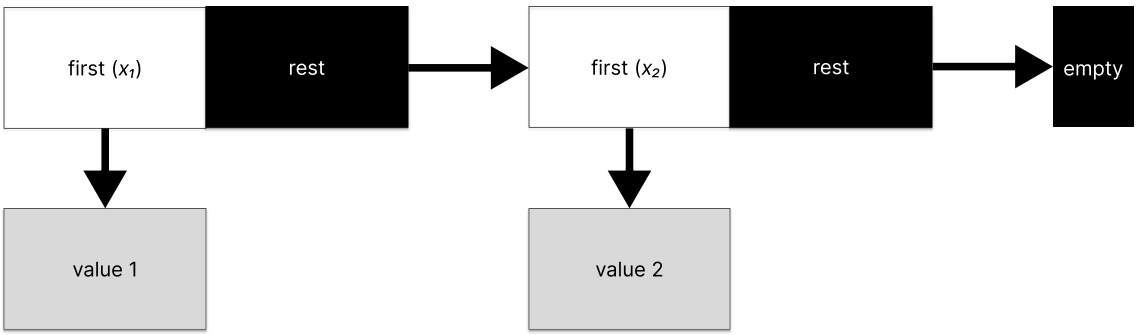

図 1: リストの構造

要素 rest はリストまたは空リストであるから，一般にリストは次のように展開できることになる．

$$\begin{aligned} [x_0, x_1, x_2, ..., x_n] &= x_0 : [x_1, x_2, ..., x_n] \\ &= x_0 : x_1 : [x_2, ..., x_n] \\ &= x_0 : x_1 : x_2 : ... : x_n : \boxslash \end{aligned} \tag{99}$$

cons 演算子 (:) は右結合する．すなわち $x_0 : x_1 : x_2 = x_0 : (x_1 : x_2)$ である．

マップ演算子 $(*)$ の実装は，リストの実装に踏み込めば簡単である．空でないリストは必ず $x : x_s$ という形をしているから，次のようにマップ演算子を定義できる．

$$\begin{aligned} f * \boxslash &= \boxslash \\ f * (x : x_s) &= (fx) : (f * x_s) \end{aligned} \tag{100}$$

♣

Haskell では任意のリスト x_s に対し，次の関数が用意されている．

$$\begin{aligned} \text{head } x_s \ldots &\ \textbf{先頭要素} \\ \text{tail } x_s \ldots &\ 2\ \textbf{番目以降の要素からなるリスト} \end{aligned} \tag{101}$$

これらは Lisp の `car` 関数，`cdr` 関数と同じものであり，この二者を用いればどのようなリストの処理も可能である．[71]

このように基本的な関数から高機能な関数を実装する方法はよく行われる．この例では cons 演算子からマップ演算子を合成した．

♣

関数はリストを受け取ることができる．次の書き方では，関数 f は整数リストの最初の要素 x と残りの要素 x_s を別々に受け取り，先頭要素だけを返す．

$$\begin{aligned} f &:: [\mathbf{Int}] \to \mathbf{Int} \\ f(x : x_s) &= x \end{aligned} \tag{102}$$

ただし，引数のリストが空リストである可能性を考慮して，式 102 は次のように書き直すべきである．

$$\begin{aligned} f &:: [\mathbf{Int}] \to \mathbf{Int} \\ f\boxslash &= 0 \\ f(x : x_s) &= x \end{aligned} \tag{103}$$

$f\boxslash$ が 0 を返すのは不自然だが，関数 f の戻り型を整数型としているためこれは仕方がない．エラーを考慮する場合は節 1.7 で述べる Maybe を使う必要がある．

♣

リストを引数にとる関数はいつでも $f(x : x_s) = \ldots$ という風にパタンマッチを行えるが，式の右辺でリスト全体すなわち $(x : x_s)$ を参照したい場合もあるであろう．そのような場合は

$$f a_s \mathbin{@} (x : x_s) = \ldots \tag{104}$$

として，変数 a_s でリスト全体を参照することも可能である．このような記法を **as パタン** と呼ぶ．[72]

♣

リストの再帰を使うと，**クイックソート** も簡単に書くことが出来る．クイックソートは次のように定義できる．

[71] Haskell では `head` 関数の利用は非推奨である．これは `head []` と書くとエラーになるからである．代わりに `headMaybe` 関数が推奨されている．

[72] Haskell では `f as @ (x : xs) = ...` と書く．

$$
\begin{aligned}
\text{quicksort}\ \varnothing &= \varnothing \\
\text{quicksort}\ (x : x_{\mathrm{s}}) &= \text{quicksort}\ l_{\mathrm{s}} \mathbin{※} [x] \mathbin{※} \text{quicksort}\ r_{\mathrm{s}} \\
&\qquad \texttt{where}\ \{l_{\mathrm{s}} = [l \mid l \in x_{\mathrm{s}}, l < x]; r_{\mathrm{s}} = [r \mid r \in x_{\mathrm{s}}, r \geq x]\}
\end{aligned} \tag{105}
$$

なお，関数は再帰させるたびに計算機のスタックメモリを消費する．これを回避するためのテクニックが，次節で述べる末尾再帰である．

1.6.3 末尾再帰

計算機科学者は，同じ再帰でも**末尾再帰**という再帰のスタイルを好む．末尾再帰とは，関数の再帰適用を関数定義の末尾にすることである．この章に出てきた階乗関数 fact を例にとろう．階乗関数 fact は

$$
\text{fact}\ x = \texttt{case}\ x\ \texttt{of}\ \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow x \times \text{fact}(x-1) \end{cases} \tag{106}
$$

のような形をしていた．末尾の関数をよりはっきりさせるために演算子 $(\times)$ を前置にして

$$
\text{fact}\ x = \texttt{case}\ x\ \texttt{of}\ \begin{cases} 0 \dashrightarrow 1 \\ \square \dashrightarrow (\times)x\ \ \text{fact}(x-1) \end{cases} \tag{107}
$$

と書いてみよう．この定義の末尾の式は $(\times)x\ \ \text{fact}(x-1)$ である．これだと末尾の関数は fact ではなく演算子 $(\times)$ なので，末尾に再帰適用を行ったことにはならない．

そこで，次のように形を変えた階乗関数 fact′ を考えてみる．

$$
\text{fact}'ax = \texttt{case}\ x\ \texttt{of}\ \begin{cases} 0 \dashrightarrow a \\ \square \dashrightarrow \text{fact}'(a \times x)\ (x-1) \end{cases} \tag{108}
$$

こうすれば末尾の関数がもとの fact′ と一致する．[73]

関数 fact が，例えば

$$
\begin{aligned}
\text{fact}\ 3 &= 3 \times \text{fact}\ 2 \\
&= 3 \times 2 \times \text{fact}\ 1 \\
&= 3 \times 2 \times 1 \times \text{fact}\ 0 \\
&= 3 \times 2 \times 1 \times 1 \\
&= 6
\end{aligned} \tag{109}
$$

と展開されるのに対し，同じく fact′ は

$$
\begin{aligned}
\text{fact}'\ 1\ 3 &= \text{fact}'(1 \times 3)\ (3-1) \\
&= \text{fact}'\ 3\ 2 \\
&= \text{fact}'(3 \times 2)\ (2-1) \\
&= \text{fact}'\ 6\ 1 \\
&= \text{fact}'(6 \times 1)\ (1-1) \\
&= \text{fact}'\ 6\ 0 \\
&= 6
\end{aligned} \tag{110}
$$

であるから，関数 fact が「横に伸びる」のに対して，関数 fact′ は「横に伸びない」ことになる．計算式が「横に伸びない」性質は，計算機のリソースを無駄に消耗しないことが期待されるため，計算機科学者が好むのである．また末尾再帰は後述する**末尾再帰最適化**をコンパイラに与える．

[73] Haskell では `fact' a x = case x of { 0 -> 1; _ -> fact' (a * x) (x - 1) }` と書く．

Haskell を含む幾つかのプログラミング言語処理系は，コンパイル時に**末尾再帰最適化**を行う．末尾再帰最適化とは，一言で言うと再帰を計算機が扱いやすいループに置き換えることである．では最初から我々もループで関数を表現しておけば，と思われるかもしれないが，再帰以外の方法でループを表現する場合には必ず変数（ループカウンタ）への破壊的代入が必要になるため，我々は末尾再帰に慎ましくループを隠すのである．[74]

1.6.4 余談：遅延評価

遅延評価のお陰で，Haskell では**無限リスト**を扱うことができる．無限リストを扱えるのは，我々がいつも遅延評価を行うからである．遅延評価とは，本当の計算は必要になるまで行わないという方式のことである．

無限リストの応用例として，素数を発見するプログラムを考えてみよう．まずある整数の素因数をすべて見つける関数 factors を定義する．関数 factors の定義は次のようになる．ただし % は整数の剰余を表す中置演算子である．

$$\begin{aligned} \text{factors} &:: \mathbf{Int} \to [\mathbf{Int}] \\ \text{factors}\ n &= [x \mid x \in [1, 2, \ldots, n], n \,\%\, x \equiv 0] \end{aligned} \tag{111}$$

関数 factors を使うと，ある整数が素数であるかどうかを判定する関数 isPrime を定義できる．関数 isPrime の定義は次のようになる．

$$\begin{aligned} \text{isPrime} &:: \mathbf{Int} \to \mathbf{Bool} \\ \text{isPrime}\ n &= \text{factors}\ n \equiv [1, n] \end{aligned} \tag{112}$$

素数は 2 以上の整数で isPrime が真であるものであるから，素数のリストを p_{s} とすると

$$p_{\mathrm{s}} = [x \mid x \in [2, 3, \ldots], \text{isPrime}\ x] \tag{113}$$

となる．最初の 500 個の素数を p_{500} とすると

$$p_{500} = 500 \downarrow p_{\mathrm{s}} \tag{114}$$

となる．

上記のすべてを Haskell で書くと次のようになる．

```haskell
-- Haskell
factors :: Int -> [Int]
factors n = [x | x <- [1..n], n `mod` x == 0]

isPrime :: Int -> Bool
isPrime n = factors n == [1, n]

ps :: [Int]
ps = [x | x <- [2..], isPrime x]

p500 = 500 `take` ps
```

このプログラムでは本格的に素数を探索することはできないが，例えば 1 万個程度の素数なら実用的な時間で計算できる．このように数学的記述をそのままプログラムに書けるところが Haskell の強みである．

♣

[74] Scheme は末尾再帰最適化を行うことが言語仕様によって決められている．

Haskell は意図しない限り遅延評価を行う．これは特に左畳み込み演算子 $\cup$ を使う場合に問題となる．いま $x_{\mathrm{s}} = [x_0, x_1, x_2, x_3]$ とすると，左畳み込み演算 $\cup_0^+ x_{\mathrm{s}}$ は

$$\begin{aligned}
\bigcup_0^+ x_{\mathrm{s}} &= \cup_0^+ (x_0 : x_1 : x_2 : \varnothing) \\
&= \bigcup_{0+x_0}^+ (x_1 : x_2 : x_3 : \varnothing) \\
&= \bigcup_{(0+x_0)+x_1}^+ (x_2 : x_3 : \varnothing) \\
&= \bigcup_{((0+x_0)+x_1)+x_2}^+ (x_3 : \varnothing) \\
&= \bigcup_{(((0+x_0)+x_1)+x_2)+x_3} \varnothing \\
&= (((0 + x_0) + x_1) + x_2) + x_3
\end{aligned} \tag{115}$$

と展開される．遅延評価のために，Haskell 処理系は値ではなく式をメモリにストアしなければならないが，左畳み込み演算は大きなメモリを必要としがちである．もし例えば予め $0 + x_0$ を先に計算しておくなど左畳み込みだけ先に評価しておけば，大いにメモリの節約になる．そのために Haskell は遅延評価無しの左畳み込み演算子を用意している．[75]

1.6.5 この章のまとめ

- 関数は最適に適用できる．再帰とは定義式の両辺に同じ関数名が現れることである．
- リストも再帰的に定義されており $[x_0, x_1, ..., x_n]$ は $x_0 : [x_1, ..., x_n]$ であり $x_0 : x_1 : ... : x_n : \varnothing$ である．
- リストを受け取る関数は $f(x : x_{\mathrm{s}})$ または $fa_{\mathrm{s}}@(x : x_{\mathrm{s}})$ のように引数を受け取れる．
- 関数の再帰的適用を行う場合は末尾再帰を行うことが望ましい．

スターリンソート

$$\begin{aligned}
\text{stalinSort} &:: \mathsf{Ord} \supset \boldsymbol{a} \Rightarrow [\boldsymbol{a}] \to [\boldsymbol{a}] \\
\text{stalinSort}\ \varnothing &= \varnothing \\
\text{stalinSort}\ [x] &= [x] \\
\text{stalinSort}\ (x : y : z_{\mathrm{s}})\ |_{x \le y} &= x : \text{stalinSort}\ (y : z_{\mathrm{s}}) \\
|_{\text{otherwise}} &= \text{stalinSort}\ (x : z_{\mathrm{s}})
\end{aligned} \tag{116}$$

[75] 遅延評価無しの左畳み込み演算子を Haskell では `foldl'` と書く．

1.7 Maybe

この章では計算結果が正しいかもしれないし，正しくないかもしれないという曖昧な状況を表す型を導入する．手始めに Python でクラス `Possibly` を実装し，それがカリー風の数式で綺麗に書けることを示す．またリストとの共通点についても見ていくことにする．

1.7.1 Possibly

計算は失敗する可能性がある．たとえば

$$z = \frac{y}{x} \tag{117}$$

のときに $x = 0$ であったとしたら，この計算は失敗する．プログラムが計算を失敗した場合，たいていのプログラマは大域ジャンプを試みる．しかし大域ジャンプは変数の書き換えを行うことであるから，別の方法が望まれる．Haskell では失敗する可能性がある場合には Maybe という機構が使える．

計算の途中で，計算にまつわる状態を残りの計算に引き継ぎたくなる場合がある．例えば，整数 x, y, z があり $z = {}^{y}/_{x}$ なる値を続く計算で利用したいとする．だが $x \equiv 0$ のときには z は正しく計算されない．こんなときプログラマが取れる手段はだいたい次のとおりだろう．

- $z = {}^{y}/_{x}$ を計算した時点で**ゼロ除算例外**を発生させ，プログラムの制御を他の場所へ移す（大域ジャンプを行う）
- グローバル変数にゼロ除算エラーが起こったことを記録しておき，z にはとりあえずの数値，例えば 0 を代入しておいて，計算を続行させる
- z にエラー状態を示す印を新たにつけておいて，計算を続行させる

大域ジャンプも，グローバル変数の書き換えも破壊的代入を伴うものであり，受け入れがたい．そこで我々は第三のエラー状態を示す印をつける方法を採用することにする．通常，変数が整数だろうが実数だろうが，計算機表現には余分なビットが残っていないので，変数をラップする次のようなクラス `Possibly` を導入することにしよう．メンバ変数 `value` が値を，メンバ変数 `valid` がエラーの有無を表す．

```
# Python
class Possibly:
  def __init__(self, a_valid, a_value = 0):
    self.valid = a_valid
    self.value = a_value
```

例えば整数値 `123` を持つ `Possibly` クラスの値 `p` は次のように生成できる．

```
# Python
p = Possibly(True, 123)
```

`Possibly` 値 `p` が計算エラーを表す場合は次のように初期化出来る．

```
# Python
p = Possibly(False)
```

ここで，引数に `1` を加えて返す関数 `f` があるとしよう．関数 `f` の定義は次の通りである．

```
# Python
f = lambda x: 1 + x
```

関数 f に直接 Possibly 値 p を食わせるとランタイムエラーを引き起こす．

```
# Python
q = f(p) # エラー!!
```

これは関数 f が引数として数値を期待していたにもかかわらず Possibly クラスの値が渡されたからである．もし関数 f のほうをいじりたくないとすれば，次のような関数 map_over を使って間接的に関数適用を行う必要がある．

```
# Python
q = map_over(f, p)
```

関数 map_over(f, p) はもし p がエラーを表す値でなければ中身の値を関数 f に適用し，その結果を Possibly クラスに包んで返す．もし p がエラー値を表す値であれば，結果もエラー値である．関数 map_over の実装は次のようになる．

```
# Python
def map_over(f, p):
  if p.valid == True:
    return Possibly(True, f(p.value))
  else:
    return Possibly(False)
```

さて，次節では以上のようなことを抽象数学的に綺麗に描いてみよう．

1.7.2 Maybe

もう一度振り出しに戻る．

いま関数 f が引数 x と y を取り，$x \neq 0$ であるならば y/x を返すとする．もし $x \neq 0$ であれば失敗を意味する Nothing（ナッシング）を返すとする．すると関数 f の定義は次のようになる．

$$fxy = \texttt{if}\ x \neq 0\ \texttt{then}\ y/x\ \texttt{else}\ \mathrm{Nothing}\ \ldots\ \textbf{不完全な定義} \tag{118}$$

残念ながら上式は不完全である．なぜならば $x \neq 0$ のときの戻り値は数であるのに対して，$x \neq 0$ のときの戻り値は数ではないからである．そこで

$$fxy = \texttt{if}\ x \neq 0\ \texttt{then}\ \mathrm{Just}_{(y/x)}\ \texttt{else}\ \mathrm{Nothing} \tag{119}$$

とする．ここに $\mathrm{Just}_{(y/x)}$ は数 y/x から作られる，値入りのコンテナである．[76]

この関数 f の戻り値はもはや整数 (**Int**) 型とは言えない．そこでこの型を $\mathbf{Maybe}_{\mathbf{Int}}$ と表して「Maybe 整数（おそらく整数）」型と呼ぶことにする．型 $\boldsymbol{a}$ から型 $\mathbf{Maybe}_{\boldsymbol{a}}$ を生成するとき，**Maybe** を**型コンストラクタ**と呼ぶ．[77]

Maybe で包まれた型を持つ変数は $z_?$ のように小さく ? をつける．例を挙げる．[78]

$$z_? :: \mathbf{Maybe}_{\mathbf{Int}} = \mathrm{Just}_1 \tag{120}$$

Maybe 変数が値を持たない場合は $z_? = \mathrm{Nothing}$ のように書く．[79]

[76] Haskell では `f x y = if x /= 0 then Just y / x else Nothing` と書く．
[77] Haskell では `Maybe a` と書く．
[78] Haskell では `zm :: Maybe Int = Just 1` と書く．
[79] Haskell では `xm = Nothing` と書く．

ここで変数 $z_?$ が取り得る値は正しく計算された値をラップしたものか，あるいはエラーを表す値 Nothing である．このように計算結果に「意味付け」をすることを文脈に入れると言う．

一度 Maybe になった変数を非 Maybe に戻すことは出来ない．変数 z が一度ゼロ除算の危険性に「汚染」された場合，その後ずっと Maybe 変数に入れ続けなければいけない．そこで，普通の変数を引数にとる関数 f に Maybe 変数 $z_?$ を食わせるには，リストの時と同じようなマップ演算子が必要になる．具体的には，変数 x が **Int** 型として，Maybe 変数 $x_? = \mathrm{Just}_x$ が与えられたとき

$$f \circledast_? x_? = \mathrm{Just}_{fx} \tag{121}$$

となるような Maybe バージョンのマップ演算子 $\circledast_?$ を用いる．ここに $f \circledast_? x_?$ の型は，もし $f :: \mathbf{Int} \to \mathbf{Float}$ ならば $\mathbf{Maybe}_{\mathbf{Float}}$ である．[80]

実際には $x_? \equiv \mathrm{Nothing}$ の可能性も考えなければならないから，Maybe バージョンのマップ演算子は

$$\begin{aligned} f \circledast_? \mathrm{Just}_x &= \mathrm{Just}_{fx} \\ f \circledast_? \mathrm{Nothing} &= \mathrm{Nothing} \end{aligned} \tag{122}$$

と定義すれば得られる．[81]

今後，普通の（引数に Maybe が来ることを想定していない）関数 f を Maybe 型である変数 $x_?$ に適用させるときには，必ず

$$z_? = f \circledast_? x_? \tag{123}$$

のように Maybe バージョンのマップ演算子 $\circledast_?$ を用いることにする．これはプログラムの安全性のためである．変数が一旦ゼロ除算の可能性に汚染されたら，最後まで Maybe に包んでおかねばならない．

♣

Python で Maybe の概念を忠実になぞることは難しい．と言うのも Python は動的型付け言語であるため，型コンストラクタという概念が無いからだ．一方で Maybe の概念を静的型付け言語である C++や Java で実現することはできる．そこで C++の本物のコードで示しておこう．ただしポインタを使わないでおいたので C++プログラマも Java プログラマも参考にできるだろう．[82]

Maybe は次の `maybe` クラステンプレートで表現できる．（Java プログラマへの注意：これは `maybe<a>` クラスの定義と同じ意味である．）

```
// C++
template <typename a> class maybe {
  private:
    a value;
    bool valid;
  public:
    maybe(): value(0), valid(false) { }  // デフォルトコンストラクタ
    maybe(a a_value): value(a_value), valid(true) { }
    a get_value() const { return value; }
    bool is_valid() const { return valid; }
```

[80] Haskell では $f \circledast_? x_?$ を `f <$> xm` と書く．
[81] Haskell では `f <$> Just x  = Just (f x); f <$> Nothing = Nothing` と書く．
[82] C++17 にはまさにこの目的のための標準テンプレート `template <typename T> std::optional<T>` がある．

```
};
```

デフォルトコンストラクタ maybe() は例外的な状況を表す Nothing を生成し，1 引数コンストラクタ maybe(a) は maybe<a> で包んだ引数値を生成する．[83]

当然我々には Maybe バージョンのマップ演算子が必要である．ここでは関数 map_over として書いてみよう．(Java プログラマへの注意 :関数 map_over はどのクラスにも属していないが，それで正解なのである．)

```
// C++
template <typename a, typename b, typename fn>
maybe<b> map_over(fn f, maybe<a> w) {
  if (w.is_valid()) {
    return maybe<b>(f(w.get_value()));
  }
  else {
    return maybe<b>();
  }
}
```

テンプレートの 3 番目の引数 fn は関数 f を受け取るために必要である．C++はコンパイル時までにすべての変数の型が決定していないといけないが，関数 f の型は関数 map_over 設計時には確定できないため，このようにテンプレートにしている．

整数 x から Maybe 値 $u_? = \mathrm{Just}_x$ を作り，関数 $gx = 1 + x$ を Maybe 値 $u_?$ に食わせて Maybe 値 $v_?$ ただし $v_? = g \circledast_? u_?$ を得ることを C++では次のように書くことになる．

```
// C++
int x = 123;
maybe<int> u(x);
auto g = [](int x) -> int { return 1 + x; };
maybe<int> v = map_over(g, u);
```

注意してほしいのは g(x) も map_over(g, u) も正当なコードだが g(u) は型エラーであることだ．また g(u.get_value()) は正当なコードだが，わざわざ u が持つ文脈を捨てることになる．

♣

リストが Maybe の中に入っている場合は，リストの各要素に関数を適用することができる．例を挙げる．

$x_? = \mathrm{Just}_{[1,2\ldots100]}$ のとき，リストの各要素に関数 $f :: \mathbf{Int} \to \mathbf{Int}$ を適用するには次のように書く．[84]

$$z_? = (f\,*) \circledast_? x_? \tag{124}$$

[83] この C++コードは，型 a が 0 を持てることを前提に書かれているため，Haskell の Maybe とは厳密には異なる．

[84] Haskell では zm = (f <$>) <$> xm と書く．最初の <$> はリストの各要素に関数 f を適用する演算子，2 番目の <$> は Maybe の中のリストの各要素に関数 f を適用する演算子である．

1.7.3 リストと Maybe

関数 f を Mayby 値 $x_?$ に適用するために

$$z_? = f \circledast_? x_? \tag{125}$$

のような Maybe バージョンのマップ演算子 $(\circledast_?)$ を使った．一方で，同じ関数 f をリスト x_s に適用するには

$$z_\mathrm{s} = f * x_\mathrm{s} \tag{126}$$

のようなリストバージョンのマップ演算子 $(*)$ を使った．

リストバージョンのマップ演算子 $(*)$ をもし C++で書くとしたら，次のようなコードになる．ここでリスト型として C++の標準テンプレートライブラリ（STL）の `std::list` クラスを流用した．

```
// C++
template <class a, class b, class fn>
std::list<b> map_over(fn f, std::list<a> xs) {
  std::list<b> y_s(xs.size());
  auto i = xs.cbegin();
  auto j = ys.begin();
  while (i != xs.cend()) {
    *j = f(*i); ++i; ++j;
  }
  return std::list<b>(ys);
}
```

この関数 `map_over` の中身部分はどうでもよろしい．それよりも，リストバージョンのマップ演算子の C++関数のインタフェースと，Maybe バージョンのマップ演算子の C++関数のインタフェースを見比べてみよう．

```
// C++
// リスト
template <class a, class b, class fn>
std::list<b> map_over(fn f, std::list<a> xs);
// Maybe
template <class a, class b, class fn>
maybe<b> map_over(fn f, maybe<a> w);
```

やはりそっくりである．であるならば，うまく統一したい．C++では次のような書き方が文法的には可能である．

```
// C++
template <class a, class b, template<class> X, class fn>
X<b> map_over(fn f, X<a> x);
```

これは一見上手く行きそうに見えるが，このコードは `map_over` のインスタンス化で躓くため，次のように `b` 型のダミー変数が必要になる．

```
// C++
template <class a, class b, template<class> X, class fn>
X<b> map_over(fn f, X<a> x, b dummy);
```

残念なことに，いずれのコードにしてもリストと Maybe の本質的な抽象化にはなってない．型 `X` がマップ可能なコンテナであることをテンプレート機構を使って保証することができないためである．この問題は C++26 で導入予定の**コンセプト**機能によって解決する見込みである．

一方で，数学者たちが見つけた圏という代数的構造が，リストも Maybe も統一的に扱うことを可能にしている．これを発見したのは Eugenio Moggi を始めとする計算機科学者たちである．この人類の英知は節 1.8 から見ていくことにしよう．

1.7.4 余談: Either

Maybe とよく似た型に Either がある．Maybe が $\boldsymbol{a}$ 型または Nothing のいずれかの値をとったように，Either は $\boldsymbol{a}$ 型または $\boldsymbol{b}$ 型のいずれかの値を取る．$\boldsymbol{a}$ 型または $\boldsymbol{b}$ 型を取る Either 型の変数 $e_!$ があるとすると，

$$e_! :: \mathbf{Either}_{\boldsymbol{ab}} \tag{127}$$

と書く．[85]

Either には値コンストラクタが 2 種類あり，それぞれ Right_x と Left_x である．値コンストラクタは

$$e_! = \mathrm{Right}_x \tag{128}$$

または

$$e_! = \mathrm{Left}_x \tag{129}$$

のように使う．[86]

Either はより複雑な計算エラーが発生する場合に用いる．Maybe が単に失敗を表す Nothing しか表現できなかったのに対し，Either は任意の型の変数で表現できる．習慣的に，正しい (right) 計算結果は Right_x 値コンストラクタで格納し，残された (left) エラーの情報は Left_x 値コンストラクタで格納する．

Either 型は C の**共有型** (`union`) や C++の**バリアント型** (`std::variant`) に近い．

1.7.5 この章のまとめ

- 計算が正しく行われるかもしれないし，正しく行われないかもしれない状態を Maybe で表現できる．
- Maybe はリストと同じく型パラメタをひとつ取る．型 $\boldsymbol{a}$ の Maybe を $\mathbf{Maybe}_{\boldsymbol{a}}$ と表記する．
- Maybe の値コンストラクタは引数をひとつ取る Just_x と引数を取らない Nothing のふたつがある．
- 関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ を Maybe 値 $x_?$ に適用するには Maybe 版のマップ演算子を用いて $z_? = f \circledast_? x_?$ と書く．このときの戻り値 $z_?$ は Maybe 値である．
- Maybe とよく似た型に Either がある．Maybe は値か Nothing かのいずれかであるが，Either は右値 Right_x か左値 Left_x かのいずれかである．
- リストのマップ演算子 $(*)$ と Maybe のマップ演算子 $(\circledast_?)$ は形がそっくりである．

[85] Haskell では `Either a b` と書く．
[86] Haskell ではそれぞれ `e = Right x` および `e = Left x` と書く．

例外と大域ジャンプ

多くのプログラミング言語が大域ジャンプをサポートしている．大域ジャンプとは，プログラムのある部分から別の部分へのジャンプを行うことを指す．多くの場合は，プログラムの逐次実行を中断し，ラベルの付けられた別の場所へ飛ぶ機能が提供される．

古典的には `goto` 命令とラベルの組み合わせが大域ジャンプに使われる．プログラム中の `goto` 命令には大した害が無いものの，ラベルはプログラムの可読性を著しく低下させる．ラベルだけでは，どこから飛んできたのか判断がつかないからである．

そもそも大域ジャンプが必要なのはエラー処理などの「例外的な事象」に限ると割り切って，専用の構文を導入したのがプログラミングにおける例外機構である．例外機構によってエラー処理の見通しは良くなるが，例外機構は参照透過性を破壊するため，関数型プログラミング言語では扱いにくい．そのために Maybe を導入したのが Haskell であり，またオプショナルなど Maybe の劣化コピーが多くのオブジェクト指向プログラミング言語で採用されている．

なお再帰では分かりづらいループに対して `goto` を使うことが手続き型プログラミングでは多く見受けられるが，Scheme のような言語では `goto` を使う代わりに継続を使うことによって見通しを良くしている．

1.8 関手

関手型クラスは，これまで見てきたリスト型や Maybe 型のような**コンテナ型**を一般化した，一段抽象度の高い概念である．この章では「型（type）の型」である**型クラス**（type class）という概念と，リスト型や Maybe 型に共通する性質を抽出した関手型クラスを導入する．**関手**（functor）は数学の**圏**（category theory）における概念で，圏から圏への写像を表す．一方 Haskell の関手 (Functor) は型クラスのひとつであり，圏論における関手と対応する概念である．本書では，数学の関手のほうを**圏論的関手**と呼び，Haskell の関手のほうを**関手型クラス**と呼ぶことにする．

1.8.1 リストの世界・Maybe の世界

$\boldsymbol{a}$ 型の変数 $x, y :: \boldsymbol{a}$ について，関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ があり

$$y = fx \tag{130}$$

であるとしよう．このように型 $\boldsymbol{a}$ で閉じた世界を仮に「$\boldsymbol{a}$ 世界」と呼ぶことにする．この世界では，関数 f は値 x を値 y に変換する．

同様に $[\boldsymbol{a}]$ 型の変数 $u_{\mathrm{s}}, v_{\mathrm{s}} :: [\boldsymbol{a}]$ について，関数 $g :: [\boldsymbol{a}] \to [\boldsymbol{a}]$ があり

$$v_{\mathrm{s}} = gu_{\mathrm{s}} \tag{131}$$

であるとしよう．このように型 $[\boldsymbol{a}]$ で閉じた世界を仮に「$[\boldsymbol{a}]$ 世界」と呼ぶことにする．この世界では，関数 g はリスト変数 u_{s} をリスト変数 v_{s} に変換する．

今度は $\mathbf{Maybe}_{\boldsymbol{a}}$ 型の変数 $s_?, t_? :: \mathbf{Maybe}_{\boldsymbol{a}}$ について，関数 $h :: \mathbf{Maybe}_{\boldsymbol{a}} \to \mathbf{Maybe}_{\boldsymbol{a}}$ があり

$$t_? = hs_? \tag{132}$$

であるとしよう．このように型 $\mathbf{Maybe}_{\boldsymbol{a}}$ で閉じた世界を仮に「$\mathbf{Maybe}_{\boldsymbol{a}}$ 世界」と呼ぶことにする．この世界では，関数 h は値 $s_?$ を値 $t_?$ に変換する．

ここで，変数 x, y と変数 $u_{\mathrm{s}}, v_{\mathrm{s}}$ はリスト値コンストラクタによって

$$\begin{aligned} u_{\mathrm{s}} &= [x] \\ v_{\mathrm{s}} &= [y] \end{aligned} \tag{133}$$

の関係にあるとしよう．リスト値コンストラクタは値を $\boldsymbol{a}$ 世界から $[\boldsymbol{a}]$ 世界へとジャンプさせる機能を持っている．また変数 x, y と Maybe 変数 $s_?, t_?$ は Maybe 値コンストラクタによって

$$\begin{aligned} s_? &= \mathrm{Just}_x \\ t_? &= \mathrm{Just}_y \end{aligned} \tag{134}$$

の関係にあるとしよう．Maybe 値コンストラクタは値を $\boldsymbol{a}$ 世界から $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界へとジャンプさせる機能を持っている．

他に $\boldsymbol{a}$ 世界から $[\boldsymbol{a}]$ 世界や $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界へと「ジャンプ」させるものがあるだろうか．よく考えてみると，マップ演算子もそうである．なぜなら $u_{\mathrm{s}} = [x], v_{\mathrm{s}} = [y]$ なのだから $\boldsymbol{a}$ 世界の関数 f と $[\boldsymbol{a}]$ 世界の関数 g は無関係ではなく

$$v_{\mathrm{s}} = gu_{\mathrm{s}} = f * u_{\mathrm{s}} \tag{135}$$

なのであるから

$$g = f * \tag{136}$$

が言える．同様に $s_? = \mathrm{Just}_x, t_? = \mathrm{Just}_y$ なのだから，$\boldsymbol{a}$ 世界の関数 f と $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界の関数

h は無関係ではなく

$$t_? = hs_? = f \circledast_? s_? \tag{137}$$

であり

$$h = f \circledast_? \tag{138}$$

である．つまりリストのマップ演算子 $*$ が関数 f を $\boldsymbol{a}$ 世界から $[\boldsymbol{a}]$ 世界へとジャンプさせ，Maybe のマップ演算子 $\circledast_?$ が関数 f を $\boldsymbol{a}$ 世界から $\mathbf{Maybe}_{\boldsymbol{a}}$ 世界へとジャンプさせているのである．ジャンプを波矢印 ($\rightsquigarrow$) で表すと，次の表のようになる．

世界（圏）	値（対象）	写像（射）	値のジャンプ	関数のジャンプ
$\boldsymbol{a}$ 世界	x, y	$y = fx$		
$[\boldsymbol{a}]$ 世界	$u_{\mathrm{s}}, v_{\mathrm{s}}$	$v_{\mathrm{s}} = gu_{\mathrm{s}}$	$x \rightsquigarrow u_{\mathrm{s}}, y \rightsquigarrow v_{\mathrm{s}}$	$f \rightsquigarrow g$
$\mathbf{Maybe}_{\boldsymbol{a}}$ 世界	$s_?, t_?$	$t_? = hs_?$	$x \rightsquigarrow s_?, y \rightsquigarrow t_?$	$f \rightsquigarrow h$

表 3: 様々な圏

いま「世界」と呼んだものを，数学者は**圏**と呼ぶ．圏とは**対象**と**射**の組み合わせである．本書では対象とは変数のことであり，射とは関数だと思えば良い．そして，圏から圏へとジャンプさせるもの ($\rightsquigarrow$) を**圏論的関手**と呼ぶ．この例で言えば値コンストラク $[x], \mathrm{Just}_x$ とマップ演算子 $*, \circledast_?$ が関手である．値コンストラクタ Just_x は $\boldsymbol{a} \to \mathbf{Maybe}_{\boldsymbol{a}}$ という型を持ち，マップ演算子 $\circledast_?$ は $(\boldsymbol{a} \to \boldsymbol{b}) \to (\mathbf{Maybe}_{\boldsymbol{a}} \to \mathbf{Maybe}_{\boldsymbol{b}})$ という型を持つ．

♣

Haskell ではマップ演算子が定義された型を**関手型クラス**と呼ぶ．具体的には，マップ演算子が定義されたすべての型は Functor 型クラスのインスタンスであるとする．つまり Functor 型クラスには一般化されたマップ演算子が定義されており，そのインスタンスであるリストや Maybe は独自のマップ演算子を定義しなければならないということである．

一般化されたマップ演算子を $\circledast$ で表そう．この $\circledast$ 演算子は

$$(\circledast) :: \mathsf{Functor} \supset \boldsymbol{f} \Rightarrow (\boldsymbol{a} \to \boldsymbol{b}) \to \boldsymbol{f}_{\boldsymbol{a}} \to \boldsymbol{f}_{\boldsymbol{b}} \tag{139}$$

という型を持つ．ここに $\mathsf{Functor} \supset \boldsymbol{f}$ は，$\boldsymbol{f}$ が Functor 型クラスに属すという制約を表している．また $\boldsymbol{f}$ は型コンストラクタであり，$\boldsymbol{f}_{\boldsymbol{a}}$ は $\boldsymbol{f}$ 型コンストラクタと $\boldsymbol{a}$ 型によって作られたコンテナ型である．

もし型コンストラクタがリスト型コンストラクタであれば，つまり $\boldsymbol{f} = [\,]$ であれば

$$(\circledast) :: (\boldsymbol{a} \to \boldsymbol{b}) \to [\boldsymbol{a}] \to [\boldsymbol{b}] = (*) \tag{140}$$

であるし，もし型コンストラクタが Maybe 型コンストラクタであれば，つまり $\boldsymbol{f} = \mathbf{Maybe}$ であれば

$$(\circledast) :: (\boldsymbol{a} \to \boldsymbol{b}) \to \mathbf{Maybe}_{\boldsymbol{a}} \to \mathbf{Maybe}_{\boldsymbol{b}} = (\circledast_?) \tag{141}$$

である．

リストと Maybe は両者ともマップ演算子（と値コンストラクタ）を持つ．両者の関係をまとてみたのが 表 4 である．オブジェクト指向プログラマなら，リストと Maybe に共通のスーパークラスを設計したくなるであろう．それが型クラス Functor である．

型	型コンストラクタ	マップ演算子	変数の具体例
$[\boldsymbol{a}]$ または $[\,]_{\boldsymbol{a}}$	$[\,]$	$*$	$[x]$
$\mathbf{Maybe}_{\boldsymbol{a}}$	$\mathbf{Maybe}$	$\circledast_?$	Just_x
$\mathsf{Functor} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f}_{\boldsymbol{a}}$	$\boldsymbol{f}$	$\circledast$	具体的な型による

表 4: 型とマップ演算子

1.8.2 関手と関手則

Haskell の関手すなわち Functor 型クラスに求められるのは，マップ演算子 $(\circledast)$ を持つことだけではない．関手のマップ演算子は，次の**関手則**を満たす必要がある．

関手則

1. 恒等射の存在（保存）：マップ演算子 $(\circledast)$ は $\mathrm{id} \circledast x_* = \mathrm{id}\, x_*$ を満たす．
2. 結合則：マップ演算子 $(\circledast)$ は $g \circledast (f \circledast x_*) = (g \circ f) \circledast x_*$ を満たす．

♡

まず関数 id を定義しておくと，これは引数をそのまま返す関数で

$$\mathrm{id}\, x = x \tag{142}$$

である．引数 x は変数でも関数でも良いので，ラムダ式を使って

$$\begin{aligned} \mathrm{id} &= \lozenge \\ &= \backslash\, x \mapsto x \end{aligned} \tag{143}$$

と書いたほうがわかり良いかもしれない．

さて，関手則には**射**（しゃ）という言葉が出てきている．数学的には，射は関数を一般化させたものであるが，Haskell では射と関数を同じ意味で用いる．射は英語の arrow または morphism の翻訳である．

恒等射の保存はリストを考えると簡単に理解できる．リスト変数 x_{s} の各要素に恒等関数 id を適用した結果は，次の式の通りリスト変数 x_{s} そのものである．[87]

$$\begin{aligned} \mathrm{id} * x_{\mathrm{s}} &= [\mathrm{id}\, x \mid x \in x_{\mathrm{s}}] \\ &= \mathrm{id}[x \mid x \in x_{\mathrm{s}}] \\ &= \mathrm{id}\, x_{\mathrm{s}} \end{aligned} \tag{144}$$

同じことを Maybe についてもやってみると，次のようになる．

[87] 任意の関数 f と任意の値コンストラクタ $\langle\!|\blacksquare|\!\rangle$ について $f \circledast \langle\!|x|\!\rangle = \langle\!|fx|\!\rangle$ が成り立つことをマップ演算子 $(\circledast)$ の準同型性と呼ぶ．Haskell では関手の準同型性を規則にしていないが，後に述べるアプリカティブ関手のインスタンスについては準同型性を要求する．

$$\begin{aligned}\mathrm{id} \circledast_? x_? &= \texttt{case}\ x_?\ \texttt{of}\ \begin{cases}\mathrm{Just}_x \dashrightarrow \mathrm{id}\,\mathrm{Just}_x \\ \boxed{} \dashrightarrow \mathrm{id}\,\mathrm{Nothing}\end{cases} \\ &= \mathrm{id}\, x_?\end{aligned} \tag{145}$$

ここで $x_? \equiv \mathrm{Nothing}$ であれば $\mathrm{id} \circledast_? x_?$ も Nothing となるので，常に $\mathrm{id} \circledast_? x_? = \mathrm{id}\, x_?$ が成立する．

関数合成の保存についても，リストを考えると理解しやすい．リスト変数 x_s の各要素に関数 f と関数 g を合成した関数 $g \circ f$ を適用した結果は，次の式の通りリスト変数 x_s の各要素に関数 f を適用した結果に関数 g を適用した結果である．

$$\begin{aligned}(g \circ f) * x_\mathrm{s} &= [(g \circ f)x \mid x \in x_\mathrm{s}] \\ &= [g(fx) \mid x \in x_\mathrm{s}] \\ &= g * [fx \mid x \in x_\mathrm{s}] \\ &= g * (f * x_\mathrm{s})\end{aligned} \tag{146}$$

なお $g * (f * x_\mathrm{s})$ をさらに次のように変形することも可能である．

$$\begin{aligned}g * (f * x_\mathrm{s}) &= g * (f * \diamond)x_\mathrm{s} \\ &= (g * \diamond) \circ (f * \diamond)x_\mathrm{s} \\ &= (g *) \circ (f *)x_\mathrm{s}\end{aligned} \tag{147}$$

式 147 より，一般性を損なうこと無く

$$g \circledast (f \circledast) = (g \circledast) \circ (f \circledast) \tag{148}$$

の関係が得られるから，関手の結合則は

$$(g \circledast) \circ (f \circledast) = (g \circ f) \circledast \tag{149}$$

とも書かれる．[88]

このような関手則は Functor 型クラスに求められる規則である．Haskell は Functor 型クラスに属する型が関手則を満たしているかチェックしないが，Haskell プログラマは Functor 型クラスの方が関手則を満たすことを期待する．

♣

[88] Haskell ではしばしば `(fmap g) . (fmap f) = fmap (g . f)` と表現される．

関数の結合則が成り立っていることも確認しよう．任意の関数 $f, g :: \boldsymbol{a} \to \boldsymbol{a}$ について

$$
\begin{aligned}
(g \circ f) \circledast x_? &= \texttt{case}\ x_?\ \texttt{of}\ \begin{cases} \mathrm{Just}_x \dashrightarrow \mathrm{Just}_{(g \circ f)x} \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\
&= \texttt{case}\ x_?\ \texttt{of}\ \begin{cases} \mathrm{Just}_x \dashrightarrow \mathrm{Just}_{g(fx)} \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\
&= \texttt{case}\ x_?\ \texttt{of}\ \begin{cases} \mathrm{Just}_x \dashrightarrow g \circledast \mathrm{Just}_{fx} \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\
&= \texttt{case}\ x_?\ \texttt{of}\ \begin{cases} \mathrm{Just}_x \dashrightarrow g \circledast (f \circledast x) \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\
&= g \circledast (f \circledast x)
\end{aligned}
\tag{150}
$$

であるから，結合則も成り立っている．

1.8.3 アプリカティブ関手

関手のマップ演算子 $(\circledast)$ をさらに汎用性のあるものにするために新しく考え出された演算子が**アプリカティブマップ演算子**である．

いま 1 変数を取る関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ があり，リスト変数 $x_{\mathrm{s}} :: [\boldsymbol{a}]$ があるとする．型 $\boldsymbol{a}$ は任意の型である．リスト変数 x_{s} の各要素に関数 f を適用した結果をリスト変数 z_{s} とする．このことを我々は関手マップ演算子 $(\circledast)$ を用いて次のように書いた．

$$
z_{\mathrm{s}} = f \circledast x_{\mathrm{s}} \tag{151}
$$

式 151 は次のように書いても同じである．

$$
z_{\mathrm{s}} = [fx \mid x \in x_{\mathrm{s}}] \tag{152}
$$

ここで 2 引数を取る関数 $g :: \boldsymbol{a} \to \boldsymbol{a} \to \boldsymbol{a}$ があるとする．この関数 g をリスト変数 x_{s} の各要素と，別なリスト変数 y_{s} の各要素に適用した結果をリスト変数 z'_{s} とする．このことを我々はどのように書いたら良いだろうか．

いま関数 g にリスト変数 x_{s} だけを適用した結果を関数 g' とする．関数 g' は次のようになる．

$$
\begin{aligned}
g' &= [gx \mid x \in x_{\mathrm{s}}] \\
&= g \circledast x_{\mathrm{s}}
\end{aligned}
\tag{153}
$$

式の右辺がリストであることから分かる通り，関数 g' はリストである．いま我々が欲しいのは，関数リスト g' をリスト変数 y_{s} の各要素に適用する演算子である．なお 式 153 の 2 行目の通り，関数リスト g' は $g \circledast x_{\mathrm{s}}$ と同じである．さて，関数リスト g' をリスト変数 y_{s} の各要素に適用する演算子を $\boxast$ とする．そうすると次のように書ける．

$$
\begin{aligned}
z'_{\mathrm{s}} &= g' \boxast y_{\mathrm{s}} \\
&= g \circledast x_{\mathrm{s}} \boxast y_{\mathrm{s}}
\end{aligned}
\tag{154}
$$

この演算子 $\boxast$ を**アプリカティブマップ演算子**と呼ぶ．アプリカティブマップ演算子は関手マップ演算子よりも汎用的である．なぜならば

$$
f \circledast x_* = \langle f \rangle \boxast x_* \tag{155}
$$

であり，関手マップ演算子 $(\circledast)$ はアプリカティブマップ演算子 $(\boxast)$ から導出できるからであ

る．ここに $\langle f \rangle$ は文脈に応じて $[f]$ であったり Just_f であったりする．記号 $\langle \blacksquare \rangle$ を**ピュア演算子**と呼ぶ．[89]

なおピュア演算子の名称は「純粋（pure）」であるが，意味合いはむしろ「不純（impure）」のほうが近い．ピュア演算子によって作られる**空**（くう）を総称して $\varnothing$ とする．空とは，リストで言えば $\varnothing$ であり，Maybe で言えば Nothing である．

ピュア演算子を用いると，式 154 の 2 行目は次のように書ける．[90]

$$z'_* = \langle g \rangle \boxast x_* \boxast y_* \tag{156}$$

式 156 はしばしば添字 $*$ を省略して次のように書かれる．

$$z' = \langle g \rangle \boxast x \boxast y \tag{157}$$

そこで，我々も誤解を招く恐れがない場合は添字 $*$ を省略することにする．

♣

ピュア演算子とアプリカティブマップ演算子を必ず持つ関手のことを**アプリカティブ関手**と呼び，型クラス Applicative に属するものとする．アプリカティブ関手は関手を拡張したものであり，次のアプリカティブ関手則を持つものとする．ただし $x_* :: \mathsf{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f_a}$ とする．

アプリカティブ関手則

1. 恒等射の存在：アプリカティブマップ演算子 $(\boxast)$ は $\langle \mathrm{id} \rangle \boxast x_* = \mathrm{id}\, x_*$ を満たす．
2. 結合則：アプリカティブマップ演算子 $(\boxast)$ は $h \boxast (g \boxast f) = \langle \circ \rangle \boxast h \boxast g \boxast f$ を満たす．
3. 準同型性：$\langle f \rangle \boxast \langle x \rangle = \langle f x \rangle$ または同じことであるが $f \circledast \langle x \rangle = \langle f x \rangle$ である．
4. 交換則：$f \boxast \langle x \rangle = \langle (\lozenge \S x) \rangle \boxast f$ が成り立つ．

♡

これらの規則のうち恒等射の存在と結合則は関手則から導かれる．

♣

アプリカティブ関手型クラスは，関手型クラスを拡張したものである．そのため，アプリカティブ関手型クラスに属する型は自動的に関手型クラスにも属する．

リスト型，Maybe 型とも，関手型クラスに属すると同時に，アプリカティブ関手型クラスにも属する．また両者ともアプリカティブ関手則を満たす．例えばリスト型の**準同型性**は次のように確かめられる．

$$f * [x] = [f x] \tag{158}$$

ここにリスト型では $\circledast = *$ であり，リスト型のピュア演算子 $\langle \blacksquare \rangle$ はリスト構成の演算子 $[\blacksquare]$ であることを思い出しておこう．

♣

アプリカティブマップ演算子を用いると**アプリカティブスタイル**という記法が使える．関数 φ, ψ が引数を文脈に入れる関数，例えば

$$\varphi, \psi :: \mathsf{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{a} \to \boldsymbol{f_a} \tag{159}$$

[89] Haskell では `pure f` と書く．
[90] Haskell では `zm = (pure g) <*> xm <*> ym` と書く．

であるとしよう．また関数 ω が 2 引数を取り

$$\omega :: \boldsymbol{a} \to \boldsymbol{a} \to \boldsymbol{a} \tag{160}$$

という型だとしよう．すると，次のようにコンテナ変数 u_*, v_* を作っておいて，関数 ω を適用させることができる．

$$\begin{aligned} u_* &= \varphi x \\ v_* &= \psi y \\ w_* &= \omega \circledast u_* \boxast v_* \end{aligned} \tag{161}$$

コンテナ変数 u_*, v_* のいずれかが $\oslash$ であればコンテナ変数 w_* の値も $\oslash$ になる．これは 2 個の計算を並列に行って，その結果をそれぞれ u_*, v_* に入れておくことを意味する．このような書き方を**アプリカティブスタイル**と呼ぶ．[11][91]

アプリカティブスタイルは，関数 ω を変数 u_*, v_* にあたかも適用したかのように見える．実際，アプリカティブスタイルはかつて次のように書くことが提案されたが，採用はされなかった．[92]

$$w = [\![\omega u v]\!] \ldots \textbf{採用されなかった文法} \tag{162}$$

1.8.4 余談：型をつくる

我々はすでにある型に別名を与える方法を見てきた．型シノニムである．例えば **String** 型は **Char** 型のリストとして定義されていて，次のように書かれている．

$$\texttt{type}\ \mathbf{String} \stackrel{\text{def}}{=} [\mathbf{Char}] \tag{163}$$

キーワード `type` は**型シノニム**（type synonym）を定義するキーワードである．型シノニムは既存の型に別名を与えるだけなので，真に新しい型を作り出しているわけではない．[12]

新しい型を作り出すにはキーワード `data` を使う．我々が今まで型（type）と呼んできたものは，実は**データ型**（data type）と呼ぶのが正式である．そこで，新しい型を作り出すキーワードも `data` なのである．型シノニムの定義には新しい型名（例えば **String**）と，元になる型名（例えば [**Char**]）をイコール ($\stackrel{\text{def}}{=}$) の両辺に書いた．新しい型を作り出す場合は，次式のようにイコールの左辺に新しい型名を書き，右辺に新しい値コンストラクタの名前を書く．

$$\texttt{data}\ \textbf{型名} \stackrel{\text{def}}{=} \textbf{値コンストラクタ} \tag{164}$$

簡単な具体例は次のようなものである．

$$\texttt{data}\ \mathbf{Zero} \stackrel{\text{def}}{=} \mathrm{Zero} \tag{165}$$

ここで左辺の **Zero** は新しい型名であり，右辺の Zero は新しい値コンストラクタの名前である．この **Zero** 型は唯一の値 Zero しか持てないので，それほど面白みのある型ではない．それでも

$$\texttt{data}\ \mathbf{Zero} \stackrel{\text{def}}{=} \mathrm{Zero}\ \texttt{deriving}\ \mathrm{Show} \tag{166}$$

というふうに型クラス Show のインスタンスであることを明示すると，Haskell は値 Zero を文字列に変換する関数 show を自動で定義してくれる．そこで

[91] Haskell では `w = omega <$> (phi x) <*> (psi y)` と書く．これは `w = do { u <- phi x; v <- psi y; return (omega u v) }` と書くのと同じである．

[92] 現在の Haskell では `w = liftA2 omega u v` と書くことで代用されている．元の提案は `w = [|omega u v|]` であった．

$$z = \mathrm{Zero} \tag{167}$$

として

$$s = \mathrm{show}\ z \tag{168}$$

とすると，s には文字列 "Zero" が入る．

♣

より興味深い型を作るには 2 種類の方向性がある．ひとつは値コンストラクタに引数を与える方法である．例えば 2 個の浮動小数点数を組み合わせた型 **XY** は次のように定義できる．[93]

$$\texttt{data}\ \mathbf{XY} \stackrel{\text{def}}{=} \mathrm{XY}_{\mathrm{Double\ Double}}\ \texttt{deriving}\ \mathsf{Show} \tag{169}$$

値 $(1.0, 2.0)$ を作り出すには

$$v = \mathrm{XY}_{1.0\ 2.0} \tag{170}$$

とする．値 v は **XY** 型の値である．

関数 addXY を次のように定義してみよう．

$$\begin{aligned} &\mathrm{addXY} :: \mathbf{XY} \to \mathbf{XY} \to \mathbf{XY} \\ &\mathrm{addXY}\ (\mathbf{XY}\ xy)(\mathbf{XY}\ x'y') = \mathrm{XY}_{x+x'\ y+y'} \end{aligned} \tag{171}$$

こうすると

$$\begin{aligned} v &= \mathrm{XY}_{1.0\ 2.0} \\ v' &= \mathrm{XY}_{3.0\ 4.0} \\ w &= \mathrm{addXY}\ vv' \end{aligned} \tag{172}$$

とすることで w には $\mathrm{XY}_{4.0\ 6.0}$ が入る．このように，値コンストラクタに引数をとる新しい型を作ることができる．その構文は次のようなものである．

$$\texttt{data}\ \textbf{型名} \stackrel{\text{def}}{=} \textbf{値コンストラクタ}_{\textbf{引数の型}} \tag{173}$$

もう一つの方向性は直和を使う方法である．直和とは，複数の型を組み合わせた型である．例えば，我々の **Bool** 型は値コンストラクタ True と False からなる直和である．実際 **Bool** 型は次のように定義されている．

$$\texttt{data}\ \mathbf{Bool} \stackrel{\text{def}}{=} \mathrm{True} \oplus \mathrm{False} \tag{174}$$

そこで **Bool** 型とよく似た我々独自の型として **Cool** 型を作ってみよう．この **Cool** 型は True, False の代わりに Yes, No, Dunno という 3 種類の値コンストラクタからなる直和である．また型クラス Show のインスタンスであることを明示して関数 show のデフォルト実装を自動で定義してくれるようにすると同時に，型クラス Eq のインスタンスであることを明示して値コンストラクタを比較する比較演算子 $(\equiv)$ も持つことを保証しよう．ただし比較演算子 $(\equiv)$ のデフォルト実装は使えないため，自前で定義することにする．

型の定義は次のようになる．[94]

$$\texttt{data}\ \mathbf{Cool} \stackrel{\text{def}}{=} \mathrm{Yes} \oplus \mathrm{No} \oplus \mathrm{Dunno}\ \texttt{deriving}\ (\mathsf{Eq}, \mathsf{Show}) \tag{175}$$

[93] Haskell では `type XY = XY Double Double deriving Show` と書く．
[94] Haskell では `data Cool = Yes | No | Dunno deriving (Eq, Show)` と書く．

比較演算子 (≡) の定義はキーワード `instance` を使って次のように書く．

$$\begin{aligned}
&\texttt{instance}\ \mathsf{Eq} \supset \mathbf{Cool}\ \texttt{where} \\
&\qquad \mathrm{Yes} \equiv \mathrm{Yes} = \mathrm{True} \\
&\qquad \mathrm{Yes} \equiv \mathrm{Dunno} = \mathrm{True} \\
&\qquad \mathrm{No} \equiv \mathrm{No} = \mathrm{True} \\
&\qquad \mathrm{No} \equiv \mathrm{Dunno} = \mathrm{True} \\
&\qquad \mathrm{Dunno} \equiv \mathrm{Yes} = \mathrm{True} \\
&\qquad \mathrm{Dunno} \equiv \mathrm{No} = \mathrm{True} \\
&\qquad \mathrm{Dunno} \equiv \mathrm{Dunno} = \mathrm{True}
\end{aligned} \tag{176}$$

否定の比較演算子 (≢) は型クラス Eq のデフォルト実装が用いられるため，自前で定義する必要はない．

♣

引数付きの値コンストラクタを使う方法と，直和を使う方法の 2 種類は組み合わせて用いることができる．例えば，次のような型クラスを作ることも可能である．[13]

$$\texttt{data}\ \mathbf{Shape} \stackrel{\text{def}}{=} \mathbf{Circle}_{\mathbf{Double}} \oplus \mathbf{Rect}_{\mathbf{Double}\ \mathbf{Double}} \tag{177}$$

1.8.5 この章のまとめ

- リスト，Maybe はよく似たマップ演算子を持つ．両者を抽象化したものが関手型クラスである．関手型クラスは他にも様々な型をインスタンスとして持つ．
- 関手型クラスに属する型は一般化されたマップ演算子 (⊛) を持つ．
- 関手型クラスに属する型は関手則を満たすように設計されなければならない．関手則とは恒等射の存在と結合則である．
- 関手型クラスを拡張したものがアプリカティブ関手型クラスである．アプリカティブ関手型クラスに属する型はアプリカティブマップ演算子 (⊠) を持つ．アプリカティブマップ演算子は複数引数を受け取る関数用のマップ演算子と考えられる．
- アプリカティブ関手型クラスに属する型はピュア演算子 (⟨■⟩) を持つ．ピュア演算子は値を最もシンプルな形でコンテナに包む．
- アプリカティブ関手型クラスに属する型はアプリカティブ関手則を満たすように設計されなければならない．アプリカティブ関手則とは恒等射の存在，結合則，準同型性，交換則である．
- Haskell では新しいデータ型を作る仕組みがあり，値コンストラクタに複数の型の変数を引数として与える方法（直積）と，値コンストラクタを選ばせる方法（直和）の両方を混在させられる．

Haskell の関手と数学の関手

Haskell の教科書では「関手」と言った場合，次のような意味を持つ場合がある．

- 関手型クラス (Functor)
- 関手型クラスのインスタンスである型（例：リストや Maybe）
- 関手型クラスのインスタンスである型の変数
- 数学での圏論における関手（functor）

このように紛らわしいことがあるため，本書では数学における「関手」のことを「圏論的関手」と呼ぶことにする．

ややこしいことに C++で `operator ()` を実装したクラスのことをファンクタ（functor）と呼ぶことがある．これは関数オブジェクト（function object）の別名で，圏論的関手や Haskell の関手型クラスとは無関係である．C++のファンクタは Haskell のラムダ式に近い．

1.9 モナド

モナド (Monad) 型クラスは，Haskell において複雑な計算や文脈付きの値をシンプルな構造で扱うための抽象的な仕組みである．モナド型クラスは関手型クラスやアプリカティブ関手型クラスをさらに拡張したもので，特に「文脈付きの計算の直列化」を可能にする．モナド型クラスに提供されるバインド演算子によって,，値が「文脈」に包まれていたとしても，その値を取り出したり，新しい計算に適用したりすることができる．これにより，「エラーが起きた場合に自動的に計算を中断する」「副作用をプログラムの他の部分から隔離する」といった柔軟な計算制御が実現できる．

1.9.1 バインド演算子

一般のマップ演算子 $(\circledast)$ をピュア演算子 $(\langle\blacksquare\rangle)$ と一般アプリカティブマップ演算子 $(\boxtimes)$ に分解することで，式の見通しを良くすることができる**アプリカティブスタイル**という記法を採用できた．いま変数 $x, y :: \boldsymbol{a}$ があり，引数を文脈に入れる関数 $\varphi, \psi :: \mathsf{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{a} \to \boldsymbol{f_a}$ および 2 引数関数 $\omega :: \boldsymbol{a} \to \boldsymbol{a} \to \boldsymbol{a}$ があるとしよう．すると，アプリカティブスタイルでは次のようにコンテナ変数 u_*, v_* に関数 ω を適用させることができる．

$$\begin{aligned} u_* &= \varphi x \\ v_* &= \psi y \\ w_* &= \omega \circledast u_* \boxtimes v_* \end{aligned} \tag{178}$$

上式をまとめて書くと次のようになる．

$$w_* = \omega \circledast (\psi y) \boxtimes (\varphi x) \tag{179}$$

コンテナ変数 u_*, v_* のいずれかが $\oslash$ であれば式全体の値が $\oslash$ になる．これは 2 個の計算を並列に行って，その結果をそれぞれ u_*, v_* に入れておき，最後に関数 ω に投げるという**制御構造**を具現化したものである．[95]

一方で，我々は直列に計算を行いたい場合もある．文脈が無い場合，その方法は単純な関数合成 $(\circ)$ である．例えば，関数 $f, g, h :: \boldsymbol{a} \to \boldsymbol{a}$ があるとき，式

$$z = (h \circ g \circ f)x \tag{180}$$

は，変数 x に対してまず関数 f を適用し，その結果に関数 g を適用し，さらにその結果に関数 h を適用するという計算を行う．では文脈がある場合はどうだろうか．我々が欲しいのは，文脈なしの変数 x を受け取って，文脈ありの戻り値を返すような関数を合成する新たな演算子である．そのような演算子は**バインド演算子**または**左バインド演算子**と呼ばれる．本書ではバインド演算子を $\heartsuit$ と書く．

関数 φ', ψ', ω' ただし

$$\varphi', \psi', \omega' :: \boldsymbol{a} \to \mathbf{Maybe}_{\boldsymbol{a}} \tag{181}$$

があるとする．変数 x に関数 φ', ψ', ω' を連続して適用しようとすると，次のようになる．

$$\begin{aligned} u_* &= \varphi' x \\ v_* &= \texttt{case}\ u_*\ \texttt{of} \begin{cases} \mathrm{Just}_u \dashrightarrow \psi' u \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\ w_* &= \texttt{case}\ v_*\ \texttt{of} \begin{cases} \mathrm{Just}_v \dashrightarrow \omega' v \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \end{aligned} \tag{182}$$

[95] 式 179 は Hasell の do 記法を用いると $w_* = \texttt{do}\ \{u \leftarrow \varphi x; v \leftarrow \psi y; \langle \omega u v \rangle\}$ と書くことが出来る．

ひとつの式にまとめても，あまり冴えない．

$$
\begin{aligned}
w_* = \texttt{case}\ v_*\ \texttt{of}\ & \begin{cases} \mathrm{Just}_v \dashrightarrow \omega' v \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\
\texttt{where}\ v_* \triangleq \texttt{case}\ u_*\ \texttt{of}\ & \begin{cases} \mathrm{Just}_u \dashrightarrow \psi' u \\ \boxed{} \dashrightarrow \mathrm{Nothing} \end{cases} \\
\texttt{where}\ u_* \triangleq \varphi' x &
\end{aligned}
\tag{183}
$$

やりたいことは変数 x に関数 φ', ψ', ω' を連続的に適用することだけである．もし Python を使っていたら，次のように簡潔に書くところだ．

```python
# Python
try:
  u = phi(x)
  v = psi(u)
  w = omega(v)
except Exception as e:
  print(f"Something went wrong: {e}")
```

このような簡潔さを手に入れるために，我々はバインド演算子 (♡) を導入するのである．関数 φ ただし

$$\varphi :: \boldsymbol{a} \to \boldsymbol{m_a} \tag{184}$$

があるとする．ここに型コンストラクタ $\boldsymbol{m}$ は関手型コンストラクタ $\boldsymbol{f}$ のようなもので，バインド演算子 (♡) が定義される特別な型コンストラクタであるとしよう．

変数 $x_* :: \boldsymbol{m_a}$ が与えられたとき，バインド演算子 (♡) は次のように作用する．

$$
\begin{aligned}
\varphi \heartsuit x_* \mid_{x_* = \langle x \rangle} &= \langle \varphi x \rangle \\
\mid_{\mathrm{otherwise}} &= \varnothing
\end{aligned}
\tag{185}
$$

ここに $\langle \blacksquare \rangle$ はピュア演算子である．また $\varnothing$ は抽象化された空で，型に応じて空リスト ($\boxslash$) または Nothing などになる．

もう一つの関数 ψ ただし

$$\psi :: \boldsymbol{a} \to \boldsymbol{m_a} \tag{186}$$

があるとき，次のように関数を合成することができる．

$$z_* = \psi \heartsuit (\varphi \heartsuit x_*) \tag{187}$$

バインド演算子 (♡) は右結合するため 式 187 は

$$z_* = \psi \heartsuit \varphi \heartsuit x_* \tag{188}$$

と書ける．この式は「変数 x_* に関数 φ を適用し，その結果に関数 ψ を適用する」と読める．[96]

このようなバインド演算子が定義された型クラスのことをモナドと呼び Monad で表す．すべてのアプリカティブ関手はモナドである．上述の型コンストラクタ $\boldsymbol{m}$ はモナド型クラスのインスタンス，すなわち Monad $\supset \boldsymbol{m}$ であった．

♣

[96] Haskell では `z = psi =<< phi =<< x` と書く．なお Haskell プログラマは演算子の左右を入れ替えた `z = x >>= phi >>= psi` という書き方を好む．

我々のバインド演算子は左バインド演算子とも言う．というのは，他に右バインド演算子 $(\vec{\heartsuit})$ もあるからである．右バインド演算子は左バインド演算子の左右を入れ替えたもので，左結合する．式 $z = \psi \heartsuit \varphi \heartsuit x$ は「変数 x に関数 φ を作用させて，さらに関数 ψ を作用させたものを z とする」と読めるが，右バインド演算子を使うと次のようにより自然言語に近い順序になる．

$$z = x \vec{\heartsuit} \varphi \vec{\heartsuit} \psi \tag{189}$$

実際，多くの Haskell プログラマが左バインド演算子よりも右バインド演算子を好む．これは UNIX シェルで次のようにすることと似ているからかもしれない．

```
$ cat x | phi | psi > z
```

上記のコマンドは，ファイル x の中身を書き出し（cat），プログラム φ（phi）が処理をして書き出し，続いてプログラム ψ（psi）が処理をして，ファイル z に書き出すことを意味する．もしプログラム phi または psi が異常終了すれば，この一連のパイプラインも異常終了する．

♣

モナドはアプリカティブ関手を拡張したもので，次の関係が成り立つ．

$$\mathsf{Monad} \subseteq \mathsf{Applicative} \subseteq \mathsf{Functor} \tag{190}$$

関手，アプリカティブ関手のように，型 $\boldsymbol{a}$ をモナドに「入れた」ものを次のように書く．

$$\mathsf{Monad} \supset \boldsymbol{m} \Rightarrow \boldsymbol{m}_{\boldsymbol{a}} \tag{191}$$

関手はマップ演算子 $(\circledast)$ を持つ型クラスであった．アプリカティブ関手はアプリカティブマップ演算子 $(\boxtimes)$ とピュア演算子 $(\langle\blacksquare\rangle)$ を持つ．ピュア演算子を直接使うことはないが，アプリカティブ関手のインスタンスとなる型ではそれぞれ独自の実装を持つ．例えばリスト型は $[\blacksquare]$ というピュア演算子を持つし，Maybe 型は $\mathrm{Just}_{\blacksquare}$ というピュア演算子を持つ．

モナドのインスタンスは追加でバインド演算子 $(\heartsuit)$ を持つことになる．歴史的な理由で，モナドのピュア演算子 $(\langle\blacksquare\rangle)$ のことをリターン（return）演算子と呼ぶが，本質は同じものである．そこで本書では一貫してピュア演算子と呼び続けることにする．[97]

関手は関手則，アプリカティブ関手はアプリカティブ関手則に従う．両方を再掲しておこう．関手則は次のようなものである．

関手則

1. 恒等射の存在（保存）：マップ演算子 $(\circledast)$ は $\mathrm{id} \circledast x_* = \mathrm{id}\, x_*$ を満たす．
2. 結合則：マップ演算子 $(\circledast)$ は $g \circledast (f \circledast x_*) = (g \circ f) \circledast x_*$ を満たす．

♡

アプリカティブ関手則は次のようなものである．

[97] 歴史的理由とは，アプリカティブ関手とモナドがそれぞれ独立して関手の拡張として定義されたことである．そのため，どちらも独立のピュア演算子を持つことになり，モナドがアプリカティブの派生として再定義された現代でも名目上は別の名前が与えられている．

アプリカティブ関手則

1. 恒等射の存在：アプリカティブマップ演算子 $(\boxast)$ は $\langle \mathrm{id} \rangle \boxast x_* = \mathrm{id}\, x_*$ を満たす．
2. 結合則：アプリカティブマップ演算子 $(\boxast)$ は $h \boxast (g \boxast f) = \langle \circ \rangle \boxast h \boxast g \boxast f$ を満たす．
3. 準同型性：$\langle f \rangle \boxast \langle x \rangle = \langle f x \rangle$ または同じことであるが $f \circledast \langle x \rangle = \langle f x \rangle$ である．
4. 交換則：$f \boxast \langle x \rangle = \langle (\diamond \,\S\, x) \rangle \boxast f$ が成り立つ．

♡

アプリカティブ関手は，アプリカティブ関手則のみならず，関手則も満たす．そして，モナドは次節で述べるモナド則と，アプリカティブ関手則，関手則を満たす．

1.9.2 モナド則

モナドは次のモナド則を満たす．ただし $x_* :: \mathsf{Monad} \supset \boldsymbol{m} \Rightarrow \boldsymbol{m_a}$ かつ $\varphi, \psi :: \mathsf{Monad} \supset \boldsymbol{m} \Rightarrow \boldsymbol{a} \to \boldsymbol{m_a}$ である．

モナド則

1. 恒等射（左単位元）の存在：バインド演算子 $(\heartsuit)$ は $\langle \diamond \rangle \heartsuit x_* = x_*$ を満たす．[98]
2. 結合則：バインド演算子 $(\heartsuit)$ は $\psi \heartsuit (\varphi \heartsuit x_*) = (\psi \heartsuit (\varphi \diamond)) \heartsuit x_*$ を満たす．[99]
3. 準同型性（右単位元の存在）：バインド演算子 $(\heartsuit)$ は $\varphi \heartsuit \langle x \rangle = \varphi x$ を満たす．

♡

Haskell には，型クラス Monad が上記のモナド則を満たすことを強制する仕組みはない．しかし，多くのプログラマーがこれらのモナド則を満たすようにモナドを実装する．モナド型クラスがモナド則を満たすことで，多くの有益なテクニックが使えるようになるからである．[14]

♣

アプリカティブ関手則は関手マップ演算子 $(\circledast)$ またはアプリカティブ関手マップ演算子 $(\boxast)$ を用いた関数マップ演算に対して，準同型性を保証するものであった．モナド則はバインド演算子 $(\heartsuit)$ を用いた関数バインド演算に対して，準同型性を保証するものである．

これまで見てきたリスト型はモナド型クラスに属する．そこでリストの準同型性を確認しておこう．いま

$$\varphi :: \boldsymbol{a} \to [\boldsymbol{a}] \tag{192}$$

として

$$f :: \boldsymbol{a} \to \boldsymbol{a} \tag{193}$$

という関数があり，両者が

$$\varphi x = [f x] \tag{194}$$

であるとしよう．このとき

$$\begin{aligned} \varphi \heartsuit [x] &= \varphi x \\ &= [f x] \end{aligned} \tag{195}$$

であり，準同型性が保たれている．

[98] 恒等射の存在における式 $\langle \diamond \rangle \heartsuit x_*$ の左単位元 $(\langle \diamond \rangle \heartsuit)$ は多くの文献で「右単位元」と書かれている．これは Haskell において式 $\langle \diamond \rangle \heartsuit x_* = x_*$ を `x >>= return = x` と書くからである．

[99] モナドの結合則は $\psi \heartsuit \varphi \heartsuit x_* = \psi \heartsuit (\varphi \heartsuit x_*)$ のようにも表現出来る．

Maybe 型もまたモナド型クラスに属する．関数 ψ が

$$\begin{aligned} \psi &:: \boldsymbol{a} \to \mathbf{Maybe}_{\boldsymbol{a}} \\ \psi x &= \mathrm{Just}_{fx} \end{aligned} \tag{196}$$

のとき，

$$\begin{aligned} \psi \heartsuit \mathrm{Just}_x &= \psi x \\ &= \mathrm{Just}_{fx} \end{aligned} \tag{197}$$

であり，準同型性が保たれている．[15], [16]

1.9.3 破壊的代入を隠すモナド

Haskell は一般的に破壊的代入を許さないが，ある条件では許す．一例は関数の参照透過性を保つ場合である．関数の参照透過性とは，関数に同じ引数を与えた場合，同じ値がいつも返ることをいう．つまり，関数は内部に一時変数を持ってもよく，その一時変数に破壊的代入を行っても，関数の参照透過性を破らない限りは許されるということである．C 言語で言えば関数内で `for` ループを持つ場合，Python で言えば関数内でイテレータを使ってループを回す場合がそれにあたる．

このように破壊的代入される変数を関数内に隠す機構をサポートする仕組みが状態トランスフォーマー（State Transformer）モナド型，略して **ST モナド**である．

まず破壊的代入を許すような変数を生成する必要がある．そこで，破壊的代入を許すような変数を生成する演算子を導入しよう．この変数の初期値を x とするとき，我々はこの演算子を $\blacksquare_x$ で表す．[100]

次に，破壊的代入を許す変数から値を取り出す演算子を導入しよう．我々はこの演算子を $\star$ で表す．値を取り出す演算子 $(\star)$ は「抜き身」では使えず，必ずバインド演算子 $(\heartsuit)$ を通して次のように使う．[101]

$$\mu = \star \heartsuit \blacksquare_x \tag{198}$$

ここに結果 μ は ST モナド型の変数で，破壊的代入を許す変数の値ではなく「初期値 x を持つ破壊的代入を許す無名の変数から値を取り出す」行動を意味する．そのため μ のことを **ST アクション**と呼ぶ．なお μ の型を明示すると次のようになる．

$$\mu :: \forall \boldsymbol{s} \mapsto \mathbf{ST}_{\boldsymbol{sa}} \tag{199}$$

ST アクション μ を評価する演算子を導入しよう．我々はこの演算子を $\pentagon$ で表す．次のプログラムは演算子 $\blacksquare$ を使って破壊的代入を許す変数を生成し，その値を取り出す例である．[102]

$$\begin{aligned} x &= 1 \\ \mu &= \star \heartsuit \blacksquare_x \\ \mathbf{main} &= \mathbf{print} \S \pentagon \mu \end{aligned} \tag{200}$$

もちろんこれはつまらないプログラムである．ここで $\star = \backslash\, v \mapsto \star\, v$ であるから，式 198 は次のように書き換えられる．

$$\mu = (\backslash\, v \mapsto \star\, v) \heartsuit \blacksquare_x \tag{201}$$

[100] Haskell では `newSTRef x` と書く．

[101] Haskell では `mu = readSTRef =<< newSTRef x` と書く．

[102] Haskell では `x = 1; mu = readSTRef =<< newSTRef x; main = print $ runST mu` と書く．

この書き換えによって，演算子 $\star$ の引数を陽に表すことが出来る．そこで $\star\, v$ の直前に変数 v に対する破壊的代入を行うこととする．その方法は 2 通りある．ひとつめの方法は次のようなもので，ここでは $\underline{v \mathrel{\overset{\star}{=}} x'}$ で破壊的代入が可能な変数を指し示す変数 v に値 x' を代入している．[103]

$$\mu = (\backslash\, v \mapsto \underline{v \mathrel{\overset{\star}{=}} x'} \gg \star\, v) \heartsuit \blacksquare_x \tag{202}$$

演算子 $\gg$ は「何か（第 1 引数）を行って，その結果を捨てて，次の何か（第 2 引数）の値を返す」という意味である．式 $\underline{v \mathrel{\overset{\star}{=}} x'}$ は「変数 v が指し示す破壊的代入が可能な変数に値 x' を破壊的に代入する」という意味で，我々は式の値には興味がないので捨ててしまうのである．

ふたつめの方法は次のようなものである．ここでは $\underline{f \star v}$ で破壊的代入が可能な変数を指し示す変数 v に関数 f を適用し，その結果で v を書き換える．[104]

$$\mu = \left(\backslash\, v \mapsto \underline{f \star v} \gg \star\, v\right) \heartsuit \blacksquare_x \tag{203}$$

演算子 $\star$ は C++における引数の**参照渡し**を実現するものと思えば良い．式 $\underline{f \star v}$ は値 fv を使って変数 v が指し示す破壊的代入が可能な変数を書き換える．C++との違いは，破壊的代入がこの v に関するラムダ式の中，あるいは同じことであるが ST モナド型の変数 μ の中でしか行えないことである．変数 v で示されている変数を ST モナド μ の外側からアクセスすることはできない．

式 203 を含むプログラムを Haskell で書くと次のようになる．

```haskell
-- Haskell
import Control.Monad
import Control.Monad.ST
import Data.STRef

f :: Num a => a -> a
f x = x + 1

m = (\v -> v `modifySTRef` f >> readSTRef v) =<< newSTRef 1

main = print $ runST m
```

我々の演算子 $\blacksquare$ は Haskell の `newSTRef` であり，我々の演算子 $\star$ は Haskell の `readSTRef` である．また我々の二項演算子 $\star$ は Haskell の `modifySTRef` である．我々の演算子 $\pentagon$ は Haskell の `runST` である．

なお上記のプログラムには登場していないが，我々の破壊的代入演算子 $\overset{\star}{=}$ は Haskell の `writeSTRef` である．

♣

破壊的代入を利用して，リストの和を取る関数 $\sum'$ を書いてみると次のようになる．[17]

$$\sum{}' x_{\mathrm{s}} = \pentagon \S \left(\backslash\, a \mapsto \left(\left(\backslash\, i \mapsto \underline{(+i) \star a}\right) *_{\mathrm{M}} x_{\mathrm{s}}\right) \gg \star a\right) \heartsuit \blacksquare_0 \tag{204}$$

式 204 では，初期値 0 を持つ破壊的代入が可能な変数 $\blacksquare_0$ を作り，変数 a で受け，それに変数 i で取り出したリスト x_{s} の各要素を足して $(+i)$ いき，その結果 $(\star\, a)$ を評価 $(\pentagon)$ している．

式 204 を含む Haskell プログラムは次のようになる．

[103] Haskell では `mu = (\v -> writeSTRef v x' >> readSTRef v) =<< newSTRef x` と書く．
[104] Haskell では `mu = (\v -> modifySTRef v f >> readSTRef v) =<< newSTRef x` と書く．

```haskell
-- Haskell
import Control.Monad
import Control.Monad.ST
import Data.STRef

sum' xs = runST $ (\a -> ((\i -> a `modifySTRef` (+ i)) `mapM_` xs) >> readSTRef a) =<< newSTRef 0

main = print $ sum' [1..100]
```

なお，まったく同じプログラムであるが，次のような **do 記法**を使うことも出来る．こちらのほうが，手続き型プログラミングに慣れたプログラマには読みやすいかもしれない．

```haskell
-- Haskell
import Control.Monad
import Control.Monad.ST
import Data.STRef

sum' xs = runST $ do
    a <- newSTRef 0
    mapM_ (\i -> modifySTRef a (+ i)) xs
    readSTRef v

main = print $ sum' [1..100]
```

ここで用いた do 記法については後述する．また `mapM` およびその亜種の `mapM_` についても後述する．

1.9.4 余談：IO サバイバルキット 3

本書冒頭の例に戻って，単語を数えるプログラムを考えよう．まずは話を簡単にするために，文字列を仮に次のように定義しておこう．

$$s = \texttt{"Hello, world! Hello, once again, to you and you and you."} \tag{205}$$

あとでファイル入力を扱えるようにするが，一旦は文字列 s を入力とする．いま解きたい問題は，文字列 s に含まれる単語を数えることである．

まずは文字列 s からアルファベット以外の文字を取り除く．これにはリストの内包表記を使ったほうが簡単であるが，ここでは内包表記を使わずに関数 clean を次のように定義しておこう．

$$\begin{aligned} \text{clean} &:: \textbf{String} \to \textbf{String} \\ \text{clean}\ \varnothing &= \varnothing \\ \text{clean}\ (x : x_{\mathrm{s}}) &= (\texttt{if}\ \text{isAlpha}\ x\ \texttt{then}\ \text{toLower}\ x\ \texttt{else}\ \texttt{' '}) : \text{clean}\ x_{\mathrm{s}} \end{aligned} \tag{206}$$

関数 clean を文字列 s に適用して文字列 t を得ておく．

$$t = \text{clean}\ s \tag{207}$$

文字列 t には小文字アルファベットと空白のみが含まれている．これを words 関数でリストにしよう．

$$u = \text{words}\ t \tag{208}$$

ここで変数 u は **String** のリスト，つまり [**String**] 型である．

単語のリスト u を sort 関数でソートして v とする．

$$v = \text{sort}\ u \tag{209}$$

変数 v はソートされた単語リストである．リスト v を group 関数でグループ化して w とする．

$$w = \text{group}\ v \tag{210}$$

この group 関数は，リスト v をグループ化してリストのリスト，つまり [[**String**]] 型のリストを返す．

リストのリスト w を count 関数で単語の出現頻度を計算して x とする．

$$x = \text{count}\ w \tag{211}$$

この count 関数は，リストのリストを受け取り，その中の各リストの要素数とリストの先頭要素からなるタプルのリストを返すように，我々自身が定義する必要がある．そこで，次の定義を用いよう．

$$\begin{aligned} \text{count} &:: [[\mathbf{String}]] \to [(\!(\mathbf{Int}, \mathbf{String})\!)] \\ \text{count}\ \varnothing &= \varnothing \\ \text{count}\ (x_{\text{s}} : x_{\text{ss}}) &= (\!(\text{length}\ x_{\text{s}}, \text{head}\ x_{\text{s}})\!) : \text{count}\ x_{\text{ss}} \end{aligned} \tag{212}$$

関数 head の使用は非推奨であるが，単純化のために使うことにする．これで変数 x は単語とその出現頻度のペアを収めたリストとなっている．

今度は，リスト x を出現頻度順にソートする必要がある．関数 sort は順序が定義された型のリストだけをソート対象とするので，任意の型のリストをソート出来る sortBy 関数を使う．この sortBy 関数は，二つの要素を比較する関数を受け取り，その比較結果に従ってリストをソートする．この比較関数を compFst としよう．関数 compFst を次のように定義する．

$$\begin{aligned} \text{compFst} &:: (\!(\mathbf{Int}, \mathbf{String})\!) \to (\!(\mathbf{Int}, \mathbf{String})\!) \to \mathbf{Ordering} \\ \text{compFst}\ (\!(a, \square)\!)(\!(b, \square)\!) &= \text{compare}\ ba \end{aligned} \tag{213}$$

関数 compare は二つの要素を比較して，結果を **Ordering** 型で返す．引数の順序を入れ替えているのは，出現頻度順にソートするためである．

ソートされた結果を次のようにリスト y とする．

$$y = \text{sortBy compFst}\ x \tag{214}$$

最後に，印字しやすいように体裁を整える関数 form を定義する．

$$\begin{aligned} \text{form} &:: [(\!(\mathbf{Int}, \mathbf{String})\!)] \to \mathbf{String} \\ \text{form}\ \varnothing &= \texttt{""} \\ \text{form}\ (x : x_{\text{s}}) &= (\texttt{let}\ (\!(a, b)\!) \triangleq x\ \texttt{in}\ \text{show}\ a \divideontimes \texttt{" "} \divideontimes b) \divideontimes \texttt{""} \divideontimes \text{form}\ x_{\text{s}} \end{aligned} \tag{215}$$

関数 form を使ってリスト y を文字列 z に変換する．

$$z = \text{form}\ y \tag{216}$$

印字には文字列をそのまま書き出すアクション **putStrLn** を用いることとし，これを **main** アクションとする．

$$\mathbf{main} = \mathbf{putStrLn}\ z \tag{217}$$

これで，入力がプログラムに固定されていることをのぞいて，プログラムが完成したことになる．せっかくなので，これまで定義した関数を doIt という名前の関数にまとめておこう．

$$
\begin{aligned}
&\mathrm{doIt} :: \mathbf{String} \to \mathbf{String} \\
&\mathrm{doIt} = \mathrm{form} \circ (\mathrm{sortBy}\ \mathrm{compFst}) \circ \mathrm{count} \circ \mathrm{group} \circ \mathrm{sort} \circ \mathrm{words} \circ \mathrm{clean}
\end{aligned} \tag{218}
$$

この「全部入り」関数 doIt を文字列 s に適用して文字列 z を得ておき，印字することにしよう．これまでの経過をまとめると次のように出来る．

$$
\begin{aligned}
\mathrm{clean} &:: \mathbf{String} \to \mathbf{String} \\
\mathrm{clean}\ \varnothing &= \varnothing \\
\mathrm{clean}\ (x : x_{\mathrm{s}}) &= (\texttt{if}\ \mathrm{isAlpha}\ x\ \texttt{then}\ \mathrm{toLower}\ x\ \texttt{else}\ '\ ') : \mathrm{clean}\ x_{\mathrm{s}} \\
\mathrm{count} &:: [[\mathbf{String}]] \to [(\!(\mathbf{Int}, \mathbf{String})\!)] \\
\mathrm{count}\ \varnothing &= \varnothing \\
\mathrm{count}\ (x_{\mathrm{s}} : x_{\mathrm{ss}}) &= (\!(\mathrm{length}\ x_{\mathrm{s}}, \mathrm{head}\ x_{\mathrm{s}})\!) : \mathrm{count}\ x_{\mathrm{ss}} \\
\mathrm{compFst} &:: (\!(\mathbf{Int}, \mathbf{String})\!) \to (\!(\mathbf{Int}, \mathbf{String})\!) \to \mathbf{Ordering} \\
\mathrm{compFst}\ (\!(a, \square)\!)(\!(b, \square)\!) &= \mathrm{compare}\ ba \\
\mathrm{form} &:: [(\!(\mathbf{Int}, \mathbf{String})\!)] \to \mathbf{String} \\
\mathrm{form}\ \varnothing &= \texttt{""} \\
\mathrm{form}\ (x : x_{\mathrm{s}}) &= (\texttt{let}\ (\!(a, b)\!) \triangleq x\ \texttt{in}\ \mathrm{show}\ a \mathbin{※} \texttt{" "} \mathbin{※} b) \mathbin{※} \texttt{"\textbackslash n"} \mathbin{※} \mathrm{form}\ x_{\mathrm{s}} \\
\mathrm{doIt} &:: \mathbf{String} \to \mathbf{String} \\
\mathrm{doIt} &= \mathrm{form} \circ (\mathrm{sortBy}\ \mathrm{compFst}) \circ \mathrm{count} \circ \mathrm{group} \circ \mathrm{sort} \circ \mathrm{words} \circ \mathrm{clean} \\
s &= \texttt{"Hello, world! Hello, once again, to you and you and you."} \\
z &= \mathrm{doIt}\ s \\
\mathbf{main} &= \mathbf{putStrLn}\ z
\end{aligned} \tag{219}
$$

こうしておけば，中間変数 t, u, v, w, x, y を使わずにプログラムを書くことができる．以上の式を Haskell で書くと次のようになる．

```haskell
import Data.Char
import Data.List

clean :: String -> String
clean "" = ""
clean (x:xs) = (if isAlpha x then toLower x else ' ') : clean xs

count :: [[String]] -> [(Int, String)]
count [] = []
count (xs:xss) = (length xs, head xs) : count xss

compFst :: (Int, String) -> (Int, String) -> Ordering
compFst (a, _) (b, _) = compare b a

form :: [(Int, String)] -> String
form [] = ""
form (x:xs) = (let (a, b) = x in show a ++ " " ++ b) ++ "\n" ++ form xs

doIt :: String -> String
doIt x = (form . (sortBy compFst) . count . group . sort . words . clean) x

s :: String
s = "Hello, world! Hello, once again, to you and you and you."

z :: String
z = doIt s

```

```
main = putStrLn z
```

次のように出力されれば，プログラムは正しく動いている．

```
3 you
2 and
2 hello
1 again
1 once
1 to
1 world
```

最後の仕上げは，関数 doIt をファイルからの入力に適用することである．と言っても，難しいことはなにもない．関数 doIt は文字列 (**String**) を受け取るのだが，ファイルからの入力は参照透過性を持たないので，戻り値を文字列ではなく，**IO 文字列** ($\mathbf{IO}_{\mathbf{String}}$) とする必要があるだけである．IO 文字列については続く節で説明する．そこで，関数 doIt′ という名前のラッパー関数を次のように定義する．

$$\begin{aligned} \mathrm{doIt}' &:: \mathbf{String} \to \mathbf{IO}_{\mathbf{String}} \\ \mathrm{doIt}' x &= \langle \mathrm{doIt}\ x \rangle \end{aligned} \tag{220}$$

関数 doIt′ は元の関数 doIt をピュア演算子でラップして，戻り値を IO 文字列にするだけである．

ファイルからの入力にはアクション **getContents** を用いる．このアクションはファイルからの入力を文字列として返す．アクション **getContents** を我々の doIt′ にバインドして，次のように IO 文字列 z' を得ておこう．

$$\begin{aligned} z' &:: \mathbf{IO}_{\mathbf{String}} \\ z' &= \mathrm{doIt}' \heartsuit \mathbf{getContents} \end{aligned} \tag{221}$$

IO 文字列 z' は文字列ではないので，アクション **putStrLn** に直接渡すことは出来ないが，バインドすることで中身を引き渡すことが出来る．次のように **main** アクションを定義すると，IO 文字列 z' の中身を印字することができる．

$$\mathbf{main} = \mathbf{putStrLn} \heartsuit z' \tag{222}$$

以上でプログラムが完成した．関数 `doIt` が定義されているとして，残りを Haskell で書くと次のようになる．[105]

```
doIt' :: String -> IO String
doIt' x = pure (doIt x)

z' :: IO String
z' = doIt' =<< getContents

main = putStrLn =<< z'
```

Haskell Stack を使っている場合，次のように実行できる．

```
$ stack run < input.txt
```

[105] 別解として `z' = doIt `liftM` getContents` と書くこともできる．

1.9.5 この章のまとめ

- アプリカティブスタイルは計算構造の一種である，並列した計算を表す．
- 純粋な関数合成も計算構造の一種である，直列した計算を表す．ただし純粋な関数合成は文脈を扱えない．
- モナド型クラスとバインド演算子 (♡) は一般的な計算構造を表現できる，より強力な仕組みである．
- モナド型クラスに属する型は関手則，アプリカティブ関手則に加えてモナド則も満たすように設計されなければならない．モナド則とはバインド演算子版の恒等射の存在，結合則，準同型性である．
- モナド型クラスのひとつ ST モナド型を用いると，破壊的代入をプログラムの他の部分から隠すことが出来る．

const 修飾子と mut 修飾子

Haskell では全ての変数に対して破壊的代入が禁止されている．例外は ST モナド型や IO モナド型の内部で行われる破壊的代入程度である．一方，C 言語ではデフォルトで変数への破壊的代入が許可されている．そこで「うっかり」破壊的代入をしないように，変数の型に `const` 修飾子をつけることが出来る．例えば `main` 関数の第 2 引数 `argv` を無意味に破壊しないように，次のコードのように `const` 修飾子を埋め込むことはよく行われている．

```
/* C */
#include <stdio.h>

int main(int argc, const char *const *argv) {
  while (--argc > 0) {
    ++argv;
    printf("%s\n", *argv);
  }
  return 0;
}
```

C 言語よりも新しいプログラミング言語である Rust では，変数はデフォルトで変更不可能になっており，変更可能な変数の型には `mut` 修飾子を付けなければならない．Haskell のように破壊的代入を禁止するよりも，Rust のほうが現実的な選択かもしれない．

1.10 IO

実用的なプログラムには必ず入出力（IO）がある．ところが IO とは参照透過性を持たない行動であり，関数型プログラミングの世界観とは相容れない．そこで，Haskell では IO をモナド型クラスで表現する．IO という「破壊活動」をモナド型クラスのインスタンスの変数に閉じ込めて，プログラムの他の部分と分離するのである．[106]

1.10.1 IO モナド

今後，モナド型クラスに属する型の変数のことを単に**モナド**と呼ぶ．またモナド型クラスではない型の変数を受け取ってモナドを返す関数を「**文脈**に入れる関数」と呼ぶ．モナド型クラスに属する型は**バインド演算子** (♡) を持つので，モナドに対してバインド演算子を用いて文脈に入れる関数を適用することができる．

ここで改めて概念を整理しておこう．表 5 に示すように，**関手型クラス** (Functor) に属する型はマップ演算子 (⊛) を持つ．**アプリカティブ関手型クラス** (Applicative) に属する型はマップ演算子に加えてアプリカティブマップ演算子 (⊠) とピュア演算子 (⟨■⟩) を持つ．**モナド型クラス** (Monad) に属する型は，マップ演算子，アプリカティブマップ演算子，ピュア演算子に加えて，バインド演算子 (♡) を持つ．

型クラス	特徴
関手	関手マップ演算子 (⊛) を持つ
アプリカティブ関手	関手の特徴に加えて…… アプリカティブマップ演算子 (⊠) とピュア演算子 ⟨■⟩ を持つ
モナド	アプリカティブ関手の特徴に加えて…… バインド演算子 (♡) を持つ

表 5: 関手・アプリカティブ関手・モナド

これまで，マップ演算子がループを，アプリカティブマップ演算子が並列な計算を，バインド演算子が直列な計算を表現できることを見てきた．またバインド演算子を用いて，破壊的代入をモナドの中に隠す方法を見てきた．

モナドとバインド演算子はまた，参照透過性を持たない入出力 (IO) をプログラムに組み込むのにも使える．

♣

まず **IO 型** (IO) を導入する．IO 型はモナド型クラスのインスタンスである．Haskell では IO 型がモナド型クラスのインスタンスであることを次のように宣言している．[107]

$$\texttt{instance Monad} \supset \mathbf{IO} \tag{223}$$

IO 型はモナド型クラスのインスタンスであるので，ピュア演算子とバインド演算子を引き継いでいる．また IO 型はリスト型と同じく型パラメタを必要とし $\mathbf{IO}_a$ のような形で使われる．

[106] Haskell では型クラス（type class）のインスタンス（instance）」は型（type）のことである．C++や Java ではインスタンスがオブジェクト（object）を意味するので，混同しないように気をつけよう．

[107] Haskell では `instance Monad IO` と表現される．

ファイルを丸ごと読み込むアクションは **getContents** である．アクション **getContents** の型は

$$\mathbf{getContents} :: \mathbf{IO}_{\mathbf{String}} \tag{224}$$

である．アクション **getContents** をバインド (♡) すると，ファイルの内容を文字列として得ることができる．例を挙げる．

$$\begin{aligned} f &:: \mathbf{String} \to \mathbf{String} \\ fx &= \texttt{"My mother said: "} \divideontimes x \\ f' &:: \mathbf{String} \to \mathbf{IO}_{\mathbf{String}} \\ f'x &= \langle fx \rangle \\ s &:: \mathbf{IO}_{\mathbf{String}} \\ s &= f' \heartsuit \mathbf{getContents} \end{aligned} \tag{225}$$

式 225 はファイル内容の先頭に一言付け加えて変数 s に代入するプログラムである．関数 f は文字列を受け取って文字列を返す参照透過性のある関数である．このような関数は「ピュアすぎて」アクションとバインド (♡) することが出来ない．そこで，関数 f をピュア演算子でラップして，戻り値を IO 文字列にする関数 f' を定義している．

アクション **getContents** は標準入力から内容を受け取るが，任意のファイルから内容を読むにはアクション **readFile** を用いる．アクション **readFile** は

$$\begin{aligned} s' &:: \mathbf{IO}_{\mathbf{String}} \\ s' &= \mathbf{readFile}\ \texttt{"input.txt"} \end{aligned} \tag{226}$$

のように使う．

♣

出力とは，破壊的代入である．そこで出力も IO モナドで表現する必要がある．出力によく使われるアクションは文字列を印字する **putStrLn** である．アクション **putStrLn** の型は

$$\mathbf{putStrLn} :: \mathbf{String} \to \mathbf{IO}_{()} \tag{227}$$

である．これはアクション **putStrLn** が文字列 (**String**) を受け取って，何らかの破壊的操作を行って，空っぽの IO アクション $\left(\mathbf{IO}_{()}\right)$ を返すことを意味する．[18]

アクション **putStrLn** に文字列を渡すと，その文字列を標準出力へ印字する．例を挙げる．

$$\mathbf{main} = \mathbf{putStrLn}\ \texttt{"Hello, world!"} \tag{228}$$

ここで文字列型 (**String**) を IO モナドに包んで返す関数 φ があるとしよう．例えば，φ が次のような関数であるとする．

$$\varphi s = \langle \texttt{"I said: "} \divideontimes s \rangle \tag{229}$$

ここに $\langle \blacksquare \rangle$ はピュア演算子である．このような関数でも，バインド演算子 (♡) を用いて，文字列を印字することができる．例を挙げる．

$$\mathbf{main} = \mathbf{putStrLn} \heartsuit (\varphi\ \texttt{"Hello, world!"}) \tag{230}$$

このように，IO モナドに包まれた関数をバインド (♡) すると，文字列を印字することができる．

型クラス Show に属する型は，値を文字列に変換する関数 show を提供する．アクション **print** は，型クラス Show に属する型の値を受け取って，その値を文字列に変換して，標準出力へ印字する．例を挙げる．

$$\begin{aligned} x &:: \mathbf{Int} = 1 \\ \mathbf{main} &= \mathbf{print}\ x \end{aligned} \tag{231}$$

この例では，型 **Int** が型クラス Show のインスタンスなので，アクション **print** に直接渡すことができる．

♣

リストに対する IO アクションはマップできる．ただし

$$\begin{aligned} x_{\mathrm{s}} &= [0, 1, ..., 9] \\ f_{\mathrm{s}} &= \mathbf{print} * x_{\mathrm{s}} \end{aligned} \tag{232}$$

としても $\mathbf{main} = f_{\mathrm{s}}$ と書くことはできない．これは関数リスト f_{s} が IO アクションではないからである．関数リスト f_{s} を IO アクションに変換するには，関数 sequence を用いて

$$\mathbf{main} = \mathrm{sequence}\ f_{\mathrm{s}} \tag{233}$$

とする．この関数リスト f_{s} を作る代わりに，次のような IO アクションを作ることが出来る．

$$\mu = \mathbf{print} *_{\mathrm{M}} x_{\mathrm{s}} \tag{234}$$

この μ は IO アクションなので $\mathbf{main} = \mu$ とすることが出来る．なお μ の値は **print** の戻り値すなわち $\emptyset$ のリスト $[\emptyset, \emptyset, ..., \emptyset]$ になる．戻り値を捨てても良い場合は

$$\mu = \mathbf{print} *_{\mathrm{M_}} x_{\mathrm{s}} \tag{235}$$

とする．[108]

1.10.2 疑似乱数

入力と似た概念に**疑似乱数**がある．疑似乱数は計算機が擬似的に発生させる乱数で，事前に値がわからない点がユーザからの入力と共通する．疑似乱数は計算機の状態に依存するので，参照透過性を持たない．次の例は変数 r に疑似乱数を代入するものである．[19]

$$r = \mathbf{randomIO} :: \mathbf{IO_{Float}} \tag{236}$$

この変数 r の値はプログラム実行時にランダムに定まる「汚染された」値である．そのため，プログラム中の他の参照透過性を持つ変数と混ぜて扱うことが出来ない．変数 r は IO モナドに包まれているので，バインド演算子 $(\heartsuit)$ を使って，文脈に入れる関数の中で使うことが出来る．一例は **print** アクションで，次の式は疑似乱数を印字するものである．

$$\mathbf{main} = \mathbf{print} \heartsuit r \tag{237}$$

繰り返し演算子の文脈ありバージョン $(\circlearrowleft_M)$ を用いると次のように**擬似乱数列**を生成できる．[109]

$$r_{\mathrm{s}} = 5 \circlearrowleft_M (\mathbf{randomIO} :: \mathbf{IO_{Float}}) \tag{238}$$

Haskell は指定された範囲の疑似乱数を生成するアクション **randomRIO** も提供している．次の例は 6 面体のサイコロを振るものである．

[108] Haskell では `mu = print `mapM_` xs` と書く．
[109] Haskell では `rs = n `replicateM` (randomIO :: IO Float)` と書く．

$$r' = \mathbf{randomRIO}\ (\!(1, 6 :: \mathbf{Int})\!) \tag{239}$$

このとき，変数 r' の値は 1 から 6 までの整数のいずれかである．

1.10.3 main アクション

Haskell では関数はすべて参照透過性を持つことが期待されている．そこで **putStrLn** のようなアクションをどのように解釈するか，改めて触れておく．アクション **putStrLn** の型は

$$\mathbf{putStrLn} :: \mathbf{String} \to \mathbf{IO}_{(\!)} \tag{240}$$

である．これはアクション **putStrLn** が文字列 (**String**) を受け取って，何らかの動作を行う「命令書」すなわち IO アクションと，付随する戻り値があることを意味する．ただし付随する戻り値のほうはユニット型 ($(\!)$) で，ユニット型の唯一の値がユニット ($\emptyset$) であるため，事実上戻り値は無いことになる．また命令書はプログラム実行時に破壊活動を行うのであるが，命令書自体は変化しないため，参照透過性を持つ．

Haskell は最初に実行されるアクションがあり，そのアクションを **main** で表している．この **main** アクションは何も受け取らず命令書の実行だけをし，何も返さない．そこで **main** の型は

$$\mathbf{main} :: \mathbf{IO}_{(\!)} \tag{241}$$

と決められている．

最も単純な **main** アクションは次のようになる．

$$\mathbf{main} = \langle \emptyset \rangle \tag{242}$$

この **main** アクションは何も受け取らず，何もせず，何も返さない．ただし Haskellランタイムは Haskell プログラム終了時に OS に対して戻り値を返す．この戻り値は，明示的に指示しない限り，プログラムの正常終了を意味する 0 である．特定の数値を戻り値として返したい場合は，アクション **exitWith** が必要になるが，これは別の章で取り上げることにする．

アクション **putStrLn** は文字列 (**String**) を受け取って，その文字列を標準出力へ印字する．このアクションを **main** として定義すると，次のようになる．

$$\mathbf{main} = \mathbf{putStrLn}\ \texttt{"Hello, world!"} \tag{243}$$

このプログラムを実行すると，標準出力に “Hello, world!” が印字される．

アクション **getContents** は標準入力から内容を受け取って，その内容を IO 文字列 ($\mathbf{IO}_{\mathbf{String}}$) として返す．アクション **putStrLn** は引数に文字列しか取れないので，もし **putStrLn** を使って標準入力の内容を印字しようと思うと，バインド演算子 ($\heartsuit$) を用いて

$$\mathbf{main} = \mathbf{putStrLn} \heartsuit \mathbf{getContents} \tag{244}$$

とすることになる．

任意の関数 f ただし $f :: \mathbf{String} \to \mathbf{String}$ をアクション **putStrLn** と **getContents** の間に挟む場合は

$$\mathbf{main} = \mathbf{putStrLn} \heartsuit (\backslash\, x \mapsto \langle f x \rangle) \heartsuit \mathbf{getContents} \tag{245}$$

とする．また文脈に入れる関数 $g :: \mathbf{String} \to \mathbf{IO}_{\mathbf{String}}$を使う場合は単に

$$\mathbf{main} = \mathbf{putStrLn} \heartsuit g \heartsuit \mathbf{getContents} \tag{246}$$

でよい．[20]

1.10.4 余談：do 記法

標準入力から 1 行を文字列として読み取って，その文字列を標準出力に書き出すだけのプログラムを Python で書くならば，次のようになるだろう．

```
# Python
import sys
def main():
  s = sys.stdin.readline()
  print(s)
main()
```

このプログラムは，標準入力から `readline()` で 1 行を読み取り，変数 `s` に代入したあと `print(s)` で標準出力へ書き出している．このような書き方を**逐次処理**あるいは**逐次実行**と呼ぶ．この書き方はプログラマの直観ともよく合致するため，多くのプログラミング言語が採用するところである．むしろ逐次処理以外のプログラミングを考えられないというプログラマもいるであろう．

Haskell にはシンタックスシュガーとして逐次処理記法が導入されている．それが**do 記法**である．標準入力から 1 行を読み取って変数 s に代入し，それを標準出力に書き出す Haskell プログラムを我々はこれまで

$$\mathbf{main} = (\backslash\, s \mapsto \mathbf{putStrLn}\ s) \heartsuit \mathbf{getLine} \tag{247}$$

と書くことにしてきた．ここに変数 s はアクション **getLine** から得られる文字列を陽に示すためにラムダ式のパラメタとして書き出したものである．この式を，キーワード `do` を用いて次のように書き直すことが出来る．この記法は「まず **getLine** の結果を s に代入し，その結果の s を **putStrLn** する」と読める．

$$\mathbf{main} = \texttt{do}\ \{s \leftarrow \mathbf{getLine}; \mathbf{putStrLn}\ s\} \tag{248}$$

式 248 を Haskell プログラムで書くと次のようになる．

```
-- Haskell
main = do { s <- getLine; putStrLn s }
```

改行を用いた記法にすると，この Haskell プログラムは次のようにより Python に近い書き方になる．

```
-- Haskell
main = do        -- In Python
  s <- getLine   -- s = readline()
  putStrLn s     -- print(s)
```

標準入力から受け取った文字列を関数 f で処理する場合，Python ならば次のようなプログラムになるであろう．

```
# Python
import sys
def main():
```

```
  s = sys.stdin.readline()
  t = f(s)
  print(t)
main()
```

同じことを Haskell で書くと次のようになる．

```
--Haskell
-- f :: String -> String
main = do { s <- getLine; let t = f s in putStrLn t }
```

改行を用いた記法にすると，この Haskell プログラムは次のようにより Python に近い書き方になる．

```
--Haskell
-- f :: String -> String
main = do        -- In Python
  s <- getLine   -- s = readline()
  let t = f s    -- t = f(s)
  putStrLn t     -- print(t)
```

Haskell プログラムで do 記法を採用するかどうかは

$$\mathbf{main} = (\backslash\, s \mapsto \mathbf{putStrLn} \heartsuit f s) \heartsuit \mathbf{getLine} \tag{249}$$

と書くか

$$\mathbf{main} = \texttt{do}\ \{s \leftarrow \mathbf{getLine}; \texttt{let}\ t \triangleq f s\ \texttt{in}\ \mathbf{putStrLn}\ t\} \tag{250}$$

と書くかの違いでしかない．プログラマは好きな方を採用すればよいのである．

♣

do 記法の利点のひとつに**継続渡しスタイル**（continuation passing style）を簡潔に書けることがある．継続渡しスタイルとは，関数に戻り値の返し先すなわち**継続**を引数として渡すスタイルである．[21]

次の例は，継続渡しではない通常の関数である．継続渡しでない書き方を**直接スタイル**（direct style）と呼ぶ．関数 sqr は引数の自乗を，関数 add はふたつの引数の和を計算する．

$$\begin{aligned} \mathrm{sqr} &:: \mathbf{Double} \to \mathbf{Double} \\ \mathrm{sqr}\ x &= x \times x \\ \mathrm{add} &:: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Double} \\ \mathrm{add}\ xy &= x + y \end{aligned} \tag{251}$$

これらの関数と標準関数 sqrt を使って，三平方の定理を計算する関数 pyth を次のように定義する．

$$\begin{aligned} \mathrm{pyth} &:: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Double} \\ \mathrm{pyth}\ xy &= \mathrm{sqrt}\ (\mathrm{add}\ (\mathrm{sqr}\ x)(\mathrm{sqr}\ y)) \end{aligned} \tag{252}$$

継続渡しとは，関数の戻り値の返し先を引数として受け取るものである．この戻り値の返し先，すなわち継続を関数 c で表すことにしよう．そうすると sqr と add は次のようになる．関数の & は継続渡しスタイルを表す．

$$\begin{aligned}
\mathrm{sqr}_{\&} &:: \mathbf{Double} \to (\mathbf{Double} \to \boldsymbol{a}) \to \boldsymbol{a} \\
\mathrm{sqr}_{\&} xc &= c(x \times x) \\
\mathrm{add}_{\&} &:: \mathbf{Double} \to \mathbf{Double} \to (\mathbf{Double} \to \boldsymbol{a}) \to \boldsymbol{a} \\
\mathrm{add}_{\&} xyc &= c(x + y)
\end{aligned} \tag{253}$$

標準関数 sqrt の継続渡し版は次のようになる．

$$\begin{aligned}
\mathrm{sqrt}_{\&} &:: \mathbf{Double} \to (\mathbf{Double} \to \boldsymbol{a}) \to \boldsymbol{a} \\
\mathrm{sqrt}_{\&} xc &= c(\mathrm{sqrt}\ x)
\end{aligned} \tag{254}$$

なお

$$\begin{gathered}
\mathrm{sqrt}_{\&} xc = c(\mathrm{sqrt}\ x) \\
\Updownarrow \\
\mathrm{sqrt}_{\&} x = \backslash\, c \mapsto c(\mathrm{sqrt}\ x)
\end{gathered} \tag{255}$$

なので，どちらの書き方をしても良い．

継続渡しスタイルで三平方の定理の計算を行うには次のようになる．

$$\begin{aligned}
\mathrm{pyth}_{\&} &:: \mathbf{Double} \to \mathbf{Double} \to (\mathbf{Double} \to \boldsymbol{a}) \to \boldsymbol{a} \\
\mathrm{pyth}_{\&} xyc &= \mathrm{sqr}_{\&} x(\backslash\, x' \mapsto \mathrm{sqr}_{\&} y(\backslash\, y' \mapsto \mathrm{add}_{\&} x'y'(\backslash\, z' \mapsto \mathrm{sqrt}_{\&} z'c)))
\end{aligned} \tag{256}$$

この計算の結果を印字するには次のようにする．

$$\mathbf{main} = \mathbf{print}\ (\mathrm{pyth}_{\&} 3.0\ 4.0\ \ \mathrm{id}) \tag{257}$$

結果は 5.0 である．

さて, 話をややこしくしただけに見える継続渡しスタイルであるが, 利点がある. その前に, 継続渡しスタイルをより簡潔に表現する方法を見ておこう．それには**継続モナド**を使う．

継続モナド版の sqr, add, sqrt は次のようになる．

$$\begin{aligned}
\mathrm{sqr}_{\mathrm{m}} &:: \mathbf{Double} \to \mathbf{Cont}_{\boldsymbol{a}\ \mathbf{Double}} \\
\mathrm{sqr}_{\mathrm{m}} x &= \langle x \times x \rangle \\
\mathrm{add}_{\mathrm{m}} &:: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Cont}_{\boldsymbol{a}\ \mathbf{Double}} \\
\mathrm{add}_{\mathrm{m}} xy &= \langle x + y \rangle \\
\mathrm{sqrt}_{\mathrm{m}} &:: \mathbf{Double} \to \mathbf{Cont}_{\boldsymbol{a}\ \mathbf{Double}} \\
\mathrm{sqrt}_{\mathrm{m}} x &= \langle \mathrm{sqrt}\ x \rangle
\end{aligned} \tag{258}$$

ここに **Cont** は継続モナド型コンストラクタである．

継続を意味する引数 c が隠されたので，簡潔になった．これらの継続モナド版を使って三平方の定理を計算する関数も作っておこう．ここで do 記法の出番である．do 記法を使うと，次のようになる．

$$\begin{aligned}
\mathrm{pyth}_{\mathrm{m}} &:: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Cont}_{\boldsymbol{a}\ \mathbf{Double}} \\
\mathrm{pyth}_{\mathrm{m}} xy &= \mathtt{do}\ \{x' \leftarrow \mathrm{sqr}_{\mathrm{m}} x; y' \leftarrow \mathrm{sqr}_{\mathrm{m}} y; z' \leftarrow \mathrm{add}_{\mathrm{m}} x'y'; z'' \leftarrow \mathrm{sqrt}_{\mathrm{m}} z'; \langle z'' \rangle\}
\end{aligned} \tag{259}$$

もうほとんど逐次処理に見えないだろうか．どうしても逐次処理をしたいのであれば Python のような他の言語を使う方が簡単ではあるが，Haskell でも出来るのである．

関数 $\mathrm{pyth_m}$ の戻り値は継続モナドであるから，それを印字するには専用の演算子 $\mathrel{\leftarrow\!\!\!\rho}$ を用いて次のようにする．

$$\mathbf{main} = \mathbf{print} \mathrel{\leftarrow\!\!\!\rho} (\mathrm{pyth_m} 3.0\ 4.0) \tag{260}$$

または，逆方向の演算子 $\mathrel{\rho\!\!\!\rightarrow}$ を用いて次のようにする．[110]

$$\mathbf{main} = (\mathrm{pyth_m} 3.0\ 4.0) \mathrel{\rho\!\!\!\rightarrow} \mathbf{print} \tag{261}$$

わざわざ継続渡しスタイルを用いるのは**カレント継続**（current continuation）を使うためである．[111]

カレント継続は，ある関数から見て，次に実行されるべき継続を表す．式 256 のような継続渡しスタイルであれば，渡された継続が次に実行されるべき継続であるが，一般の関数からは次に実行されるべき継続が見えない．次に実行すべき継続を ♪ で表すことにすると $\mathrm{sqr}_\&, \mathrm{add}_\&, \mathrm{sqrt}_\&$ は次のようになる．

$$\begin{aligned} \mathrm{sqr}_\& x ♪ &= ♪(x \times x) \\ \mathrm{add}_\& xy ♪ &= ♪(x + y) \\ \mathrm{sqrt}_\& x ♪ &= ♪(\mathrm{sqrt}\ x) \end{aligned} \tag{262}$$

カレント継続 ♪ を自動的に生成する演算子 ♫ を考える．この演算子 ♫ は関数 f を受け取り，次のように $f♪$ を返す．

$$♫ f = f ♪ \tag{263}$$

ここで注意しないといけないことがある．記号 ♪ はカレント継続を表す記号であるが，これはプログラマから見ることは出来ず，またプログラム中で動的に変化する．一方で，カレント継続を与える演算子 ♫ は実在する演算子で，プログラマが書くことが出来る．[112]

カレント継続は ♫ 演算子に続く関数に引数として渡されるので，次のようにラムダ式で受け取ることが出来る．

$$♫(\backslash\, q \mapsto \ldots) \tag{264}$$

この式では変数 q にはカレント継続が拘束される．本書ではシンタックスシュガーとして次の記法も導入しておこう．カレント継続を引数として渡してやる演算子 $\mathrel{\leftarrow\!\!\!\backslash}$ を導入する．演算子 $\mathrel{\leftarrow\!\!\!\backslash}$ はラムダ式と同じように

$$\mathrel{\leftarrow\!\!\!\backslash}\, q \mapsto \ldots \tag{265}$$

として使う．ただし引数 q には実行環境がカレント継続を拘束する．一般のラムダ式のように，プログラマが引数の値を与える必要はない．[113]

引数 q はカレント継続なので，関数 f の中で呼び出すということは，関数 f からの脱出を意味する．これを利用して，関数からの**早期リターン**のようなテクニックを実現することも可能である．例えば $\mathrm{pyth_m} xy$ を変形して，もし $x < 0$ または $y < 0$ であれば戻り値 0.0 で早期リターンするようには次のようにする．

---

[110] Haskell では $\mathrel{\rho\!\!\!\rightarrow}$ を `runCont` 関数を用いて書く．例えば $\mathbf{main} = (\mathrm{pyth_m} 3.0\ 4.0) \mathrel{\rho\!\!\!\rightarrow} \mathbf{print}$ は `main = (pyth_m 3.0 4.0) `runCont` print` と書く．

[111] Current continuation の日本語訳はまだ決まっていないようである．日本のプログラマは current continuation をしばしば CC と略すほか「現在の継続」と呼ぶこともある．本書では current continuation を「カレント継続」と訳すことにした．

[112] Haskell では ♫ を `callCC` と書く．

[113] Haskell では $\mathrel{\leftarrow\!\!\!\backslash}$ に直接対応する演算子は無い．しかし $\mathrel{\leftarrow\!\!\!\backslash}\, q \mapsto f$ は `callCC $ \q -> f` と書ける．

$$\begin{aligned}\mathrm{pyth}_{\mathrm{cc}} &:: \mathbf{Double} \to \mathbf{Double} \to \mathbf{Cont}_{a\ \mathbf{Double}} \\ \mathrm{pyth}_{\mathrm{cc}} xy &= \backslash q \mapsto \texttt{do}\ \{\mathrm{when}\ (x \leq 0 \vee y \leq 0)(q\ 0.0); \\ &\qquad x' \leftarrow \mathrm{sqr}_{\mathrm{m}} x; y' \leftarrow \mathrm{sqr}_{\mathrm{m}} y; z' \leftarrow \mathrm{add}_{\mathrm{m}} x' y'; z'' \leftarrow \mathrm{sqrt}_{\mathrm{m}} z'; \langle z'' \rangle\}\end{aligned} \tag{266}$$

ここに関数 when は第 1 引数が真であれば第 2 引数を実行する関数である．関数 when の第 2 引数と戻り値はモナド型クラスのインスタンスである必要がある必要があるため，モナドの中でしか利用できない．

関数 $\mathrm{pyth}_{\mathrm{cc}}$ を使ったプログラムを Haskell で書くと次のようになる．

```haskell
import Control.Monad.Cont
sqr_m :: Float -> Cont a Float
sqr_m x = pure (x * x)

add_m :: Float -> Float -> Cont a Float
add_m x y = pure (x + y)

sqrt_m :: Float -> Cont a Float
sqrt_m x = pure (sqrt x)

pyth_cc :: Float -> Float -> Cont a Float
pyth_cc x y = callCC $ \q -> do
  when (x <= 0 || y <= 0) (q 0.0)
  x' <- sqr_m x
  y' <- sqr_m y
  z' <- add_m x' y'
  z'' <- sqrt_m z'
  pure z''

main = (pyth_cc 5 12) `runCont` print
```

1.10.5 この章のまとめ

- Haskell における入出力（IO）は IO 型を通して行われる．IO 型は IO モナド型クラスのインスタンスである．
- IO 型の戻り値を返す関数を IO アクションと呼ぶ．
- ファイルの読み込みは $\mathbf{getContents} :: \mathbf{IO}_{\mathbf{String}}$ で行うことが出来る．
- バインド演算子 (♡) を用いると，IO アクションを直列につなげることが出来る．
- 疑似乱数も IO 型を通して取得する．
- main アクションも IO 型である．
- do 記法を使うと，バインド演算子を用いた式を手続き型プログラミング風に書くことが出来る．
- do 記法とカレント継続を組み合わせると，プログラムはより手続き型風になる．

作用と副作用

関数 $z = fx$ の参照透過性を保ったまま，関数 f に副作用を持たせるには次のように定義すると良い.

$$(\!|z, w'|\!) = f(\!|x, w|\!) \tag{267}$$

引数 x に注目すると $z = fx$ の関係性は保たれており，一方で w で表現される内部状態は関数 f の実行後に w' に変化するものと理解することが出来る.

そのため, 副作用を隠すためにはモナド型クラスはなくても良い. モナド型クラスを導入する理由は関数合成と，逐次実行を担保するためである．[22]

2 Haskell プログラミング

プログラムとは，動けば正義である．プログラミング言語は機械語の糖衣構文であるし，計算機はチューリング機械のバリエーションである．そのためプログラムの記述方法はプログラムの本質ではない．ただし，人間がプログラムを読むことを想定すると，プログラムをどれやけ優しく読めるようにするかは大きな議題である．

2.1 Hello, again.

2.2 データ型

2.3 文字列・ベクトル

2.4 末尾再帰

2.5 Y コンビネータ

2.6 main の引数と戻り値

2.7 継続

2.8 状態

2.9 テンプレート

2.10 他の言語との比較

3 プログラミングと代数構造

3.1 モノイド

3.2 圏論

3.3 関数型

3.4 関手・アプリカティブ関手

3.5 モナド

3.6 非決定生

3.7 カテゴリ

3.8 アロー

3.9 クライスリカテゴリ

3.10 圏論とプログラミングをつなぐ

4 執筆メモ

$(\mathbb{M}, \heartsuit, \langle\diamond\rangle)$ はモノイドである．

—

4.0.1 モノイド

📍 **TK**

整数全てからなる**集合**を $\mathbb{Z}$ で表すことにする．計算機科学で整数と言うと，本当の整数と，例えば -2^{63} から $2^{63}-1$ までの間の整数の意味と両方あるが，今は前者の意味である．

集合 $\mathbb{Z}$ の任意の**元**（**要素**）z を $z \in \mathbb{Z}$ と書く．

二つの整数 $z_1, z_2 \in \mathbb{Z}$ があるとしよう．両者の間には**足し算** $(+)$ が定義されており，その結果すなわち**和**もまた整数である．ここで $z_1 + z_2 \in \mathbb{Z}$ であるとき，演算子 $+$ が集合 $\mathbb{Z}$ に対して**全域性**を持つと言う．

一般に集合 $\mathbb{A}$ の元に対して二項演算子 $\circledcirc$ が定義されていて，$a_1, a_2 \in \mathbb{A}$ のときに

$$a_1 \circledcirc a_2 \in \mathbb{A} \tag{268}$$

である場合，つまり演算子 $\circledcirc$ が集合 $\mathbb{A}$ に対して全域性を持つ場合，組み合わせ $(\mathbb{A}, \circledcirc)$ を**マグマ**と呼ぶ．組み合わせ $(\mathbb{Z}, +)$ はマグマの例であり，$(\mathbb{Z}, \times)$ もマグマの例である．

他に**論理集合** $\mathbb{B} = \{\top, \bot\}$ に対して，**論理和** $(\vee)$ は全域性を持つから，組み合わせ $(\mathbb{B}, \vee)$ はマグマであるし，同様に**論理積** $(\wedge)$ も全域性を持つから，組み合わせ $(\mathbb{B}, \wedge)$ もマグマである．論理集合 $\mathbb{B}$ とは論理型 **Bool** を数学風に言い換えたものである．[114]

マグマのうち，演算を 2 回続ける場合，その順序によって結果が異ならない，つまり

$$a_1 \circledcirc (a_2 \circledcirc a_3) = (a_1 \circledcirc a_2) \circledcirc a_3 \tag{269}$$

ただし $a_1, a_2, a_3 \in \mathbb{A}$ のとき，組み合わせ $(\mathbb{A}, \circledcirc)$ のことを**半群**と呼ぶ．この〜〜〜で表される性質を**結合性**と呼ぶ．組み合わせ $(\mathbb{Z}, +), (\mathbb{Z}, \times), (\mathbb{B}, \vee), (\mathbb{B}, \wedge)$ はすべて半群である．

ところで，整数全体の集合 $\mathbb{Z}$ には特別な元 $0 \in \mathbb{Z}$ がある．この元 0 は $z \in \mathbb{Z}$ のとき

$$0 + z = z + 0 = z \tag{270}$$

という性質を持つ．この 0 を演算 $+$ における**単位元**と呼ぶ．足し算のことを**加法**とも言うので 0 のことは**加法単位元**と呼ぶこともあるし，文字通り**零元**と呼ぶこともある．

一般に，$a \in \mathbb{A}$ として

$$0 \circledcirc a = a \circledcirc 0 = a \tag{271}$$

であるとき，元 0 を単位元と呼ぶ．[115]

組み合わせ $(\mathbb{A}, \circledcirc)$ が半群のとき，集合 $\mathbb{A}$ の単位元 0 との組み合わせ $(\mathbb{A}, \circledcirc, 0)$ のことを**モノイド**または**単位的半群**と呼ぶ．例えば $(\mathbb{Z}, +, 0)$ はモノイドであるし，$(\mathbb{Z}, \times, 1), (\mathbb{B}, \vee, \bot), (\mathbb{B}, \wedge, \top)$ もモノイドである．

[114] 記号 $\top$ は True を，記号 $\bot$ は False を抽象化したものである．記号 $\wedge$ は and と読み，記号 $\vee$ は or と読む．

[115] 厳密には $0_{\text{left}} \circledcirc a = a$ のとき 0_{left} を**左単位元**と呼び，$a \circledcirc 0_{\text{right}} = a$ のとき 0_{right} を**右単位元**と呼ぶが，本書では両者を区別せず単位元と呼ぶ．

このように，数学者は数の性質を抽象化し，集合とその集合に対する演算というものの見方をよく行う．プログラミングの言葉で言えば，複数のクラスに共通のインタフェースを定義するようなものである．

型 $\boldsymbol{a}$ を任意の型としたとき，型 $\boldsymbol{a} \to \boldsymbol{a}$ の関数もまたモノイドとなる．まず「何もしない」関数 id を次のように定義する．

$$\mathrm{id} = \lozenge \tag{272}$$

これはもちろん $\mathrm{id} = \backslash\, x \mapsto x$ の意味で，引数をそのまま返す関数である．

我々は関数を合成できる．そこで $\boldsymbol{a} \to \boldsymbol{a}$ 型の関数 f と関数 id の合成を考えると $\mathrm{id} \circ f = f \circ \mathrm{id} = f$ であり，また関数 $f, g, h :: \boldsymbol{a} \to \boldsymbol{a}$ について $(h \circ g) \circ f = h \circ (g \circ f)$ でるから，組み合わせ $(\boldsymbol{a} \to \boldsymbol{a}, \circ, \mathrm{id})$ はモノイドであることがわかる．

📍 **TK**

$(\mathbb{Z}, +, 0)$ はモノイドである．

同じ型を持つ関数を集めた関数の集合 $\mathbb{F}$ があるとする．任意の関数 $f \in \mathbb{F}$ について $\mathrm{id}\, f = f$ なる関数 id があり，かつ任意の関数 $f, g, h \in \mathbb{F}$ に対して $(h \circ g) \circ f) = h \circ (g \circ f)$ が成り立つとする．このとき組み合わせ $(\mathbb{F}, \circ, \mathrm{id})$ は**モノイド**であるという．

- 単位元の存在
- 結合律

4.1 直積と直和

5 Haskell の深い部分

5.1 関数・関手・アプリカティブ関手・モナド

5.1.1 関数と関手

📍 **TK** 関数と関手

関数は関手である．関手とはマップ演算子 $(\circledast)$ を持つ型クラスのことであった．関数が関手であるならば，関数もマップ演算子を持っていないといけない．そこで，関数のマップ演算子とは何かを考えてみる．

いま関数 f が $f :: \boldsymbol{r} \to \boldsymbol{q}$ という型を持っているとする．この式は $\to$ を二項演算子，すなわち 2 引数関数とみなせば $f :: (\to)\boldsymbol{rq}$ と等価である．全く形式的に，$((\to)\boldsymbol{r})$ なる型コンストラクタがあるとして

$$\boldsymbol{r} \to \boldsymbol{q} = ((\to)\boldsymbol{r})_{\boldsymbol{q}} \tag{273}$$

であると考えてみる．型 $\boldsymbol{q}$ から型コンストラクタ $((\to)\boldsymbol{r})$ によって型 $(\boldsymbol{r} \to \boldsymbol{q})$ が作られると考えるのだ．[116]

📍 **TK** マップ演算子の型.

関数のマップ演算子 $\left(\circledast_{((\to)\boldsymbol{r})}\right)$ と関数の合成演算子 $(\circ)$ は同じ型を持つ.

📍 **TK** なぜなら

$$\begin{aligned}(\circledast) &:: \mathsf{Functor} \supset \boldsymbol{f} \Rightarrow (\boldsymbol{a} \to \boldsymbol{b}) \to \boldsymbol{f}_{\boldsymbol{a}} \to \boldsymbol{f}_{\boldsymbol{b}} \\ \left(\circledast_{((\to)\boldsymbol{r})}\right) &:: (\boldsymbol{a} \to \boldsymbol{b}) \to ((\to)\boldsymbol{r})_{\boldsymbol{a}} \to ((\to)\boldsymbol{r})_{\boldsymbol{b}} \\ &= (\boldsymbol{a} \to \boldsymbol{b}) \to (\boldsymbol{r} \to \boldsymbol{a}) \to (\boldsymbol{r} \to \boldsymbol{b})\end{aligned} \tag{274}$$

$$(\circ) :: (\boldsymbol{a} \to \boldsymbol{b}) \to (\boldsymbol{t} \to \boldsymbol{a}) \to (\boldsymbol{t} \to \boldsymbol{b}) \tag{275}$$

幸い，我々は関数のマップ演算子の実装に関しては，型さえ守っていれば（そして 節 1.8.2 で述べる関手則さえ守っていれば）自由に選べる．そこで

$$\circledast_{((\to)\boldsymbol{r})} = \circ \tag{276}$$

としておこう．これが Haskell における関数のマップ演算子の定義である．

♣

関数はアプリカティブ関手でもある．アプリカティブ関手には，アプリカティブマップ演算子とピュア演算子が定義されるのであった．そこで，関数版のアプリカティブマップ演算子を $\boxtimes_{((\to)\boldsymbol{r})}$ とし，関数版のピュア演算子を $\langle x \rangle_{((\to)\boldsymbol{r})}$ と書くことにしよう．

ピュア演算子は $\boldsymbol{q} \to (\boldsymbol{r} \to \boldsymbol{q})$ 型を持たなければならない．従って関数版のピュア演算子は変数から関数を作るとも考えられる．我々は関数版のピュア演算子として

$$\langle x \rangle_{((\to)\boldsymbol{r})} = \backslash \square \mapsto x \tag{277}$$

を採用する．このラムダ式には特別に const と呼ばれているので，次のように書いても良い．[117]

$$\langle x \rangle_{((\to)\boldsymbol{r})} = \mathrm{const}\ x \tag{278}$$

♣

[116] Haskell では $((\to)\boldsymbol{r})$ を `((->)r)` と書く．
[117] Haskell では $\langle x \rangle_{((\to)\boldsymbol{r})}$ を `const x` と書く．

関数版のアプリカティブマップ演算子を $\boxast_{((\to)\boldsymbol{r})}$ とすると，その型は

$$\boxast_{((\to)\boldsymbol{r})} :: ((\to)\boldsymbol{r})_{\boldsymbol{a}\to\boldsymbol{b}} \to ((\to)\boldsymbol{r})_{\boldsymbol{a}} \to ((\to)\boldsymbol{r})_{\boldsymbol{b}} \tag{279}$$

つまり

$$\boxast_{((\to)\boldsymbol{r})} = (\boldsymbol{r} \to \boldsymbol{a} \to \boldsymbol{b}) \to (\boldsymbol{r} \to \boldsymbol{a}) \to (\boldsymbol{r} \to \boldsymbol{b}) \tag{280}$$

である．

我々は関数版アプリカティブマップ演算子として

$$g \boxast_{((\to)\boldsymbol{r})} f = \backslash\, x \mapsto gx(fx) \tag{281}$$

とする．これは，関数版のピュア演算子の定義と，一般マップ演算子と一般アプリカティブマップ演算子の関係 $f \circledast x_* = \langle f \rangle \boxast x_*$ から導かれる．すなわち

$$\begin{aligned} \langle g \rangle \boxast f &= \backslash\, x \mapsto \langle g \rangle x(fx) \\ &= \backslash\, x \mapsto (\backslash\, \square \mapsto g)x(fx) \\ &= \backslash\, x \mapsto g(fx) \\ &= g \circ f \end{aligned} \tag{282}$$

であるからである．

📍 **TK** マージ

$$f :: \boldsymbol{r} \to \boldsymbol{q} \tag{283}$$

$$f :: \langle \boldsymbol{r} \to \blacklozenge \rangle_{\boldsymbol{q}} \tag{284}$$

$$f :: \langle (\to)\boldsymbol{r}\blacklozenge \rangle_{\boldsymbol{q}} \tag{285}$$

```
f :: ((->) r) q
```

$$f_2 \circ f_1 = f_2 \circledast f_1 \tag{286}$$

$$\begin{aligned} \mathrm{id} \circ f = \mathrm{id} \circ f = f \\ (h \circ g) \circ f = h \circ (g \circ f) \end{aligned} \tag{287}$$

5.1.2 クライスリスター

関数 φ に作用するクライスリスター演算子 $\star$ を $\varphi^\star = (\varphi \heartsuit \lozenge)$ と定義する．クライスリスターを用いると，モナド則は次のように書き直せる．

$$\begin{aligned} \varphi^\star \langle x \rangle &= \varphi x \\ \langle \lozenge \rangle^\star x_* &= x_* \\ \psi^\star(\varphi^\star x_*) &= (\psi^\star(\varphi \lozenge))^\star x_* \end{aligned} \tag{288}$$

5.2 型と種

5.2.1 型の計算

5.2.2 種

📍 **TK** 種

$$\mathfrak{K} \rightarrow \mathfrak{Type} \tag{289}$$

See https://zenn.dev/mod_poppo/books/haskell-type-level-programming/viewer/types-and-kinds

5.3 マクロと Template Haskell

📍 **TK** マクロ

6 執筆ノート

- オブジェクト指向プログラミングとの関係

項目	内容	Haskell コード
Function application	$z = fx$	`z = f x`
List map	$z_{\mathrm{s}} = f * x_{\mathrm{s}}$	`zs = f `map` xs`
Maybe map	$z_? = f \circledast_? x_?$	`zm = f <$> xm`
Functor map	$z_* = f \circledast x_*$	`zm = f <$> xm`
Applicative map	$z_* = f_* \boxast x_*$	`zm = fm <*> xm`
Applicative map with 2 arguments	$z_* = g_* \boxast x_* \boxast y_*$	`zm = gm <*> xm <*> ym`
Applicative map with 2 arguments	$z_* = g \circledast x_* \boxast y_*$ $= [\![g x_* y_*]\!]$	`zm = gm <$> xm <*> ym`
Monadic function application	$z_* = \varphi x$	`zm = phi x`
Bind	$z_* = \varphi \heartsuit x_*$	`zm = phi =<< xm`
Double bind（右結合）	$z_* = \psi \heartsuit \varphi \heartsuit x_*$	`zm = psi =<< phi =<< xm`

表 6: メモ

$$
\begin{aligned}
&\texttt{class Eq} \supset \boldsymbol{a} \texttt{ where} \\
&\qquad (\equiv) :: \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool} \\
&\qquad (\not\equiv) :: \boldsymbol{a} \to \boldsymbol{a} \to \mathbf{Bool} \\
&\qquad (\equiv) = \neg(x \not\equiv y) \\
&\qquad (\not\equiv) = \neg(x \equiv y)
\end{aligned} \tag{290}
$$

$$
\texttt{data}\ \mathrm{TrafficLight} = \mathrm{Red} \curlyvee \mathrm{Yellow} \curlyvee \mathrm{Green} \tag{291}
$$

$$
\begin{aligned}
&\texttt{instance Eq} \supset \mathrm{TrafficLight} \texttt{ where} \\
&\qquad \mathrm{Red} \equiv \mathrm{Red} = \mathrm{True} \\
&\qquad \mathrm{Yellow} \equiv \mathrm{Yellow} = \mathrm{True} \\
&\qquad \mathrm{Green} \equiv \mathrm{Green} = \mathrm{True} \\
&\qquad \square \equiv \square = \mathrm{False}
\end{aligned} \tag{292}
$$

6.0.1 数学における圏と関手

Haskell における**モナド**（Monad）は、数学の圏論において定義される圏論的モナドとほぼ同じ概念である．数学的には，モナドはある圏 $\mathcal{C}$ 上の関手 $\mathfrak{T} : \mathcal{C} \rightsquigarrow \mathcal{C}$ と，ふたつの自然変換（単位 η と乗法 μ）から構成される三つ組 $(\mathfrak{T}, \eta, \mu)$ であり，以下の条件（モナド則）を満たす．

- 単位則: $\mu \bullet \mathfrak{T}\eta = \mu \bullet \eta\mathfrak{T} = \mathrm{id}_{\mathfrak{T}}$ 📍 **TK**
- 結合則: $\mu \bullet \mathfrak{T}\mu = \mu \bullet \mu\mathfrak{T}$ 📍 **TK**

Haskell では、型コンストラクタ m が $\mathcal{C}$ 上の関手 $\mathfrak{T}$ に対応し `return` や `pure` が η、`join` やバインド演算子 (>>=) が μ に対応する。つまり Haskell のモナドは、圏論における自己関手と 2 つの自然変換からなるモナド構造のプログラミング言語への実装例である。

より詳しくは：

- 圏論的モナドは「自己関手と 2 つの自然変換（η, μ）が単位則・結合則を満たす構造」。
- Haskell のモナドは圏論的モナドの公理を関数型プログラミングに持ち込んだもので、型と操作（関数・return・bind）の組としてモナド則を満たす。

まとめ: Haskell のモナドは、圏論におけるモナド（自己関手＋2 つの自然変換＋モナド則）のプログラム上での具体化に他ならない。

$$
\begin{aligned}
&\mu \mathbin{⁂} \mathrm{id} \ggg \mu = \alpha \ggg \mathrm{id} \mathbin{⁂} \mu \ggg \mu \\
&\eta \mathbin{⁂} \mathrm{id} \ggg \mu = \lambda \\
&\mathrm{id} \mathbin{⁂} \eta \ggg \mu = \rho
\end{aligned}
\tag{293}
$$

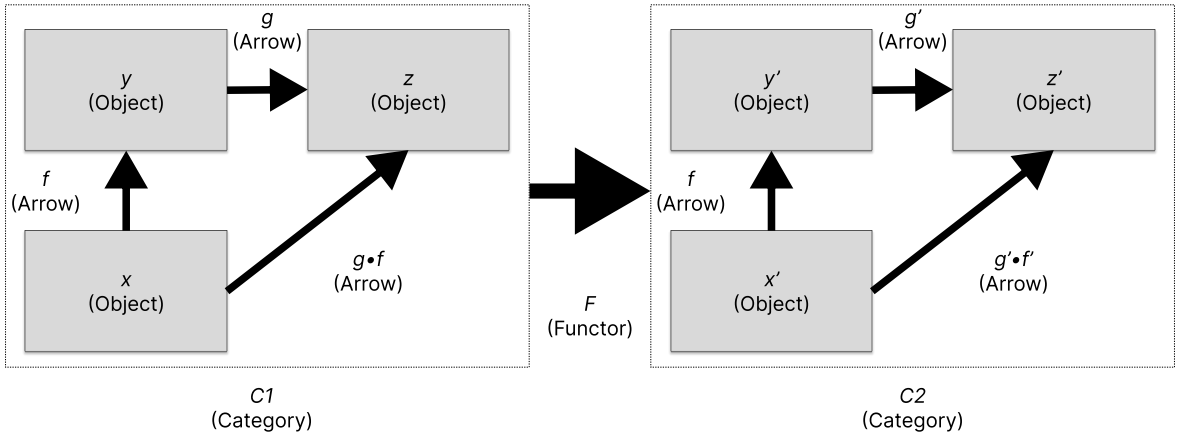

図 2: 数学における圏（Category）と関手（Functor）

6.0.2 Haskell における関手型クラスとそのインスタンス

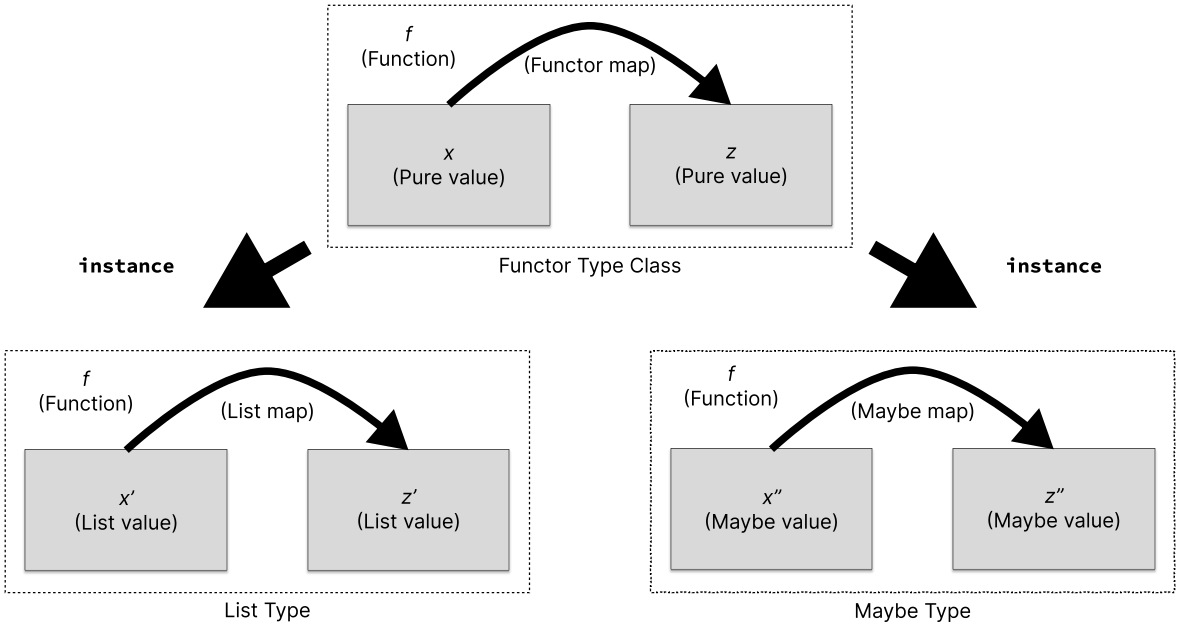

図 3: Haskell の関手型クラス（Functor Type Class）とそのインスタンス

6.0.3 Haskell におけるバインド演算子とモナド型クラスのインスタンス

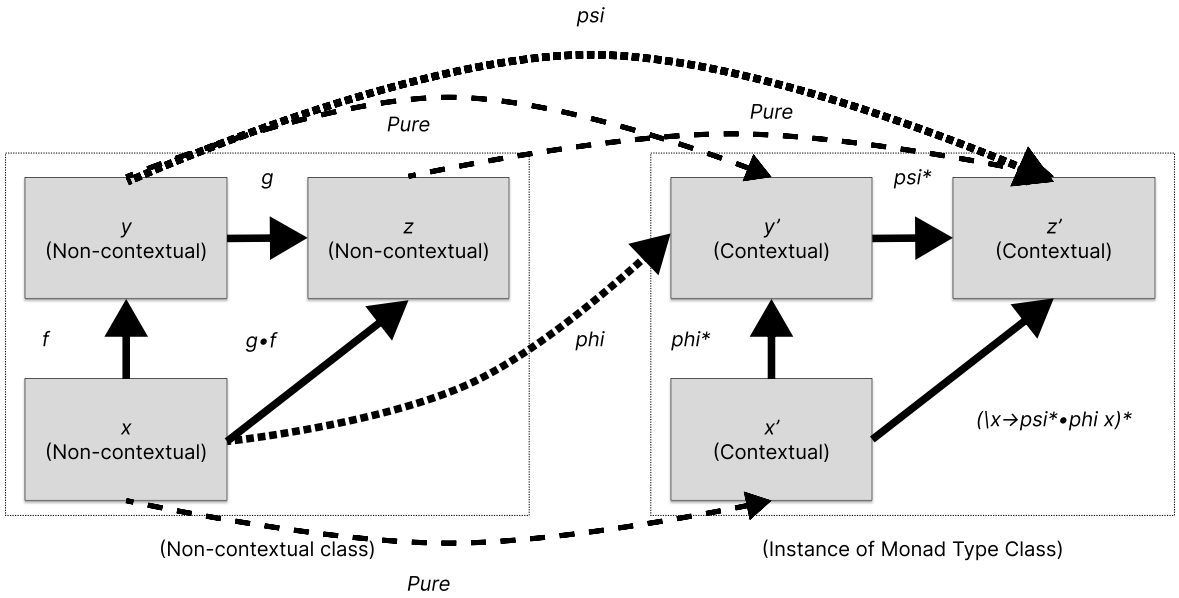

図 4: Haskell のバインド演算子とモナド型クラスのインスタンス

6.0.4 memo

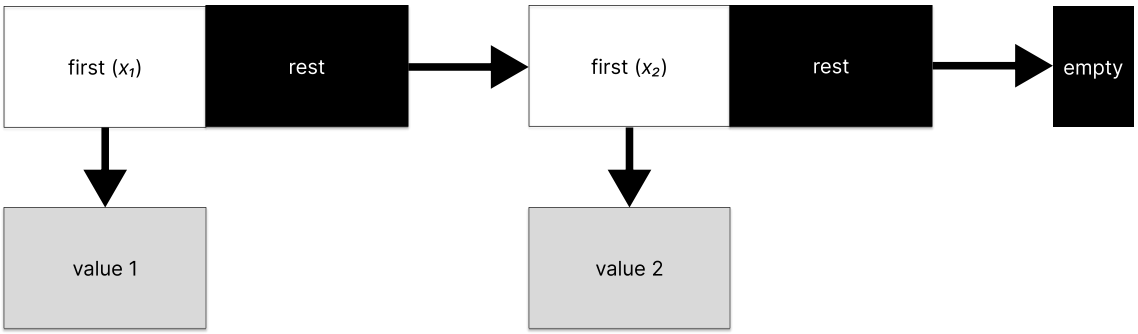

図 5: memo

6.0.5 関数型クラス

6.0.6 class and instance

$$\begin{aligned} &\forall \boldsymbol{m}\ \texttt{class}\ \mathsf{Monoid} \supset \boldsymbol{m}\ \texttt{where} \\ &\qquad \exists \emptyset_{\boldsymbol{m}} :: \boldsymbol{m} \\ &\qquad \exists (+_{\boldsymbol{m}}) :: \boldsymbol{m} \to \boldsymbol{m} \to \boldsymbol{m} \end{aligned} \tag{294}$$

$$\begin{aligned} &\texttt{instance}\ \mathsf{Monoid} \supset \mathbf{String}\ \texttt{where} \\ &\qquad \emptyset_{\boldsymbol{m}} = \texttt{""} \\ &\qquad (+_{\boldsymbol{m}}) = \divideontimes \end{aligned} \tag{295}$$

$$\begin{aligned} &\forall \boldsymbol{f}\ \texttt{class}\ \mathsf{Functor} \supset \boldsymbol{f}_{\blacklozenge}\ \texttt{where} \\ &\qquad \exists (\circledast) :: \boldsymbol{a} \to \boldsymbol{b} \to \boldsymbol{f}_{\boldsymbol{a}} \to \boldsymbol{f}_{\boldsymbol{b}} \end{aligned} \tag{296}$$

参考文献

[1] Apple,「Think Different」. [Online]. 入手先: https://designmanifestos.org/apple-think-different/

[2] Bartosz Milewski,「Category Theory for Programmers」. [Online]. 入手先: https://bartoszmilewski.com/

[3] Bungen Kato（加藤文元）, はじめての圏論. 講談社.

[4] TIOBE,「TIOBE Index」. [Online]. 入手先: https://www.tiobe.com/tiobe-index/

[5] Bryan O'Sullivan, *Real World Haskell: Code You Can Believe In*. O'Reilly Media.

[6] Graham Hutton, *Programming in Haskell (2nd edition)*. Cambridge University Press.

[7] Miran Lipovaca, *Learn You a Haskell for Great Good!*. O'Reilly Media, Inc.

[8] Richard Bird 著，山下伸夫訳, *Haskell* による関数プログラミングの思考法.

[9] Will Kurt 著，株式会社クイープ訳, 入門 *Haskell* プログラミング.

[10] C++ Reference,「Member access operators」. [Online]. 入手先: https://en.cppreference.com/w/cpp/language/operator_member_access.html

[11] legokichi,「Applicative スタイル f <> m1 <*> m2 を読み解く」. [Online]. 入手先: https://qiita.com/legokichi/items/2f2bb996ba4b5a2e4f07

[12] Mizunashi Mana,「data/newtype/type の使い方」. [Online]. 入手先: https://haskell.jp/blog/posts/2020/how-to-use-type-newtype-data.html

[13] Damepo,「Haskell での型レベルプログラミング」. [Online]. 入手先: https://zenn.dev/mod_poppo/books/haskell-type-level-programming

[14] funnycat,「モナド則を噛み砕く :基本から学ぶその本質と重要性」. [Online]. 入手先: https://zenn.dev/funnycat/articles/d92e16dfc59a49

[15] Kazu Yamamoto,「関手、Applicative、Monad の法則」. [Online]. 入手先: https://kazu-yamamoto.hatenablog.jp/entry/2019/04/11/111238

[16] Ryusei Yamaguchi,「圏論のモナドの法則から Haskell でよくみる形のモナド則を導き出す」. [Online]. 入手先: https://qiita.com/mandel59/items/87aebbd964ca82f74904

[17] 7shi,「Haskell 状態系モナド超入門」. [Online]. 入手先: https://qiita.com/7shi/items/2e9bff5d88302de1a9e9

[18] Minegishi Rei,「Haskell でファイルを読み書きする方法【Haskell 入門】」. [Online]. 入手先: https://minegishirei.hatenablog.com/entry/2023/12/15/093109

[19] School of Haskell,「Random numbers in Haskell」. [Online]. 入手先: https://www.schoolofhaskell.com/school/starting-with-haskell/libraries-and-frameworks/randoms

[20] Mizunashi Mana,「IO モナドと副作用」. [Online]. 入手先: https://haskell.jp/blog/posts/2020/io-monad-and-sideeffect.html

[21] Wikipedia,「Continuation-passing style」. [Online]. 入手先: https://en.wikipedia.org/wiki/Continuation-passing_style

[22] Kei Sugimoto,「サルでもわかる IO モナド①－副作用の除去」. [Online]. 入手先: https://hot-heart-cool-mind.hatenablog.com/entry/393131366.html

索引

IO 型 74
IO 文字列 72
IO モナド 23
ST アクション 67
ST モナド 67
UNIX コマンド 10
as パタン 42
cons 演算子 41
do 記法 69, 78
アキュムレータ 35
アクション 23, 39
アプリカティブ関手 58
アプリカティブ関手型クラス 74
アプリカティブスタイル 58, 59, 63
アプリカティブマップ演算子 57
アンカリー化 28
インスタンス 29
演算子 20
階乗 40
型 14, 26
型クラス 29, 53
型コンストラクタ 47
型シノニム 34, 59
型推論 27
型注釈 14, 26
型パラメタ 29
型変換 24
型変換演算子 30
カリー化 28
カレント継続 81
関手 53
関手型クラス 53, 54, 74
関手則 55
関数 9, 14
関数型プログラミング 7, 9
関数型プログラミング言語 14
関数合成演算子 11, 21
関数適用演算子 22
関数の合成則 21, 22
ガード 18
共有型 51
局所変数 16
疑似乱数 76
擬似乱数列 76
クイックソート 42
空 58, 64
空リスト 33
繰り返し演算子 33
クラステンプレート 32
クロージャ 16
クヌースの矢印 31
継続 14, 79
継続モナド 80
継続渡しスタイル 79
結合 34
圏 53, 54
圏論的関手 53, 54
高階関数 22
交換則 20
束縛 14
恒等関数 22
固定長整数型 26
コンセプト 51
コンテナ型 32, 53
コンテナ変数 33
合成 21
再帰 40
再帰適用 40
最大公約数 40
作用素 20
参照透過性 9, 14
参照渡し 68
射 54, 55
写像 27
集合 14
準同型性 58
条件式 18
条件分岐 9
スペシャルバージョン 17
制御構造 18, 63
整数型 14, 26
静的型付け 26
セクション 20
ゼロ除算例外 46
全称量化子 30
前置 20

前置演算子 20
早期リターン 81
総和演算子 35
大域ジャンプ 14
対象 54
多相型 29
多相的 29
畳み込み演算子 23, 35
多倍長整数型 26
タプル 28
単項演算子 20
単項マイナス 20, 21
単精度浮動小数点数型 27
短絡評価 18
遅延評価 9
逐次処理 78
逐次実行 9, 78
直接スタイル 79
中置 20
直和 60
適用 15
デフォルト引数 17
データ型 27, 59
定数 9
等差数列 32
トリプル 28
動的型付け 26
内部状態 14
二項演算子 20
入出力 9, 74
ノルム 37
ハイライト 9
破壊的代入 9, 14
ハスケル・カリー 28
バリアント型 51
場合分け 18
倍精度浮動小数点数型 27
バインド演算子 63, 74
パタンマッチ 17
引数 9, 15
左結合 15
左作用素 15
左バインド演算子 23, 63
ピュア演算子 58
浮動小数点数型 14, 26
部分適用 15
文脈 48, 63, 74
ブール型 26, 27
平坦化演算子 36
ヘテロジニアスな配列 32
ペア 28
ホモジニアスな配列 32
マップ演算子 23, 36
末尾再帰 43
末尾再帰最適化 43, 44
右結合 28
右畳み込み 36
右バインド演算子 65
無限リスト 9, 44
無名関数 16
文字型 26, 27
文字列 24, 34
戻り値 9
モナド 9, 23, 63, 64, 74
モナド型クラス 74
ユーザ入力 14
ユニット 28, 29
ユニット型 28
要素 32
ラムダ式 9, 16
リスト 32
リスト値コンストラクタ 32
リスト型コンストラクタ 32
リスト内包表記 33
リスト変数 32
リテラル 14
ループ 9, 14
例外 14
レキシカルクロージャ 23